



International Software Testing Qualifications Board

Certified Tester Game Testing (CT-GaMe) Syllabus

Version 1.0.1

한글 Korean v1.0

(사) KSTQB

Korean Software Testing Qualifications Board

Copyright Notice

Copyright Notice © International Software Testing Qualifications Board (hereinafter called ISTQB®). ISTQB® is a registered trademark of the International Software Testing Qualifications Board.

All rights reserved.

Copyright © 2022, the authors Andrey Konushin (chair), Alexander Alexandrov, Evgeny Glushkin, Dmitry Karasev, Elena Karaseva, Elizaveta Kruchinina, Vadim Lukovaty, Dmitry Melishev, Maxim Nikolaev, Alexander Prokhorov, Anton Savvateev, Ayrat Sayfullov, Pavel Sharikov, Kirill Shevelev, Artyom Stukalov, Nikita Sysuev, Tatiana Tepaeva, Alexander Torgovkin, Margarita Trofimova, Yaroslav Vereshchagin, Svetlana Yushina, Lyubov Zhuravleva.

The authors hereby transfer the copyright to the ISTQB®. The authors (as current copyright holders) and ISTQB® (as the future copyright holder) have agreed to the following conditions of use:

Extracts, for non-commercial use, from this document may be copied if the source is acknowledged. Any Accredited Training Provider may use this syllabus as the basis for a training course if the authors and the ISTQB® are acknowledged as the source and copyright owners of the syllabus and provided that any advertisement of such a training course may mention the syllabus only after official Accreditation of the training materials has been received from an ISTQB®-recognized Member Board.

Any individual or group of individuals may use this syllabus as the basis for articles and books, if the authors and the ISTQB® are acknowledged as the source and copyright owners of the syllabus.

Any other use of this syllabus is prohibited without first obtaining the approval in writing of the ISTQB®.

Any ISTQB®-recognized Member Board may translate this syllabus provided they reproduce the abovementioned Copyright Notice in the translated version of the syllabus.

목차

0. 실러버스 소개	7
1. 게임 테스트의 특수성 - 75 분	12
1.1 게임 테스트 개요	13
1.1.1 게임 테스트의 특징	13
1.1.2 게임 제품 리스크	13
1.1.3 게임 테스트와 관련된 결함	13
1.1.4 테스트가 비디오게임의 제품 리스크를 완화하는 방법	15
1.1.5 테스트와 “플레이”의 차이	16
1.2 게임 개발팀 내 일반적인 역할	17
1.3 게임 소프트웨어 개발 수명 주기 전반에 걸친 테스트 활동	18
2. 게임 메커니즘 테스트 - 180 분 (K3)	20
2.1 게임 메커니즘	21
2.1.1 게임 메커니즘 유형	21
2.1.2 게임플레이 메커니즘 테스트와 비-게임플레이 메커니즘 테스트의 차이	22
2.1.3 핵심 메커니즘 테스트와 메타 메커니즘 테스트의 차이	22
2.1.4 클라이언트, 서버, 클라이언트-서버 메커니즘 테스트의 차이	23
2.1.5 게임 메커니즘 결함 및 발생 원인 예	24
2.2 게임 메커니즘 테스트 접근법	25
2.2.1 게임 소프트웨어 개발 수명주기 전반에서 게임 메커니즘 테스트의 절차 및 접근법	25
2.2.2 게임 메커니즘 테스트의 중요성	26
2.2.3 게임 메커니즘 리뷰의 중요성	27
2.2.4 세션 재개 후 및 사용자가 비활성 상태일 때의 비디오게임 상태 테스트	27

3. 그래픽 테스트 - 165 분 (K3)	31
3.1 게임 그래픽의 원리와 개념	32
3.1.1 게임 그래픽 콘텐츠의 특징	32
3.1.2 그래픽 콘텐츠 결함 유형	35
3.2 게임의 그래픽 테스트 접근법	40
3.2.1 아티스틱(Artistic) 테스트	40
3.2.2 기술적 테스트	41
3.2.3 게임플레이 테스트	43
3.3 그래픽 테스트 실행	43
3.3.1 오브젝트 개발의 여러 단계에서의 그래픽 테스트 실행	43
3.3.2 역사적 사실의 정확도를 위한 그래픽 테스트	45
3.4 그래픽 테스트 지원 도구	46
4. 사운드 테스트 - 190 분 (K3)	48
4.1 게임 사운드 콘텐츠의 특징	49
4.1.1 사운드 유형	49
4.1.2 음향 효과와 기술	51
4.1.3 사운드 영역	51
4.2 사운드 콘텐츠의 결함 유형	52
4.3 게임 사운드 콘텐츠 테스트 접근법	54
4.3.1 콘텐츠 청음 테스트	54
4.3.2 음악과 게임 사운드의 믹스 테스트	55
4.3.3 음악 작곡 테스트	55
4.4 사운드 테스트 실행	56
4.4.1 게임 소프트웨어 개발 수명 주기에서 오디오-음악 콘텐츠 테스트 레벨	56
4.4.2 게임에 사운드 통합	56
4.4.3 참여자의 책임	56

4.4.4 사운드 오브젝트 테스트의 테스트 활동 수행 절차 및 접근법	57
4.5 사운드 테스트를 위한 도구 지원	58
5. 게임 레벨 테스트 - 65 분 (K2).....	60
5.1 게임 레벨 설계 원칙과 개념	61
5.1.1 "레벨"이라는 용어와 게임 프로젝트의 장르에 따른 특수성.....	61
5.1.2 레벨 디자인 결함 유형 이해	62
5.2 게임 레벨 테스트의 단계 및 실행.....	64
5.2.1 게임 레벨 설계 및 테스트의 기본 단계.....	64
5.2.2 참여자의 책임 영역	67
5.3 게임 레벨 테스트를 위한 도구 지원	67
6. 게임 컨트롤러 테스트 - 95 분 (K2)	69
6.1 게임 컨트롤러의 원리와 개념	70
6.1.1 게임 컨트롤러의 유형.....	70
6.1.2 게임 컨트롤러 특성과 관련된 결함.....	71
6.2 게임 컨트롤러 테스트 접근법	72
6.3 게임 컨트롤러 테스트를 위한 도구 지원.....	73
7. 현지화 테스트 - 155 분 (K3)	75
7.1 현지화 테스트의 원칙 및 개념.....	76
7.1.1. 현지화 및 국제화.....	76
7.1.2 게임과 애플리케이션 소프트웨어 현지화의 차이	78
7.1.3 현지화 테스트 단계	78
7.2 현지화 결함의 유형과 그 원인.....	82
7.2.1 게임의 현지화 결함의 가능한 원인.....	82
7.2.2. 현지화 결함 및 리스크	82
7.3 현지화 테스트 접근법 및 실행.....	84
7.3.1 전체 현지화 테스트와 부분 현지화 테스트의 차이.....	84

7.3.2 게임 소프트웨어 개발 수명 주기에서의 현지화 테스트 프로세스와 접근법	84
7.3.3 현지화 테스트 유형	86
7.3.4 참여자의 책임 영역	87
7.4 현지화 테스트를 위한 도구 지원	87
8. References	89
8.1 Standards	89
8.2 ISTQB documents	89
8.3 Books	89
8.4 Links (Web/Internet)	90

0. 실러버스 소개

0.1 이 문서의 목적

이 실러버스는 ISTQB® 게임 테스트(Certified Tester Game Testing, 약어: CT-GaMe) 자격을 위한 기반이 됩니다. ISTQB®는 다음과 같이 이 실러버스를 제공합니다:

0. 각 회원국(National Board)이 그들의 현지 언어로 번역하고 교육 제공자를 인증할 수 있도록 합니다. 국가별 위원회는 실러버스를 그들의 특정 언어 요구에 맞게 조정하고 현지 출판물에 맞게 참조를 수정할 수 있습니다.
1. 시험 기관이 각 실러버스의 학습 목표(Learning objectives)에 맞춰 현지 언어로 시험 문제를 도출할 수 있도록 합니다.
2. 교육 기관이 교재를 제작하고 적절한 교육 방법을 결정할 수 있도록 합니다.
3. 자격시험 응시자가 시험을 준비할 수 있도록 합니다 (실러버스의 일부로 또는 독립적으로).
4. 소프트웨어 및 시스템 엔지니어링 국제 커뮤니티가 소프트웨어 및 시스템 테스트 전문성을 높이고 서적과 논문 작성을 위한 기반이 되도록 합니다.

ISTQB®는 사전에 서면승인을 받는 조건으로, 타 기관이 해당 실러버스를 기타 목적으로 사용할 수 있도록 할 수 있습니다.

0.2 게임 테스트 인증 (CT-GaMe)

게임 테스트 인증 시험은 게임 테스트에 대한 지식을 넓히고자 하거나 게임 테스트 분야의 전문 경력을 시작하고자 하는 사람을 대상으로 합니다. 이 인증 시험은 또한 게임 소프트웨어 엔지니어링에 종사하며, 게임 테스트에 대한 이해를 높이고자 하는 분들도 대상으로 합니다.

이 실러버스에서는 게임 테스트의 다음 주요 측면을 다룹니다:

- 기술적 측면,
- 방법론적 측면,
- 조직적 측면.

0.3 테스터를 위한 커리어패스 (Career Path)

Foundation Level (CTFL) 기초 레벨을 기반으로, 게임 테스트는 전문 테스터의 커리어패스를 구축할 수 있습니다. 게임 테스트 자격증을 취득한 사람은 Foundation Level (CTFL)에서 획득한 테스트에 대한 전반적인 이해를 확장해 게임 프로젝트에서 효과적으로 일할 수 있게 됩니다.

게임 테스트 인증서를 가진 사람은 CT-GaMe 약어를 사용할 수 있습니다.

ISTQB®의 커리어 패스에 대한 최신 개요는 [ISTQB-Web]을 방문해 확인 가능합니다.

0.4 비즈니스 성과

이 섹션에서는 게임 테스트 자격증을 취득한 응시자가 달성할 것으로 기대되는 비즈니스 성과를 설명합니다.

GaMe-1 비디오 게임과 게임 소프트웨어 테스트의 기본 개념을 설명할 수 있다

GaMe-2 이해관계자의 요구사항과 기대에 따른 리스크, 목표 및 게임 소프트웨어 요구사항을 결정할 수 있다

GaMe-3 기본 게임 소프트웨어 테스트를 개념적으로 설계, 구현 및 실행할 수 있다

GaMe-4 게임 소프트웨어 테스트에 대한 방법론과 그 목적을 인식할 수 있다

GaMe-5 게임 테스트를 지원하는 도구를 파악할 수 있다

GaMe-6 테스트 활동이 소프트웨어 개발 수명주기와 어떻게 연계되는지 결정하고, 비디오 게임 개발 및 출판 비용을 줄일 수 있다

0.5 출제 가능한 학습 목표 및 인지 수준

학습 목표(LOs)는 비즈니스 목표 달성을 지원하며 게임 테스트 자격증을 위한 시험을 출제하는 데에도 사용됩니다. 각 학습 목표는 인지 과정(지식 수준, K-level)을 참조합니다.

K-level 또는 인지 수준은 Bloom 의 개정된 분류학에서 학습 목표를 분류하는 데 사용됩니다 [Anderson01]. ISTQB®는 이 분류학을 사용해 자체 실러버스 시험을 설계합니다(자세한 내용은 부록 A 를 참조하세요).

이 실러버스는 세 가지 다른 K-level(K1 에서 K3 까지)을 참고합니다:

K-level	키워드	설명
1	기억하다	응시자는 용어나 개념을 기억하거나 인지해야 한다.
2	이해하다	응시자는 문항 주제 관련 구문에 대한 설명을 선택해야 한다.
3	적용하다	응시자는 개념이나 기법의 올바른 적용을 선택해 주어진 상황에 적용해야 한다.

일반적으로, 이 교육 프로그램의 모든 부분은 K1 레벨의 학습 목표를 포함합니다. 다시 말해, 자격증 취득을 위해 응시자는 용어나 개념을 인지하고, 기억하며, 재생산할 수 있어야 합니다. K2 및 K3 레벨의 학습 목표는 이를 포함하는 장(chapters)의 앞부분에 표시됩니다.

0.6 게임 테스트(CT-GaMe) 자격 시험

게임 테스트 자격 시험은 이 실러버스를 기반으로 합니다. 시험 문제에 대한 정답은 이 실러버스에서 하나 이상의 섹션에 기반한 자료를 사용할 수 있습니다. 서문 및 부록을 제외한 실러버스의 모든 섹션은 시험문제 출제 대상입니다. 표준 및 서적은 참고 자료로 포함되어 있지만, 실러버스에 해당 표준이나 서적에서 가져와 요약해 사용된 내용 이상은 시험 대상이 아닙니다.

게임 테스트 자격증의 시험 구조 및 규칙에 대해서는 [ISTQB_EXAM_S&R]을 참조하세요.

ISTQB® Foundation Level [ISTQB_FL_SYL] 자격증을 게임 테스트 자격시험 응시 전에 취득해야 합니다. 더불어, 다음과 같은 사항들이 또한 응시자에게 강력히 권장됩니다:

- 게임 소프트웨어 개발이나 테스트에 대한 최소한의 경험 (예, 게임 소프트웨어 사용자 인 수 테스터 1년 경험 등)
- 각국 ISTQB 인증 회원 보드들 중 하나에 의해 ISTQB® 기준으로 인증된 교육 과정 수강.

0.7 인증

ISTQB® 회원 보드는 이 실러버스를 준수하는 교육 기관을 인증할 수 있습니다.

교육 기관들은 인증을 수행하는 회원 보드 또는 기관으로부터 인증 가이드라인을 획득해야 합니다. 인증된 교육 과정은 이 실러버스에 부합하는 것으로 인정받으며, 과정의 일부로서 ISTQB® 시험을 치를 수 있는 권한을 갖습니다.

이 실러버스에 대한 인증 가이드라인은 ISTQB® 프로세스 관리 및 규정 준수 워킹 그룹에서 발행한 일반 인증 가이드라인을 따릅니다.

0.8 Handling of Standards

게임 테스트 실러버스에는 표준들(예: IEEE, ISO 등)이 참조됩니다. 이러한 참조의 목적은 프레임워크를 제공하거나(품질 특성에 관한 ISO 25010 참조 등), 또는 독자가 원하는 경우 추가 정보의 출처를 제공하는 것입니다. 실러버스가 표준 문서를 참조로 사용하고 있다는 점을 유의해 주기 바랍니다. 표준 문서들은 시험 목적으로 의도된 것은 아닙니다. 표준에 대한 자세한 정보는 8. 참조 문헌 섹션을 참조하세요.

0.9 최신 상태 유지

소프트웨어 산업은 빠르게 변화합니다. 이러한 변화에 대처하고 이해관계자들에게 관련성 깊은 최신 정보를 제공하기 위해, ISTQB® 워킹 그룹들은 [ISTQB-Web]에 표준의 변경, 산업 내의 새로운 발생

사항 등에 대한 참조 문서 링크를 만들었습니다.

이 정보는 이 실러버스 관련 시험문제로 출제되지 않습니다.

0.10 상세 수준

이 실러버스의 상세 수준은 국제적으로 일관된 과정과 시험을 가능하게 합니다. 이 목표를 달성하기 위해, 실러버스는 다음과 같이 구성됩니다:

- 게임 테스트 자격증의 의도를 설명하는 일반 교육 목표,
- 학생들이 기억할 수 있어야 하는 용어 목록,
- 달성해야 할 인지 학습 결과를 설명하는 각 지식 영역에 대한 학습 목표,
- 인정된 문헌이나 표준과 같은 출처에 대한 참조를 포함한 주요 개념의 설명.

실러버스 내용은 게임 테스트의 전체 지식 영역을 설명하는 것이 아니라, 게임 테스트 실러버스에서 다룰 상세 수준을 반영합니다. 대부분의 게임 프로젝트에 적용될 수 있는 테스트 영역과 기법에 초점을 맞춥니다.

0.11 실러버스의 구성

스페셜리스트 게임 테스트 실러버스는 게임 테스트 전문가로서 갖춰야 할 필수 지식을 다루는 다음 일곱(7)개의 장(Chapter)로 구성되어 있습니다.

각 장의 최상위 제목은 해당 장에 대한 최소 시간을 명시합니다; 장 하위 수준에서의 시간은 제공되지 않습니다. 인증된 교육 과정의 경우, 실러버스는 일곱 개 장에 걸쳐 총 15 시간 25 분 이상의 수업 시간이 요구됩니다:

- 제1장: 75분 게임 테스트의 특수성
 - 테스터는 게임 소프트웨어 테스트 관련 기본 원칙, 게임 테스트가 필요한 이유, 테스트 목표 및 게임 테스트와 게임 플레이의 차이점을 학습한다.
 - 테스터는 게임 테스트 과정, 주요 활동 및 작업 결과물에 대해 이해한다.
- 제2장: 180분 게임 역학 테스트
 - 테스터는 다양한 게임 역학과 그 테스트 접근법을 구분하는 법을 학습한다.
 - 테스터는 게임 역학 테스트에 사용되는 동적 및 정적 기법을 학습한다.
- 제3장: 165분 그래픽 테스트
 - 테스터는 게임 제품 그래픽 콘텐츠의 특징과 유형, 그래픽 테스트 실행의 모범 사례 및 도구 지원에 대해 학습한다.
 - 테스터는 관련된 사람들의 책임 영역에 대해 배운다.

- 제4장: 190분 사운드 테스트
 - 테스터는 게임 제품 사운드 콘텐츠의 유형과 기술, 사운드 테스트 실행의 모범 사례 및 도구 지원에 대해 학습한다.
 - 테스터는 관련된 사람들의 책임 영역에 대해 배운다.
- 제5장: 65분 게임 레벨 테스트
 - 테스터는 게임 장르와 도구 지원에서의 레벨 설계와 테스트 접근법의 의존성을 학습한다.
 - 테스터는 관련된 사람들의 책임 영역에 대해 배운다.
- 제6장: 95분 게임 컨트롤러 테스트
 - 테스터는 게임 컨트롤러의 유형, 관련 결함 및 그 가능 원인에 대해 학습한다.
 - 테스터는 컨트롤러를 테스트하는 접근법과 도구 지원에 대해 학습한다.
- 제7장: 155분 현지화 테스트
 - 테스터는 현지화와 국제화의 기초와 차이점, 그 위험성, 테스트 접근법 및 도구 지원에 대해 학습한다.
 - 테스터는 관련된 사람들의 책임 영역에 대해 배운다.

1. 게임 테스트의 특수성 - 75 분

테스팅 용어

기능 테스트(Functional testing), 플레이 테스트(playtest)

비디오게임 전용 용어

3D 모델(3D model), 컨셉 단계(concept stage), 비디오게임 디자이너(video game designer), 멀티플랫폼(multiplatform), 후반-작업 단계(post-production stage), 사전-제작 단계(pre-production stage), 개발 단계(production stage), 비디오게임(video game)

1 장 학습 목표

1.1 게임 테스트 개요

GaMe-1.1.1 (K1) 게임 테스트의 목적 및 세부 사항을 인식할 수 있다

GaMe-1.1.2 (K2) 게임 소프트웨어의 제품 리스크 예를 제시할 수 있다

GaMe-1.1.3 (K2) 게임 테스트와 관련된 구체적인 결함의 예를 제시할 수 있다

GaMe-1.1.4 (K2) 게임 테스트의 리스크를 완화할 수 있는 방법을 요약할 수 있다

GaMe-1.1.5 (K2) 게임 테스트 활동과 게임플레이를 비교할 수 있다

1.2 게임 개발팀 내 일반적인 역할

GaMe-1.2.1 (K1) 게임 개발팀 내 구체적인 역할 및 작업(tasks)을 인식할 수 있다

1.3 게임 소프트웨어 개발 수명 주기 전반에 걸친 테스트 활동

GaMe-1.3.1 (K1) 게임 소프트웨어 개발 수명 주기 전반에 걸친 테스트 활동을 상기할 수 있다

1.1 게임 테스트 개요

1.1.1 게임 테스트의 특징

소프트웨어 테스트에서는 다양한 전문 분야를 구별하는 것이 일반적이다. 일부 테스터는 애플리케이션의 성능 테스트에, 또 일부는 모바일 애플리케이션 테스트에 중점을 두어 보안 시스템의 취약점을 찾거나 컴퓨터 게임에서 결함을 찾기도 한다.

게임은 다양한 장르와 싱글/멀티 플레이어 모드가 제공되지만, 게임 테스트는 이러한 특성과는 무관하다고 할 수 있다. 이 실라버스의 2.1절은 게임 메커니즘(game mechanics)의 기능 및 비기능 테스트 방법을 자세히 설명하고 있다.

이 실라버스에서 "게임", "컴퓨터 게임", "비디오 게임"은 모두 같은 의미로 사용된다. 이는 실라버스 내용을 보다 쉽게 이해할 수 있도록 하기 위함이다. 또한, 이 실라버스에서 언급하는 게임은 소프트웨어 게임을 의미하며, 스포츠 게임이나 도박과는 혼동되어서는 안 된다.

1.1.2 게임 제품 리스크

비디오게임 산업이 폭발적으로 성장하면서 게임은 다양한 플랫폼에 배포되기 시작했다. 게임은 점차 커지고 복잡해졌으며 기술적인 요구사항도 높아지고 있다. 이와 동시에 게임 사용자도 늘어나고 품질에 대한 요구도 크게 증가하면서 더욱 정교하고 까다로워지고 있다.

일반적인 소프트웨어의 프로젝트 및 제품 리스크는 비디오 게임과 게임 개발에도 적용된다. 따라서 게임은 다른 소프트웨어와 마찬가지로 테스트가 필요하다. 소프트웨어 중 게임에서만 발생하는 리스크도 있으며 이들은 더 많은 주의를 요한다. 다음은 이런 리스크의 일부를 나열한 것이다.

- 부정행위로 인한 부당한 이득 획득
- 플레이어의 주관적인 의견에 따라 시장의 성공이 좌우되는 것
- 멀티 플랫폼 호환 문제
- 게임 메커니즘의 효과를 예측하는 것의 복잡성 (2.1 절 참조)

테스터는 다음과 같은 영역에서 결함을 찾을 수 있다.

- 개발 초기 게임 소프트웨어 아키텍처에서
- 사운드 디자인에서
- 이미 출시 준비가 된 비디오게임의 텍스트에서

1.1.3 게임 테스트와 관련된 결함

비디오게임에서 자주 발생하는 결함에는 다음과 같은 것이 존재한다:

- 그래픽과 애니메이션 문제
- 비디오게임 세계의 물리 법칙 위반
- 비디오게임 인공지능 결함

- 비디오게임 플롯(plot) 구성의 부정확성
- 일부 인터페이스 요소의 잘못된 동작 (예: 공중에서 나타나는 게임 캐릭터, 적을 공격하지 않는 사례, 공중에 떠 있는 자동차)

이러한 결함은 상대적으로 쉽게 발견할 수 있지만, 결함을 재현하는 정확한 순서를 식별하는 것이 어려울 수 있다. 이러한 복잡성으로 인해 게임 테스트는 어렵다. 우수한 테스터는 의도된 게임 시나리오를 기반으로 테스트를 설계하고, 명세를 충족하는지 확인한 다음, 설계된 시나리오와의 차이(deviations)를 분석하고, 그런 차이로 나타나는 결과를 보고해야 한다.

네거티브 테스트(Negative testing)

게임 테스트는 다른 소프트웨어와 마찬가지로 명백한 결함 뿐만 아니라 사용자의 예외적인 행동의 결과로 나타나는 암묵적인 결함까지 찾는 것이 중요하다. 예를 들어, 상대방과 싸워서 잠긴 문의 열쇠를 얻지 않고, 사용자가 레벨 전체를 돌아다니며 상자를 수집해서 사다리를 만들어서 울타리를 넘어 게임이 의도한 어려운 싸움을 피할 수 있다.

게임에 사용되는 건물의 3D 모델과 같은 오브젝트 간의 상호작용이 잘못 설정된 경우, 결함이 발생할 수 있다. 예를 들어, 플레이하는 캐릭터가 벽을 통과하는 결함이 생길 수 있는데, 이런 결함은 전반적인 사용자 경험을 해치거나 의도하지 않은 이점을 제공할 수 있다. 이러한 결함은 멀티플레이어 게임에서 특히 중요하다. 벽 뒤에 숨어서 상대방은 자신을 보지 못하지만, 본인은 상대방을 쏠 수도 있다. 이와 같은 리스크 때문에 게임 프로젝트에서 네거티브 테스트를 수행하는 것은 중요하다. 노력 없이 쉽게 승리하기 위해서 개발자가 의도한 대로 플레이 하지 않고, 시스템을 "브레이크(break)" 하거나 우회하려고 시도하는 플레이어가 있을 수 있다. 그들이 무조건 제3의 소프트웨어나 특수한 명령어를 사용해야 하는 것도 아니다. "적절한" 결함을 찾는 것으로 충분할 수 있다. 이러한 플레이어는 의도적으로 게임 논리에서 결함을 찾아, 무한한 자원이나 강력한 무기 등 개인적인 이익을 위해 결함을 이용한다. 수익을 창출해야 하는 게임일 경우, 이것은 게임 개발사의 수익 감소로 이어질 수 있다.

플레이어 의견에 대한 의존성(dependence)

게임 소프트웨어는 다른 소프트웨어에 비해 최종 플레이어의 주관적이고 감정적인 판단에 훨씬 더 영향을 받는다. 비즈니스 문제를 해결하도록 설계된 소프트웨어는 기능의 지원 여부가 가장 중요한 요소이다. 그러나 게임 소프트웨어의 경우, 사용자의 관심과 게임의 첫인상(impressions of the gaming session)이 가장 중요하다. 플레이어는 일부 기능 결함은 눈감아 넘길 수도 있지만, 게임이 지루하거나 단조로울 경우, 아니면 오래된 그래픽을 사용할 경우 게임 플레이 자체를 하지 않을 수 있다. 또한, 많은 비디오게임 플레이어는 게임 평론가, 리뷰어, 기타 오피니언 리더(opinion leader)의 피드백을 기반으로 제품을 구매 또는 플레이하게 된다.

멀티플랫폼

게임 소프트웨어의 중요한 특징은 여러 플랫폼에서 많은 사용자가 흔히 사용한다는 점이다. 게임 스튜디오 및 퍼블리셔는 가능한 한 많은 사용자를 확보하기 위해 다양한 개인용 컴퓨터(PC), 웹, 모바일 기기, 콘솔 등을 포함한 여러 플랫폼에 게임을 배포한다.

모든 업데이트는 가용한 모든 플랫폼에서 테스트되어야 한다. 이는 심각한 리스크가 되며 필요한 테스트 시간도 크게 늘어나게 한다.

게임이 컴퓨터(medium-sized computer)에서는 잘 실행되지만, 최신 콘솔에서는 다양한 결함이 발생할 수 있다. 기술 간 비효율적인 커뮤니케이션이나 원래 의도한 플랫폼이 아닌 (아웃소싱 된) 제3의 조직에서 개발한 다른 플랫폼으로 이식하기 위한 요구사항의 불완전성 등으로 결함이 발생할 수도 있다.

이런 리스크를 완화하기 위해서는 게임 소프트웨어 개발과 테스트, 다양한 하드웨어 구성에서의 게임 기능과 성능 테스트, 또 각 플랫폼 고유 특성을 고려한 테스트 수행 등에 더 많은 시간을 할애할 필요가 있다.

비디오게임을 게임 콘솔 상에서 테스트하는 것도 고려해야 하는 특별한 점이다. 게임 콘솔은 게임만을 위해 설계된 기기로 모바일 기기나 컴퓨터와 달리 다른 소프트웨어는 아무것도 가지고 있지 않다.

비디오게임을 테스트하기 위해 블랙박스 테스트 기법을 사용하면 플랫폼 간 차이는 거의 없다. 그러나 게임 콘솔 제조 업체가 비디오게임 출시 전 준수해야 할 고유한 요구사항을 가지고 있을 수 있다. 이러한 요구사항은 기밀 유지 계약에 따라 개발자와 퍼블리셔에게 제공되는 기밀문서이다. 각 요구사항 체크리스트는 여러 범주로 구성되며, 콘솔 제조 업체에서 거부당하지 않으려면 게임은 그것을 준수해야 한다 [URL1], [URL2].

따라서 콘솔용 비디오게임을 테스트하는 사람은 기본적인 소프트웨어 테스트 방법 외에도 이러한 요구사항을 준수하는지 테스트해야 한다.

결함을 식별하고 분석해서 그 원인을 파악하기 위해 특수한 콘솔을 사용해야 할 수도 있다. 이 콘솔은 일반 콘솔과 유사하지만, 게임 개발과 테스트에 도움이 되는 추가 모드를 제공한다. 승인 개발자와 테스터가 이 콘솔을 사이트에 등록하고 활성화하면 추가 모드를 사용할 수 있다.

1.1.4 테스트가 비디오게임의 제품 리스크를 완화하는 방법

비디오게임, 특히 오픈-월드(open-world) 게임은 너무 복잡해서 게임 오브젝트, 이벤트, 요소 등의 모든 조합을 테스트하는 것이 불가능하다. 전체 소프트웨어 개발 수명 주기 동안 각 조합을 한 번씩만 테스트하는 것도 많은 시간과 노력이 필요하며, 게임이 업데이트되면 수정 사항에 대한 확인 테스트를 수행하는 것도 추가적인 노력이 필요하다.

자원을 효율적으로 할당하고 발생 가능성 및 게임에 미치는 영향을 기반으로 결함을 평가하는 데 리스크 관리가 사용된다.

각 테스트 케이스에 대해 테스트 우선순위를 결정하기 위해 두 가지 질문이 필요하다:

1. 플레이어가 여기에서 결함을 찾을 가능성은 얼마나 될까?
2. 결함을 식별하면 플레이어와 게임 개발사(the company that owns the game)에 어떤 영향을 미칠까?

예를 들어, 게임이 순차적인 구조를 가지고 사용자가 콘텐츠에 단계적으로 접근하는 경우, 첫 번째 게임 세션 동안 식별되는 결함은 아마 50번째 세션 때 발생하는 결함보다 수정의 우선순위가 높을 (그렇다고 심각도가 더 높지는 않을 수 있음) 것이다. 충성도가 높은 사용자가 이미 게임에 많은 시간을 보냈다면 나타난 결함을 더 침착하게 받아들일 가능성이 높으며, 사용자의 전반적인 게임 인상에 큰 영향을 미치지 않을 것이다. 이러한 결함이 튜토리얼 중 모든 새로운 플레이어에게 영향을 미친다면 전혀 다른 문제가 된다. 이 경우 많은 플레이어가 게임을 떠날 수 있다.

테스팅을 통해 리스크를 완화하려면 게임에서 개별 결함을 찾는 것보다 결함 그룹을 식별하는 것이 중요하다. 이를 통해 개발자는 게임 코드 또는 아키텍처 문제를 동시에 해결하고, 최종 제품의 품질을 향상시킬 수 있다.

결함 그룹은 중요 영역에 있는 결함의 집합이다. 게임 제품에는 그래픽, 사운드, 현지화, 클라이언트-서버 상호작용, 하드웨어 등 결함 발생 가능성이 큰 영역이 있다[Nystrom14].

플레이어가 게임을 배우거나 튜토리얼을 수행할 때 게임에 대한 충성도를 얻는 것이 중요하다. 이 단계에서 플레이어는 제품을 평가할 수 있는 수많은 게임 기능을 접하게 된다. 그렇기 때문에 이 초기 단계는 일반적으로 매우 중요한 것으로 인식된다.

1.1.5 테스팅과 “플레이”의 차이

컴퓨터 게임 테스터라는 직업의 본질이 하루 종일 동료들과 온라인 상에서 게임을 하는 것이라는 인상을 받을 수 있다. 그러나 이 말은 사실과 거리가 상당히 멀다.

테스터는 새로운 프로젝트를 알아 갈 때 또는 플레이 테스트를 수행할 때를 제외하고는 그냥 플레이하는 경우가 거의 없다. 되려 하는 일의 대부분은 다양한 유형의 테스팅을 수행하는 것과 관련이 있으며, 이는 여러 측면에서 다른 소프트웨어의 테스트와 동일하다.

일반 사용자는 게임을 완료하거나 상대를 이기거나 즐거운 시간을 보내기 위해 게임을 시작하는 반면, 테스터는 게임이 명세에 명시된 요구사항을 충족하는지 확인한다. 또한, 테스터의 개인 성향이 게임의 의도된 대상 고객과 일치하지 않을 수 있다. 그럼에도 불구하고 테스터는 게임 오브젝트가 올바르게 표시되는지 등을 테스트하기 위해 같은 레벨을 반복해서 진행해야 한다. 테스터가 이러한 반복적인 작업을 수행할 때 경험하는 단조로움은 개발자가 생각한 게임플레이를 가로막는 결함을 발견했을 때 얻는 만족감으로 보상될 수 있다.

다른 실라버스와의 관계

이 실라버스에서 얘기하는 접근법은 게임 테스팅에만 적용되며 소프트웨어 테스팅(예: 게임 메커니즘

테스팅)과 하드웨어 테스팅(예: 컨트롤러 테스팅) 모두에 적용된다.

게임 테스팅의 일부, 특히 리그레션 테스팅, 성능 테스팅, 사용성 테스팅과 같은 테스트를 실행하는 것은 다른 소프트웨어 테스팅과 크게 다르지 않다. 테스팅 기본 개념, 비기능 테스팅 기법, 모바일 플랫폼에서의 소프트웨어 테스팅은 다른 ISTQB® 실라버스에서 자세히 다루고 있다.

1.2 게임 개발팀 내 일반적인 역할

대부분의 컴퓨터 게임 개발팀의 역할도 다른 소프트웨어 개발에서의 역할과 크게 다르지 않지만 이에 국한되지는 않는다. 여기에는 비즈니스 책임자(owner), 프로젝트 관리자, 분석가, 개발자, 게임 디자이너, 테스터 등이 포함된다. 소규모 팀에서는 한 사람이 동시에 여러 역할을 맡을 수 있다. 다음은 게임 소프트웨어 프로젝트 내 일반적인 역할과 수행 활동을 요약한 것이다. 게임 개발에만 있는 영역을 위해 팀에 있는 추가 역할은 이 실라버스의 해당하는 장에서 설명하고 있다.

역할	책임
비즈니스 책임자	● 사업 개발 ● 마케팅 정책 정의 ● 신제품 착수 ● 팀 빌딩 참여 가능
프로젝트 관리자	● 수행할 작업(tasks)을 설정하고 모니터링 ● 소프트웨어 개발 수명 주기 단계 추적
분석가	● 제품 요구사항 식별 ● 초기 명세 작성 ● 명세 업데이트 및 최신화
개발자	● 코드 개발 및 디버깅 ● 결함 수정 ● 단위 테스트 개발
게임 디자이너	● 게임 메커니즘, 밸런스, 플롯, 레벨, 게임 경제 설계 ● 게임 문서 작성 ● 게임 콘텐츠 기획 및 개발
테스터	● 테스트 모델 및 테스팅 전략 개발 ● 테스트 데이터 설계 및 준비 ● 명세 리뷰 ● 다양한 유형의 테스팅 수행 ● 결함 식별 및 분석

1.3 게임 소프트웨어 개발 수명 주기 전반에 걸친 테스트 활동

컨셉 단계

이 단계의 주요 목표는 개발팀과 고객이 미래 제품에 대한 공통된 비전에 합의하는 것이다. 게임의 전반적인 비전, 장르, 설정, 게임플레이의 본질, 메커니즘과 주요 기능을 간략하게 설명하는 문서 형태로 아이디어를 작성한다.

이 단계에서 리뷰할 수 있는 산출물은 다음과 같다:

- 비전 문서
- 컨셉 문서
- 제품 및 기술 제약

이 단계에서 생성되는 테스트웨어는 다음과 같다:

- 테스트 전략
- 테스트 계획 초안
- 상위 수준의 테스트 노력 추정치
- 테스트 환경

사전 제작 단계

이 단계의 주요 목표는 초기 동작 버전인 게임 프로토타입을 만드는 것이다. 프로토타입은 주요 게임플레이를 초기 품질 수준으로 구현한다. 이 단계에서 핵심 게임 메커니즘과 초기 가설을 테스트하고(2장 참조) 팀의 기술적 능력을 평가한다. 테스터, 게임 디자이너, 개발팀의 기타 구성원이 프로토타입 테스트에 참여할 수 있다.

요구사항 초안을 리뷰하여 개발팀은 구현 가능성을 검토하고, 테스트팀은 메커니즘 간의 상호작용과 구현된 프로토타입의 기능을 식별하게 된다. 또한, 테스터는 테스트 계획 및 테스트 일정 문서 작성에 기여한다.

게임에 대한 문서가 작성되면 테스트 계획을 작성 및 업데이트하고 문서를 리뷰한다. 이러한 활동은 사전 제작 단계에서 제품의 기술적 리스크를 완화한다.

이 단계에서 리뷰할 수 있는 산출물은 다음과 같다:

- 요구사항 명세
- 게임 프로토타입
- 이전 단계에서 작성한 테스트 산출물

이 단계에서 생성 및 업데이트되는 테스트웨어는 다음과 같다:

- 테스트 전략
- 테스트 계획
- 테스트 일정
- 테스트 환경

개발 단계

개발 단계에서는 게임이 개발된다. 이 단계가 가장 긴 단계이며 일반적으로 게임의 알파 및 베타 버전으로 나뉘지게 된다.

이 단계에서 테스터는 체크리스트와 테스트 조건 세트를 마무리하고 구현된 제품 구성 요소의 기능 및 비기능 테스트를 수행한다. 단위 통합, 시스템, 인수 테스트 등이 수행된다 ([ISTQB_FL_SYL] 참조). 이 단계에서 가장 많은 수의 결함이 식별되기 때문에 테스터는 확인 및 리그레션 테스트를 수행하게 된다.

이 단계에서 생성 및 업데이트되는 테스트웨어는 다음과 같다:

- 테스트 계획
- 테스트 일정
- 테스트 케이스, 테스트 조건, 체크리스트, 테스트 스위트, 스크립트
- 결함 보고서
- 테스트 보고서

후반 작업 단계

게임 테스트의 후반 작업 단계에는 일반적인 소프트웨어의 유지 보수 기간 동안 수행되는 테스트 활동이 포함된다. 여기에는 보통 [ISTQB_FL_SYL]에서 설명하는 것처럼 업데이트 및 리그레션 테스트가 포함된다.

2. 게임 메커니즘 테스트 - 180 분 (K3)

테스팅 용어

에드혹 테스트(Ad hoc testing), 기능 테스트(functional testing)

비디오게임 전용 용어

클라이언트 메커니즘(Client mechanics), 컨셉 단계(concept stage), 핵심 메커니즘(core mechanics), 비디오게임 디자이너(video game designer), 비디오게임 메커니즘(video game mechanics), 메타 메커니즘(meta mechanics), 사전-제작 단계(pre-production stage), 개발 단계(production stage), 서버 메커니즘(server mechanics), 비디오게임 상태(video game state)

2 장 학습 목표

2.1 게임 메커니즘

GaMe-2.1.1 (K2) 게임 메커니즘의 유형을 분류할 수 있다

GaMe-2.1.2 (K2) 게임플레이 메커니즘과 비게임플레이 메커니즘 테스트의 차이를 설명할 수 있다

GaMe-2.1.3 (K2) 핵심 메커니즘과 메타 메커니즘 테스트의 차이를 설명할 수 있다

GaMe-2.1.4 (K2) 클라이언트, 서버, 클라이언트-서버 메커니즘 테스트의 차이를 설명할 수 있다

GaMe-2.1.5 (K2) 게임 메커니즘 결함 사례를 제시할 수 있다

2.2 게임 메커니즘 테스트 접근법

GaMe-2.2.1 (K2) 게임 개발 여러 단계의 주요 접근법과 테스트 대상을 요약할 수 있다

GaMe-2.2.2 (K2) 게임 메커니즘 테스트의 중요성을 인식할 수 있다

GaMe-2.2.3 (K2) 게임 메커니즘을 설명하는 문서 리뷰의 중요성을 인식할 수 있다

GaMe-2.2.4 (K3) 게임 메커니즘 테스트의 주요 접근법을 적용할 수 있다

2.1 게임 메커니즘

2.1.1 게임 메커니즘 유형

1 장에서 게임의 핵심적인 요소 중 하나는 플레이어(audience)를 늘리고 유지하기 위해 사용자(user)의 관심을 유도하는 것이라고 언급했다. 플레이어는 게임의 그래픽, 게임플레이 그리고 게임플레이어를 구성하는 메커니즘을 평가하게 된다.

비디오게임 메커니즘은 모든 게임의 핵심이다. 비디오게임 메커니즘은 게임과 사용자 간의 상호작용 방법을 정의하며 게임과 사용자 간의 영향 및 피드백과 정의된 요구사항 안에서 비디오게임의 상태 변화를 고려해서 정해진다. 메커니즘은 게임의 양상과 목표에 따라 여러 유형으로 나눌 수 있다.

메커니즘과 플레이어의 상호작용 유형에 따라:

- 게임플레이 메커니즘은 사용자가 가용한 정보를 바탕으로 의식적으로 게임 시스템과 상호 작용하면서 행동하는 것과 관련이 있다. 사용자는 행동의 결과를 명확하게 확인하면서 비디오게임 상태의 변화에 영향을 미치게 된다.
- 비-게임플레이 메커니즘은 사용자가 자신의 행동을 통해 게임 환경 상태에 영향을 줄 수 없거나, 주더라도 제한적일 때와 관련이 있다. 사용자는 게임 메커니즘의 일정 부분은 확인할 수 없기 때문에 자신의 영향력을 확인하지 못할 수도 있다.

주요 게임플레이에 미치는 영향에 따라:

- 핵심 메커니즘은 게임의 기초를 형성하고 사용자가 게임 전반에서 반복적으로 하는 동작을 정의한다. 이는 제작사가 게임에 넣어 놓은 어떤 경험을 사용자가 체험할 수 있도록 하기 위한 것이다.
- 메타 메커니즘은 주요 게임플레이에 포함되지 않지만, 그것에 영향을 미칠 수는 있다. 이는 게임 디자이너가 의도한 행동을 플레이어가 하도록 하는 것을 목표로 한다 (예: 게임으로 돌아오기, 계속 플레이하기, 구매하기). 이러한 메커니즘은 핵심 메커니즘과 결합해서 게임을 더 흥미롭게 만들 수 있다.

게임 구조의 아키텍처에 따라:

- 클라이언트 메커니즘은 사용자 디바이스에서 발생하는 사용자 행동 처리와 관련이 있다.
- 서버 메커니즘은 게임 서버에서 처리되는 메커니즘과 관련이 있다.
- 클라이언트-서버 메커니즘은 사용자 디바이스와 서버 모두에서 처리되는 메커니즘과 관련이 있다. 이때는 서버와 클라이언트 간 지속적인 데이터 통신이 있다.

2.1.2 게임플레이 메커니즘 테스트와 비-게임플레이 메커니즘 테스트의 차이

사용자는 게임에 구현된 모든 메커니즘과 직접 상호작용하지는 않는다.

게임플레이 메커니즘은 공식 또는 비공식 명세에 명시되어 있을 수 있지만, 비-게임플레이 메커니즘은 사용자에게 설명되어야 한다. 게임플레이 메커니즘은 게임을 플레이하는 목적 달성에 직접적인 영향을 미치는 경우가 많다. 예를 들어, 게임 캐릭터로 제한된 시간 내에 코인을 수집해야 하는 상황을 들 수 있다. 코인을 수집하기 위한 움직임과 코인 수를 늘리는 기능은 게임 메커니즘의 예로 볼 수 있다. 비-게임플레이 메커니즘은 보통 사용자에게 보이지 않지만, 사용자의 행동으로 발현될 수 있다. 예를 들어, 사용자가 게임 내에서 어떤 제품을 구매하면 관련 비-게임 메커니즘이 순차적으로 시작될 수 있다. 예를 들어:

1. 사용자의 게임 계정 상태를 가져오기 위해 데이터베이스로 호출이 이루어진다.
2. 구매 비용을 인출하기 위한 거래가 이루어진다.
3. 구매가 기록된다. 즉, 수행된 동작에 대한 정보가 특수 파일에 기록된다.
4. 구매 세부 사항이 담긴 이메일이 사용자에게 전송된다.

또한, 구매 날짜와 제품 유형을 기록해서 유사한 제품의 할인 정보가 자동으로 이메일로 전송되도록 할 수 있다. 비-게임플레이 메커니즘은 드러나지 않기 때문에 공식 요구사항으로 명시되어야 한다.

게임플레이 및 비-게임플레이 메커니즘의 테스트는 기능 테스트이다 [ISTQB_FL_SYL].

2.1.3 핵심 메커니즘 테스트와 메타 메커니즘 테스트의 차이

핵심 메커니즘과 메타 메커니즘 테스트 접근법은 그 영향에 따라 달라진다.

게임에서 하나의 핵심 메커니즘을 제거하면 게임플레이가 완전히 바뀌게 된다. 예를 들어, 레이싱 시뮬레이터에서 자동차의 움직임을 제거하면 게임을 더 이상 "레이싱"으로 분류할 수 없다.

메타 메커니즘의 예로는 레이스 트랙의 유형에 따라 특정 옵션을 가진 자동차 모델을 선택하거나 전투에 앞서 플레이어의 캐릭터에게 새로운 갑옷을 구입하여 장착하는 것을 들 수 있다. 게임에서 어떤 메타 메커니즘을 제거했다고 전체 게임의 본질이 바뀌는 경우는 거의 없다.

기능 요구사항 외에도 핵심 및 메타 메커니즘은 테스트 방법에 직접적인 영향을 미치는 비기능 요구사항이 있을 수 있다.

예를 들어, 플레이 가능한 캐릭터의 외형을 변경하는 메타 메커니즘의 경우 가장 중요한 것은 기능이 올바르게 동작하는 것이다. 플레이어가 버튼을 누른 뒤 몇 초 후에 외형이 바뀌는 것처럼 메커니즘이 약간 느리게 동작하더라도 테스터나 사용자가 이것을 결함으로 간주하지 않는다. 그러나 캐릭터가 점프하거나 전투에서 어떤 능력을 사용하는 핵심 메커니즘에 이러한 지연이 있다면 플레이어가 이기는 것을 방해할 수 있기 때문에 매우 치명적이다. 그래서 핵심 메커니즘에는 성능 테스트도 중요하게 된다.

핵심 및 메타 메커니즘의 본질도 테스트를 수행하는 시점에 영향을 미친다. 일반적으로 이런 메커니즘의 검증은 게임 개발의 여러 단계에서 시작된다. 게임 캐릭터의 움직임, 보상받기 등 핵심 메커니즘은 게임의 초기 버전에 구현된다. 동시에 기능 정확성과 목표 고객의 관심 수준에 대한 검증도 시작된다.

메타 메커니즘은 일반적으로 개발 단계에서 추가되고 테스트된다. 메타 메커니즘이 게임 세션의 평균

길이, 초대 친구 수, 평균 구매 규모, 가장 인기 있는 게임 제품 등에 미치는 영향에 대한 평가는 베타 테스트 단계에서 이루어진다. 동일한 시점에 가장 효과적인 메커니즘을 판단하기 위해 A/B 테스트를 사용하는 경우도 많다.

2.1.4 클라이언트, 서버, 클라이언트-서버 메커니즘 테스트의 차이

클라이언트, 서버, 클라이언트-서버 메커니즘을 테스트해야 하는 필요성은 게임 자체의 아키텍처에 따라 다르다. 각 유형은 구현 방법에 따라 테스트 접근법이 달라야 한다.

클라이언트 메커니즘은 플레이어의 기기에서 처리되므로 사용자는 클라이언트에 저장된 모든 데이터에 접근할 수 있다. 일부 게임은 서버 없이 클라이언트에서만 실행될 수 있다 (즉, 모든 메커니즘이 클라이언트에서 실행됨). 일반적으로 이런 게임은 1 인용 게임으로 인터넷 연결이 필요하지 않다.

게임 클라이언트는 플레이어의 기기에서 실행되는 코드와 모든 변수, 피해 계산 공식, 게임 레벨, 캐릭터 모델 등을 가지고 있다. 그렇기 때문에 플레이어는 원한다면 게임 클라이언트의 모든 데이터를 수정할 수 있다. 클라이언트를 수정해서 사용자는 기술의 속도, 캐릭터 체력, 퀘스트 완료 보상의 가치 등을 높일 수 있다.

이러한 클라이언트 변경은 플레이어에게 게임 내 이점을 제공할 수 있지만 1 인용 게임의 경우 보통 심각한 결함으로 간주하지 않는다. 예를 들어, 1 인용 게임에 컴퓨터와 칼로 싸우는 메커니즘이 구현돼 있을 수 있다. 이 경우 무기를 규정하는 변수는 서버에서 검증하지 않는다. 따라서, 특정 플레이어가 게임 클라이언트를 수정해서 검의 데미지를 증가시키면 자신의 게임플레이에만 영향을 주게 된다. 각자의 기기로 같은 게임을 하는 다른 플레이어에게는 영향을 미치지 않는다.

클라이언트 메커니즘은 멀티플레이어 게임에도 있을 수 있지만 여전히 다른 플레이어에게는 영향을 미치지 않는다. 예를 들어, 레벨 맵(map)을 보거나 캐릭터 인벤토리를 여는 등의 메커니즘은 클라이언트에서만 처리되고 서버를 사용하지 않는 기능 테스트로 검증할 수 있다.

클라이언트 메커니즘 테스트는 일반적으로 블랙박스 테스트를 사용하여 수행한다. 이 작업을 수행할 때 테스터는 애플리케이션의 사용자 인터페이스를 확인해야 할 수 있다. 이 경우 실제 결과를 얻기 위해 실제 사용자 인터페이스를 사용해서 게임을 테스트하기도 한다.

서버 메커니즘은 서버에만 구현된다. 이런 특징으로 인해 메커니즘의 논리는 사용자 개입으로부터 보호된다. 플레이어가 원격 서버에서 실행되는 프로세스에 직접 영향을 줄 수 없으므로 개발자는 게임 논리 계산의 가장 중요한 부분을 원격 서버로 이관한다.

서버 메커니즘 테스트는 멀티 플레이어 온라인 게임에서 가장 중요하다. 예를 들어, 플레이어가 대인전(PvP) 모드로 팀전을 시작하면, 게임 등급과 캐릭터 레벨이 비슷한 아군과 적군을 서버에서 선정한다. 그렇게 하지 않으면 참가자 전원에게 재미없고 불편한 경쟁이 될 수 있다.

서버에 구현된 기능을 테스트하는 방법은 클라이언트 메커니즘을 테스트하는 방법과 다르다. 여기서 테스터는 사용자 인터페이스가 필요 없는 경우가 많다. 테스트는 서버 콘솔 또는 특별한 도구(special tools)를 사용하여 수행한다. 테스터는 로그(log)를 분석하거나 데이터베이스와 상호작용하는 데 대부분의 시간을 보낸다.

서버 메커니즘에서 가장 중요한 테스트 유형은 기능 테스트, 성능 테스트, 보안 테스트이다.

클라이언트-서버 메커니즘은 클라이언트와 서버를 다루며, 앞서 언급한 두 가지 유형의 메커니즘이 가진 특징을 모두 가지고 있다. 논리의 일정 부분은 플레이어의 디바이스에서 처리되고, 나머지는 원격 서버에서 처리된다. 이러한 메커니즘의 일부는 지속적으로 서로 "통신"하여 서로 데이터를 주고받는다. 멀티 플레이어 게임에서 서버는 클라이언트에서 데이터를 수신한 다음, 이루어진 작업의 영향을 받는 모든 플레이어에게 작업 결과를 보낸다.

서버에 도착한 데이터는 반드시 검증된다. 예를 들어, 사용자가 게임 내 상점에서 무언가를 구매하려는 경우, 서버는 플레이어가 구매에 필요한 게임 내 통화가 충분한지 확인해야 한다. 플레이어가 게임에서 우위를 점하기 위해 디바이스에서 클라이언트의 일부를 수정해서 서버에 잘못된 값을 보내려고 하더라도 서버는 이를 올바른 데이터로 대체해야 한다.

테스터는 클라이언트-서버 메커니즘을 테스트할 때 최대한 많은 메커니즘을 포함한 하나 이상의 테스트 케이스를 만들게 된다.

이러한 테스트의 예로는 클라이언트가 게임 서버에 로그인하여, 컴퓨터가 제어하는 상대를 제거하는 경우를 들 수 있다. 이 테스트를 구성하는 데 필요한 행동의 수는 많지 않지만, 거의 모든 클라이언트-서버 상호작용 메커니즘을 여기에 사용하게 된다:

- 클라이언트에서 권한 부여를 담당하는 인증 서버로 요청을 전송한다.
- 인증 서버에서 데이터베이스로 요청을 전송하고, 인증을 수행하고, 이전 비디오게임 상태로 복원한다.
- 서버에 이미 로딩된 맵을 선택하거나 서버에 새로운 게임 메커니즘을 생성한다.
- 맵이 위치한 게임 메커니즘 서버에 클라이언트를 연결한다.
- 서버가 게임 내 오브젝트를 생성하고 클라이언트에 오브젝트 표시를 명령하면, 클라이언트는 캐릭터와 주변 환경을 렌더링하여 화면에 표시한다.
- 새로운 게임 오브젝트를 표시하는 동안 맵 안에서 캐릭터를 이동한다.
- 컴퓨터가 제어하는 적과 그것의 공격을 식별해서 클라이언트에게 공격 정보를 표시한다.
- 서버로부터 "공격" 명령을 수신한다.
- 캐릭터와 몸(Mob, mobile object)이 공격에 적합한 거리에 있는지, 캐릭터와 몸 사이에 공격에 방해하는 물체가 없는지 확인하여 공격이 가능한지 확인한다.
- 공격이 가능한 경우 서버에서 몸에게 피해 수신에 대한 메시지를 전송한다.
- 서버에서 몸이 받은 피해량을 계산하여 몸의 체력을 해당 변수 값만큼 줄인다.
- 몸이 플레이어에게 가한 피해를 표시한다.

2.1.5 게임 메커니즘 결함 및 발생 원인 예

게임 메커니즘에는 게임 오브젝트에 대한 영향과 이러한 영향의 결과를 알리는 피드백이 포함된다. 두 가지를 합치게 되면 게임의 독창적인 특징과 적합한 게임 동작이 만들어진다. 대부분의 경우 게임

은 다양한 반응 요소와 피드백 요소들을 사용한다.

다음 결함 유형이 게임 메커니즘과 관련이 있다:

- 메커니즘의 기능적 결함
- 게임의 타당성 인식 결함
- 특정 게임 환경에서 나타나는 메커니즘의 효과성 결함

게임의 기능적 결함은 플레이어가 게임 메커니즘 결함에 대해 이야기할 때 가장 많이 언급된다. 예를 들어, 무기가 재장전 되지 않거나, 아이템을 수집하지 못하거나, 쌍안경을 써도 이미지가 가까워지지 않는 경우를 들 수 있다. 이러한 결함은 보통 개발자가 만든 게임 코드에 있는 오류로 발생한다. 그리고 눈에 띄며, 식별하고 수정하기가 쉽다.

게임의 타당성 인식 결함은 특정 메커니즘을 사용하는 동안 게임에서 응답을 얻는 것과 관련이 있다. 플레이어가 게임이 끝난 이유를 이해하지 못하면 문제가 있다. 그렇기 때문에 게임 메커니즘에 시각 및 음향 효과가 수반되는 경우가 많다. 폭발을 보여주는 시각적 효과, 시간의 경과를 알려주는 스톱워치 소리, 또는 단순한 텍스트 메시지도 그런 효과를 낼 수 있다. 메커니즘 사용에 대한 응답 자체의 유무가 테스트되고, 그 유무의 근거가 평가된다. 효과의 동작 및 정확성은 다른 유형의 테스트(그래픽 또는 사운드 테스트)으로 검증된다.

세 번째 유형의 결함은 메커니즘의 범위 및 효과성과 관련이 있다. 메커니즘은 그 자체로는 잘 동작하지만, 게임플레이 내에서 동작하지 않을 수 있다. 이러한 상황을 피하기 위해 메커니즘은 게임 레벨의 다른 메커니즘, 오브젝트, 기타 필요한 콘텐츠와 함께 테스트한다. 따라서 통합 테스트는 게임플레이에서 메커니즘의 효과성을 확인하기 위해 사용성 테스트와 함께 수행한다.

예를 들어, 게임 디자이너가 행동과 특성이 다른 두 명의 적을 게임에 추가했다고 하자. 첫 번째는 빠르고 이리저리 뛰어다니지만, 공격력이 약하고, 두 번째는 느리고 둔하지만, 공격력이 강하다. 특성 면에서 두 상대는 동일해 보인다. 각각 고유의 장단점이 있다. 각자 다른 행동 알고리즘을 가질 수 있으며 이론적으로 플레이어에게 같은 수준의 위협이 된다. 그러나 게임 레벨의 많은 부분이 플레이어의 캐릭터가 오를 수 있는 여러 개의 언덕으로 구성되어 있으면 상황이 바뀐다. 느린 적이 접근할 수 없는 높이에 있는 플레이어는 안전하게 적을 공격할 수 있다. 거기에 반해 마찬가지로 언덕을 사용하며 빠른 속도로 움직이고 높이 점프할 수 있는 다른 적을 상대하는 데에는 어려움을 겪게 된다. 이 문제의 해결책은 메커니즘을 수정하거나 레벨에 있는 오브젝트의 위치를 바꾸는 것일 수 있다.

이러한 테스트의 복잡성은 그러한 상황과 그때 메커니즘이 어떻게 동작할지 예측하는 것과 관련이 있다. 따라서 메커니즘과 환경 오브젝트 간의 상호작용 문제를 찾는 것은 여러 플레이어 그룹과 함께 하는 베타 테스트 단계 내 애드혹 테스트의 일환으로 수행하는 경우가 많다.

2.2 게임 메커니즘 테스트 접근법

2.2.1 게임 소프트웨어 개발 수명주기 전반에서 게임 메커니즘 테스트의 절차 및 접근법

소프트웨어 개발의 여러 단계에서 다양한 테스트 접근법을 사용하여 게임 메커니즘을 테스트할 수 있다.

게임 프로토타입을 만드는 단계에서는 핵심 메커니즘만 구현하게 된다. 여기서 테스터의 주요 작업은 다음 특성에 대해 게임 메커니즘을 테스트하는 것이다.

- 기능 정확성
- 플레이어가 가질 관심 수준
- 플레이어가 느낄 매력

모든 메커니즘과 메커니즘의 동작 방식을 리뷰하여 비디오게임 설계 문서에 완전하고 모호하지 않게 설명되어 있는지 확인한다. 사용자와 메커니즘의 상호작용에 대해 허용되는 모든 옵션(option)을 확인해야 하며 각 메커니즘이 전체 게임플레이에 미치는 영향을 결정해야 한다.

개발 단계에서는 구현된 메커니즘이 명시된 요구사항을 충족하는지 확인하기 위한 기능 테스트를 수행한다. 일반적으로 여기서 테스트는 테스트 조건 및 체크리스트 세트를 사용하여 수행한다. 테스트 절차는 메커니즘의 모든 측면을 최대한 다루는 것을 목표로 해야 한다.

탐색적 테스트와 애드혹 테스트는 서로 다른 메커니즘의 상호작용을 테스트하는 데 효과적이다. 특히, 대규모 멀티플레이어 게임처럼 테스트 조건으로 정의하기에는 게임의 메커니즘과 다양한 구성 요소 간의 상호작용 방법이 너무 많은 경우에 더 그러하다.

비기능 테스트의 일환으로 메커니즘이 게임의 전반적인 성능에 미치는 영향을 테스트해야 한다. 자원(resource) 사용률(예를 들어, 메모리, CPU 등)과 실제 결과(예를 들어, 게임 클라이언트의 멈춤 횟수, 프레임 속도의 현저한 감소 등)를 확인한다. 다양한 구성 요소와 소프트웨어의 호환성, 특히 바이러스 백신 프로그램과의 호환성도 테스트해야 한다.

게임에 서버가 필요한 경우, 서버 성능도 테스트한다. 성능 테스트는 계획된 메커니즘 중 성능에 영향을 줄 수 있는 것들이 구현되는 개발 단계에서 계획하고 실행한다. 성능 테스트의 계획 및 실행에 대한 자세한 내용은 [ISTQB_FL_PT]에 설명되어 있다.

많은 수의 플레이어를 동원한 베타 테스트를 수행할 때 메타 메커니즘에 상당한 주의를 기울여야 한다. 플레이어가 느끼는 게임의 편의성과 명확성이 평가된다. 게임이 아닌 소프트웨어를 테스트하는 것과 마찬가지로 이 테스트의 주요 목적은 명백한 기능 결함을 찾는 것이 아니라 최종 플레이어로부터 피드백을 받는 것이다. 이상적으로는 명백한 기능 결함은 앞선 단계에서 식별되어야 한다. 대규모 멀티플레이어 게임처럼 일부 게임의 경우, 최종 플레이어가 초기 알파 테스트에도 참여하고 피드백을 제공할 수 있다.

게임이 시장에 출시된 후 테스터의 임무는 플레이어가 보고한 메커니즘 결함을 처리 및 확인하고 게임에 추가되는 새로운 메커니즘을 테스트하는 것이다.

2.2.2 게임 메커니즘 테스트의 중요성

게임 메커니즘은 게임의 핵심이다. 플레이어의 관심을 유지하기 위해 중요한 사용자 경험의 근간을 형성한다. 게임 메커니즘은 모든 게임의 핵심이기 때문에 올바르게 동작하는지 테스트하는 것이 테스터에게 가장 높은 우선순위를 가지게 되며 이때 발견하는 결함이 가장 중요한 경우가 많다.

캐릭터의 점프가 작동하지 않는 경우처럼 핵심 메커니즘에 결함이 생기게 되면 특정 장애물을 극복하지 못해서 결국 게임을 완수하지 못하게 될 수도 있다.

심지어 게임에서 거의 사용되지 않는 메커니즘이거나 게임을 완수하는 데 꼭 필요한 활동이 아니더라도(예: 캐릭터가 말을 타는 능력) 잘못된 동작은 게임 전체에 대한 인식에 부정적인 영향을 미치게 된다.

2.2.3 게임 메커니즘 리뷰의 중요성

게임 개발에서 게임 메커니즘을 설명하는 게임 문서의 작성은 중요한 단계이다. 게임 문서에는 게임 메커니즘 외에도 여러 정보가 담기게 된다.

개발 단계 초기에 게임 메커니즘 명세를 리뷰하면 나중에 많은 문제를 피할 수 있다. 이러한 문제는 개발 프로세스뿐만 아니라 최종 제품에 구현된 게임 메커니즘에 대한 플레이어의 반응과 관련이 있을 수 있다.

리뷰 과정에서 테스터는 다음 품질 특성[ISO 25010]에 주의를 기울여야 한다:

- 설명의 완전성
- 현실과의 일치
- 문서의 구조와 탐색 용이성

게임 개발 프로세스에서 리뷰로 예방할 수 있는 문제:

- 모호하고 구체적이지 않은 메커니즘 설명으로 인한 메커니즘의 본질에 대한 개발자의 잘못된 이해와 필요 노력에 대한 잘못된 평가
- 새로운 메커니즘과 기존 메커니즘의 비 호환성
- 특정 프로젝트에 메커니즘을 구현할 수 없는 경우

리뷰로 예방할 수 있는 메커니즘에 대한 플레이어의 인식과 관련된 문제:

- 특정 게임에서 메커니즘의 전반적인 부적절함
- 흥미롭지 않은 메커니즘
- 불공정한 메커니즘
- 게임에서 너무 드물게 사용되는 메커니즘

2.2.4 세션 재개 후 및 사용자가 비활성 상태일 때의 비디오게임 상태 테스트

비디오게임 상태

비디오게임 상태란 특정 순간에 게임 내의 모든 오브젝트를 설명하는 모든 변수와 변수 값을 말한다. 게임이 복잡해지고 플레이어가 수행할 수 있는 행동이 많아질수록 각 오브젝트는 더 많은 변수를 가지게 된다.

모든 변수는 조건에 따라 드러난 것과 숨겨진 것으로 구별할 수 있다.

드러난 변수의 값에 대한 정보는 플레이어에게 명시적으로 제공된다. 여기에는 다음이 포함된다:

- 완료한 작업으로 인한 게임 내 사용자 경험치 수준

- 선택한 무기의 특성
- 인벤토리에 허용되는 아이템 수
- 게임 내 상점의 상품 가격

숨겨진 변수는 개발자에 의해 플레이어가 인지하지 못하는 사이에 게임 진행에 영향을 미치기 위해 사용한다. 예를 들어, 플레이어가 알고 있는 공격 및 방어 수치 외에 캐릭터는 정확도 변수를 가지고 있을 수 있는데, 이 수치는 플레이어에게는 숨겨지며 공격이 목표물에 맞을 확률을 결정할 때 사용된다. 플레이어는 그러한 변수의 존재와 정확한 영향을 알지 못하는 경우가 많다.

숨겨진 변수는 게임의 다양한 측면을 담당할 수 있다:

- 드랍률, 즉 보상으로 무작위 아이템을 획득할 확률
- 발사 속도 또는 다양한 표면에서 물체의 이동 속도

플레이어의 선택은 게임 플롯의 전개에 영향을 미친다. 플레이어가 제어하는 게임 캐릭터와 기타 여러 오브젝트를 정의하는 고유의 숨겨진 명시적 변수 집합이 있다.

비디오게임 상태 테스트의 목적

게임 변수를 테스트할 때 중요한 요소 중 하나는 게임이 변수의 실제 값을 정말로 사용하는지 테스트하는 것이다.

드러난 변수의 경우, 표시된 값을 실제로 사용된 값과 시각적으로 비교하여 테스트를 수행한다. 예를 들어, 공격력이 1 인 게임 캐릭터가 적에게 10 만큼의 피해를 입힌 경우, 플레이어가 공격력을 2 로 올리면 동일한 적에게 가해지는 피해량도 증가해야 한다(예를 들면, 20 으로).

이러한 테스트는 새로운 피해량 정보가 “공격력이 1 올라가면 피해량은 10 만큼 증가하게 된다”라는 문구처럼 게임 내 도움말 형식으로 플레이어에게 제공되는 경우에 특히 중요하다. 이 경우, 플레이어도 도움말에 표시된 값을 실제 피해량과 비교할 수 있다.

일반적인 플레이어는 보통은 특정 변수 값을 고려하지 않고, 피해량 변화의 적절성과 일관성에만 관심을 둔다. 플레이어는 피해량이 정확하지 않은 것은 결함으로 간주하지 않을 수 있지만, 피해량이 바뀌지 않거나 되려 감소하게 되면 분명 인식하게 된다.

전문 테스터는 문서에 명시된 계산 공식 및 원칙에 따라 테스트를 수행한다. 이러한 정보를 문서에서 얻을 수 없다면 테스터는 플레이어와 마찬가지로 상식에 따라 상대적으로 평가를 수행하게 된다.

숨겨진 변수를 테스트하기 위해 테스터는 때때로 게임 데이터베이스에 접근해야 할 수도 있다. 그렇게 해서 테스터는 변수의 실제 값을 예상 값과 비교할 수 있다.

또한, 사용자가 게임 중에 캐릭터와 연관된 드러난 변수를 바꿀 수 있다면, 변수를 변경하는 것 자체와 표시된 값의 정확성도 테스트하게 된다.

비디오게임 상태 테스트는 게임 저장 및 불러오기와 밀접한 관련이 있다.

저장 및 불러오기

게임을 단일 세션 내에 완료하는 게 불가능하거나 원칙적으로 게임플레이에 끝이 없는 경우도 많다. 따라서 게임은 플레이어가 매번 처음부터 시작할 필요가 없도록 “저장과 불러오기”라는 비디오게임 상태 기능을 지원하게 된다.

저장을 사용하는 다른 이유로는 사용자가 게임 세계를 탐험하고, 게임이 제시하는 문제를 해결할 다양한 방법을 찾거나, 경우에 따라서는 게임 자체의 복잡성을 관리하기 위해서일 수도 있다. 플레이어가 어려운 장애물 앞에서 생존할 수 있는 기회가 있다면, 잘못된 행동으로 진행이 완전히 무효가 되는 경우보다 실패 비용이 적어진다.

저장은 하드 드라이브 또는 클라우드에 저장된 게임 진행 상황 정보를 말한다. 저장은 게임의 현재 상태에 대한 정보가 포함된 파일이 될 수 있다. 이러한 파일의 크기는 게임 자체보다 매우 작으며 일반적으로 다음 정보를 포함한다:

- 상태 정보를 저장해야 하는 오브젝트 목록
- 각 오브젝트에 대한 변경되는 변수 목록
- 게임에 현재 무슨 일이 일어나고 있는지 확인할 수 있는 각 변수에 대한 구체적인 코드

거의 모든 변수는 오브젝트가 될 수 있다. 그리고 그 상태는 게임 시스템에 의해 수정될 수 있으며 기억되어야 한다. 오브젝트에는 캐릭터의 체력 수준, 학습한 능력의 수, 게임 맵에서 캐릭터의 좌표 등이 있을 수 있다.

저장 유형과 테스트 영역

저장 방법, 저장 파일 형식, 저장된 변수 목록은 게임마다 다를 수 있다. 비디오게임의 역사에서 진행 상황을 저장하기 위한 접근법은 여럿 발명되었다. 가장 일반적으로 사용되는 것은 아래에 설명되어 있다.

저장에 체크포인트 사용

저장은 개발자가 지정한 순간에 자동으로 발생한다. 다음 체크포인트에 도달하면 플레이어의 현재 진행 상황 정보를 덮어쓰게 된다. 체크포인트에 저장된 비디오게임 상태는 현재 게임 세션에 대해서만 저장되며 게임이 종료되면 삭제된다.

게임이 사용자가 체크포인트에 도달했음을 명시적으로 알려주지 않고 진행 상황을 저장하는 경우도 있다. 저장 및 불러오기의 정확성을 테스트하기 위해 테스터는 모든 체크포인트의 정확한 위치 목록을 제공하는 문서가 필요하다. 다른 저장 방법을 사용하는 경우 문서에 이러한 정보가 없거나 구체적으로 설명되어 있지 않을 수 있다.

고정 지점 저장 사용

개발자는 플레이어가 원할 경우 현재 진행 상황을 수동으로 저장할 수 있는 특수 지점을 어떤 장소에 배치하기도 한다. 이러한 지점은 일반적으로 소수의 비디오게임 오브젝트로 둘러싸여 있어서 저장 파일의 크기를 제한할 수 있게 해준다.

자동 저장

이 방법은 체크포인트와 고정 지점 저장을 조합한 것이다. 플레이어의 진행 상황은 여러 지점에서 게임플레이 전반에 걸쳐 자동으로 저장된다. 예를 들어, 종료 시 자동 저장 기능을 가진 게임이 많다. 게임을 시작할 때 사용자는 새 게임을 시작하거나 이전에 저장한 게임을 불러올 수 있다.

이 경우 테스터의 임무는 저장이 제대로 되었는지, 모든 지점에서 달성한 비디오게임 상태를 불러올 수 있는지 확인하는 것이다.

수동 또는 자유 저장

사용자는 게임 메뉴의 특정 항목을 사용하여 게임 중 언제든지 저장할 수 있다. 하나의 키를 누르거나 한 번의 클릭으로 저장 및 불러오기가 수행되도록 빠른 저장을 구현할 수도 있다.

자동 저장과 마찬가지로 수동 저장은 비디오게임 상태 정보가 포함된 특수 파일을 생성한다. 이 경우 테스트에는 파일에서 불러오는 정보의 정확성, 파일명의 정확성, 운영 체제에서 파일의 위치, 새로운 버전의 게임에서의 올바른 동작 그리고 저장 파일 사용 가능성을(예를 들어, 다른 컴퓨터에서) 확인하는 것이 포함될 수 있다.

다음 설명에 해당하는 게임에서는 필요한 게임 레벨을 올바르게 불러오는지 테스트하는 것으로 충분하다:

- 게임이 올바른 경로와 통과 방법이 하나만(예: 미로를 통과하는 방법이 하나만) 있다고 가정할 수 있다.
- 플레이어가 제어하는 게임 캐릭터가 레벨에 따라 바뀌지 않는다(예: 공격력, 점프 범위, 레벨에서의 이동 속도, 그리고 레벨 통과 속도를 향상시킬 수 있는 기타 특성이 증가하지 않음).

다음 설명에 해당하는 게임의 저장에는 게임 세션, 레벨, 캐릭터 등에 대한 더 구체적인 정보가 포함된다.

- 게임 캐릭터가 많은 변수(예를 들어, 이동 속도, 점프의 범위 및 높이, 공격력, 인벤토리, 3D 모델 외관, 기타 특성)를 가지고 있다.
- 위에서 설명한 변수가 플레이어의 게임 내 행동 및 결정에 따라 변경될 수 있다(예를 들어, 플레이어가 미로에서 아이템을 발견하고 이동 속도가 증가함).

이런 게임의 경우 테스터는 필요한 게임 레벨과 고유 정보뿐만 아니라 개별 플레이어 및 게임 캐릭터에 관한 고유 정보를 불러오는 것의 정확성을 테스트해야 한다. 저장을 테스트할 때 테스터는 다음을 확인해야 한다:

- 플레이어가 이전 게임 세션을 마친 게임 레벨에 있다.
- 플레이 가능한 캐릭터가 아이템을 가지고 있다.
- 게임 캐릭터의 속도 변수가 실제로 아이템으로 인해 증가했다.

3. 그래픽 테스트 - 165 분 (K3)

테스팅 용어

플레이 테스트(Playtest)

비디오게임 전용 용어

3D 모델(3D model), 애니메이션(animation), 충돌(collisions), 비디오게임 레벨(video game level), 타격 영역(hit box), 디테일 정도(level of detail (LoD)), 매핑(mapping), 조작성(rigging), 장면 조명(scene lighting), 스킨닝(skinning), 텍스처(textures), 시각효과(visual effect (VFX))

3 장 학습 목표

3.1 게임 그래픽의 원리와 개념

GaMe-3.1.1 (K2) 게임 그래픽 콘텐츠의 특징을 설명할 수 있다

GaMe-3.1.2 (K2) 그래픽 콘텐츠 결함 유형을 분류할 수 있다

3.2 게임의 그래픽 테스트 접근법

GaMe-3.2.1 (K2) 예술성 테스트의 주요 접근법을 요약할 수 있다

GaMe-3.2.2 (K2) 기술적 테스트의 주요 접근법을 요약할 수 있다

GaMe-3.2.3 (K2) 게임플레이 테스트의 주요 접근법을 요약할 수 있다

3.3 그래픽 테스트 실행

GaMe-3.3.1 (K3) 그래픽 테스트의 기본적인 접근법을 적용할 수 있다

GaMe-3.3.2 (K2) 역사적 사실의 정확도를 위한 그래픽 테스트의 중요성을 설명할 수 있다

3.4 그래픽 테스트 지원 도구

GaMe-3.4.1 (K2) 그래픽 테스트 도구의 사용을 요약할 수 있다

3.1 게임 그래픽의 원리와 개념

3.1.1 게임 그래픽 콘텐츠의 특징

모바일 게임이든 대규모 게임이든 모든 게임은 사용자가 게임 진행 상황을 시각적으로 확인할 수 있게 하는 다양한 그래픽 요소를 가지고 있다.

여러 전문가가 게임용 그래픽 오브젝트를 만드는 과정에서 각자의 역할을 수행한다:

역할	책임
아티스트	게임에서 사용되는 다양한 그래픽 콘텐츠 제작.
3D 모델러	게임 오브젝트의 3D 모델 제작.
텍스처 디자이너	게임 오브젝트의 텍스처 제작.
애니메이션 전문가	게임 오브젝트의 애니메이션 작업.
테크니컬 아티스트	기술적 테스트를 수행.
테스터	비디오게임 엔진에서 그래픽 오브젝트 테스트.

게임 테스터는 게임 예산과 그래픽 콘텐츠의 양에 관계없이 각 게임에 부분적으로, 전체적으로 또는 개별적으로 존재하는 다양한 영역을 염두에 두고 테스트해야 한다.

테스팅을 위해 고려해야 하는 게임의 주요 영역은 다음과 같다.

레벨(게임 맵)

레벨은 게임 내 가상 세계의 독립된 영역으로, 일반적으로 건물이나 도시와 같은 특정 장소를 나타낸다. 이 용어는 초기 TRPG(tabletop role-playing games)에 있었던 (게임의 대부분이 진행됐던) 던전의 레벨에서 유래했다. 플레이어는 던전 깊숙한 곳(레벨 1)에서 시작하여 지상에 도달하기 위해 점점 어려워지는 레벨을 거쳐 마지막 레벨(예: 레벨 100)을 통과하면 게임에서 승리했다. 현재는 어떤 형태로든 거의 모든 비디오게임에 레벨이 존재한다.

모델

모델은 컴퓨터 그래픽 오브젝트를 말한다. 이미지를 만드는 방법에 따라 그래픽을 다음처럼 나눌 수 있다:

- 2D 그래픽
- 3D 그래픽
- 컴퓨터 생성 이미지 (CGI, Computer generated images) 그래픽

2D 그래픽

2D 컴퓨터 그래픽은 그래픽 정보의 표현 방식과 그에 따른 이미지 처리 알고리즘에 따라 분류된다. 일반적으로 2D 컴퓨터 그래픽은 벡터(vectors)와 래스터(rasters)로 구성되지만, 프랙털(fractal)을 사용한 이미지 표현 방식도 있다.

3D 그래픽

3D 그래픽은 3D 공간의 오브젝트를 사용한다. 3D 컴퓨터 그래픽은 영화와 컴퓨터 게임에 널리 사용한다.

3D 그래픽은 다음이 될 수 있다:

- 폴리곤(다각형, Polygonal)
- 복셀(3D 화소, Voxels)

폴리곤 그래픽은 3D 컴퓨터 그래픽 및 볼류메트릭 모델링(volumetric modeling)에서 다면체 오브젝트의 모양을 정의하는 꼭짓점, 모서리 그리고 면의 모음이다. 면은 일반적으로 렌더링(rendering)을 단순하게 하는 삼각형, 사각형 또는 기타 단순한 볼록 다각형(convex polygons)을 사용한다.

복셀은 6개의 직사각형 폴리곤을 합쳐 놓은 가상 요소이다. 가상 세계의 모든 것(즉, 픽셀, 폴리곤 및 복셀)은 물리적 화면에 픽셀로 표시되어야 한다. 복셀 그래픽은 래스터 그래픽과 유사하다. 오브젝트는 3D 모양의 집합, 대부분의 경우에 큐브로 구성된다.

컴퓨터 생성 이미지(CGI) 그래픽

CGI는 계산을 기반으로 컴퓨터가 획득한 3D 이미지로 시각 예술, 인쇄, 텔레비전 및 시뮬레이터에서 영화적인 특수 효과를 내는데 사용한다. 동영상 이미지는 CGI 그래픽의 세부 영역인 컴퓨터 애니메이션으로 만들어진다.

텍스처

텍스처는 폴리곤 모델의 표면에 적용된 비트맵(bitmap)으로 색상 또는 돌출 효과(relief illusion)를 제공한다. 넓게 봤을 때 텍스처는 조각(sculptural) 이미지 표면의 패턴으로 생각할 수 있다. 텍스처를 사용하면 폴리곤으로 생성하기에는 자원적으로 비효율적인 작은 표면 오브젝트, 예를 들어 피부의 흉터, 옷의 주름, 벽과 흙 표면의 작은 돌과 기타 오브젝트를 재현할 수 있다.

텍스처 표면의 품질은 최소 텍스처 단위당 픽셀 수를 나타내는 텍셀(texels)에 의해 결정된다.

텍스처 테스트의 주요 목적은 시각적 오브젝트의 텍스처 존재 유무 및 표시 정확성과 균일성을 테스트하는 것이다. 게임 텍스처의 해상도가 높다면 낮은 품질의 텍스처는 없어야 한다

충돌(Collisions)

충돌은 서로 충돌하는 오브젝트를 처리하는 특성이다.

컴퓨터 게임, 특히 콘솔 게임은 제한된 하드웨어 자원과 매우 제한된 게임플레이 시간에 많은 작업을 분배해야 하는 어려움이 있다. 뿐만 아니라 비교적 원시적이고 부정확한 충돌 감지 알고리즘으로 충돌을 개발해야 한다. 그럼에도 불구하고 게임 개발자들은 게임 오브젝트 간의 상호작용을 눈으로 보기에 믿을 수 있고 비교적 사실적인 방법으로 표현할 수 있었다.

대부분의 컴퓨터 게임에서 충돌과 침입을 피해야 하는 주요 오브젝트는 레벨의 풍경과 환경이다. 여

기에는 산, 나무, 건물, 울타리와 같은 정적이고 상호작용하지 않으며, 파괴할 수도 없는 구조물이 포함된다. 이 경우 캐릭터는 하나의 점으로 표현되며, 공간의 이진 분할 방법(즉, 유클리드 공간을 볼록 집합과 초평면으로 재귀적으로 분할하는 방법)이 사용된다. 결과적으로 오브젝트는 관찰자로부터의 거리에 따른 시각적 오브젝트 정렬 및 충돌 감지를 포함하여 3D 컴퓨터 그래픽에 대한 작업을 효율적으로 수행하기 위해 사용되는 데이터 구조 형태로 제공된다. 이는 캐릭터를 나타내는 점이 환경(지형)에 있는지를 테스트하는 실행 가능하고, 간단하면서 효과적인 방법을 제공한다. 캐릭터와 다른 동적 오브젝트 간의 충돌은 별도로 고려하고 처리한다. 충돌에 대한 자세한 내용은 3.1.2절에서 찾을 수 있다.

애니메이션

모델에 애니메이션을 적용하기 전에 "본(skeleton)"이 만들어진다. 이 과정을 리깅(rigging)이라고 한다. 애니메이션이 필요한 모든 생명체와 오브젝트 모델은 가상 본(skeleton)을 가질 수 있다. 예를 들어, 주인공의 망토를 들 수 있다. 모델의 본(skeleton)은 손을 움직이면 손바닥의 뼈도 움직이는 것처럼 서로 의존적이다. 모든 애니메이션은 리깅과 스킨닝(skinning)이 얼마나 잘 됐는지에 따라 달라진다. 애니메이션은 움직이는 이미지, 즉 물체의 움직임 및 모양 변경을 구현하는 환상을 만드는 기술이다. 모핑(morphing)은 빠른 주기(손으로 그린 애니메이션의 경우 초당 12프레임, 컴퓨터 애니메이션의 경우 초당 30프레임)로 서로 교차하는 정적 이미지(프레임) 시퀀스를 사용한다.

시각효과(Effects)

시각효과는 두 가지 주요 유형으로 나눌 수 있다:

- 게임플레이 효과 (상호작용 효과)
- 자연 효과 (환경 효과)

적용되는 원칙은 프로젝트에 따라 다르다.

게임플레이 시각효과는 여러 장르 중에서도 격투 게임과 PRG에서 더욱 중요하다. 예를 들어, 플레이할 수 있는 캐릭터가 다른 플레이어의 플레이 가능한 캐릭터를 밀면 각각의 올바른 상호작용 애니메이션이 재생되어야 한다. 슈팅 게임 등 다른 게임 장르도 있으며 게임플레이만큼 자연 효과가 중요한 특히 사실적인 장르도 있다. 자연 효과의 예로는 폭포, 안개, 비 등이 있다.

조명(Scene Lighting)

플레이어가 장면을 보려면 조명이 필요하다. 빛은 감정에 영향을 미친다. 이미지와 감정적 반응 사이의 연결고리는 캐릭터, 플롯, 사운드, 게임 메커니즘 구축에 사용할 수 있는 또 하나의 강력한 수단을 제공한다. 이 경우 빛과 표면의 상호작용은 밝기, 색상, 대비, 그림자, 또 기타 여러 효과에 영향을 줄 수 있다.

인간의 눈은 구조적 특성상 110° 이내에서 3차원으로 물체를 인식하고, 색깔의 경우 더 좁은 범위에서 인식한다. 인간이 가장 먼저 인식하는 것이 중심 시야(앞에서 언급한 범위 내)이기 때문에 디자인

너가 의도한 대로 플레이어가 확실히 보아야 하는 중요한 요소는 여기에 두어야 한다. 주변 시야는 정황을 제공하고 중심 시야를 보강한다. 비디오게임에서 주변 시야에 속한 요소가 필요한 정황을 제공하지 않거나, 중심 시야에 있는 요소와 모순되면 디자이너와 플레이어 간의 연결이 끊어지게 된다. 다음은 조명의 올바른 사용에 대한 예이다:

- 중심 시야로 떨어지는 빛이 플레이어를 안내한다.
- 조명은 프레임을 바꿀 수도 있다. 예를 들어, 손전등을 사용하면 그것이 플레이어의 주요 광원이 된다. 변경된 관점은 밝혀진 영역에 플레이어의 시야를 잡아 두고 강한 대비로 다른 모든 것을 볼 수 없도록 한다.
- 조명은 두려움과 같은 분위기를 연출하고 조성할 수 있다. 예를 들어, 어둠 속 어딘가 끔찍한 적이 숨어 있을 때 플레이어는 일정한 긴장 상태를 유지할 수 있다.
- 빛의 방향은 게임 레벨에서 특정 요소를 찾는 것을 더 쉽게 하거나 더 복잡하게 만들 수 있다.
- 조명을 부족하게 또는 과다하게 주는 방법을 통해 플레이어가 특별한 아이템 사용이나 행동을 하도록 강요할 수 있다.

조명은 게임의 미학을 만들고 게임 경험에 영향을 미치기 때문에 가장 중요한 벨리데이션(validation) 대상 중 하나이다. 따라서 시각 예술, 영화, 건축에서 볼 수 있는 많은 조명 기술이 컴퓨터 게임 가상 공간의 미학을 보완하고, 플레이어의 경험을 향상하는 데 사용된다. 그러나 영화나 연극과는 매우 다르게 게임의 조명 환경은 역동적이고 예측하기 힘들다. 정적 조명 외에 동적 조명도 사용한다. 조명은 상호작용성과 적합한 감정을 더해주는 역할을 한다.

역사적 사실의 정확성(Historical accuracy)

그래픽 요소, 애니메이션, 효과 등 모든 항목은 전체 플레이 스타일과 맞아야 한다. 오브젝트, 모델 또는 위치를 설명하는 텍스트 또는 음성은 이전 프로토타입에서의 설명과 일치해야 한다.

풍부하고 복잡한 그래픽 콘텐츠는 컴퓨터 게임과 다른 소프트웨어의 주요 차이점 중 하나이다. 이것은 테스트 접근법에 큰 영향을 준다. 게임에서 그래픽을 테스트하려면 테스터는 물리학 및 광학 지식도 있어야 하며, 색상 조합 기술 및 역사 지식도 가지고 있어야 한다. 그래픽 테스트가 중요한 테스트 활동 중 하나인 이유는 여기에 대부분의 결함이 집중돼 있으며, 게임에 대한 사용자의 인식과 게임 경험에 직접적인 영향을 미치기 때문이다.

3.1.2 그래픽 콘텐츠 결함 유형

그래픽 테스트에서 많이 식별하는 결함 유형은 시각적 결함이다. 그래픽 결함에는 화면의 이미지 깨짐, 텍스트 없음, 이미지 특정 영역이 예기치 않게 잘리는 문제 등이 포함된다.

모바일 게임용 그래픽과 애니메이션을 만들 때 PC와 동일한 엔진을 사용한다. 차이는 특정 플랫폼과 해당 플랫폼에 사용되는 하드웨어 및 기능에 맞게 엔진을 조정한다는 것이다[Gregory18]. 따라서 이러한 게임에서 발생하는 결함은 비슷하다.

텍스처 누락 (Lack of texture)

텍스처를 테스트할 때 가장 자주 발생하는 결함은 다음과 같다:

- 그래픽 오브젝트의 텍스처 누락
- 게임플레이 중 텍스처 사라짐
- 임시 텍스처 및 스텝(stubs)

과거 경험에 비춰봤을 때 처음 두 가지 유형의 결함은 그래픽 프로세서의 성능 부족 또는 오래된 그래픽 카드 드라이버와 관련된 경우가 많다. 이에 비해 임시 텍스처는 개발 과정에서의 오류로 나타날 수 있다.

세부 표현 수준 (LoD, Level of Detail)

최근 3D 게임에서는 카메라와 (플레이어의 관점에서) 거리가 다른 많은 물체를 동시에 표현하는 경우가 있다. 시스템 부하를 줄이기 위해 개발자는 LoD 기술을 사용한다.

개발자가 새로운 게임 오브젝트(예: 나무, 건물, 자동차)를 만들 때 오브젝트 모델의 여러 변형을 게임에 추가한다. 여기에는 폴리곤이 적고 형상(geometry)이 단순한 모델(로우-폴리 모델, low-poly models)과 폴리곤이 많고 보다 구체적인 모델(하이-폴리 모델)이 포함된다. 카메라까지 거리에 따라 폴리곤 수가 다른 동일한 오브젝트를 모델을 사용하여 표현한다. 카메라에 근접하여 최대 이미지 품질을 달성해야 하는 경우엔 하이-폴리 모델을 사용한다. 카메라가 멀어지면 폴리곤이 적게 사용된 덜 상세한 모델로 대체한다. 충분히 먼 거리에서는 모델이 실루엣으로만 표현되거나, 전혀 렌더링 되지 않는다. 이로써 처리해야 하는 폴리곤의 수를 줄이고, 게임 성능을 높일 수 있다.

예를 들어, 수평선에 있는 숲의 경우 부조와 반사(reflections) 없이 색상만 표현되는 저해상도 텍스처를 사용할 수 있다. 플레이어가 가까이 다가가면 나무가 자세히 그려진다.

LoD 기술은 자동화된 프로그램이 인간 플레이어를 대신하여 게임을 플레이하는 봇의 애니메이션, 본(skeleton), 인공지능에도 사용된다. 예를 들어, 프레임에 다수의 적이 있는 슈팅 게임에서 봇은 다양한 수준의 세부 정보를 갖게 된다. 배경에 있는 봇은 텍스처 수준이 낮은 단순한 방식으로 묘사되는 반면, 근처의 적은 정교한 애니메이션으로 신중하게 그려지고 플레이어와 싸울 만큼 똑똑해 보여야 한다.

개발자는 또한 현재 프레임 속도, 화면에서 오브젝트의 이동 속도 및 동시에 표시되는 오브젝트의 총 수에 따라 세부 표현 수준을 변경한다.

LoD 사용으로 인한 결함은 주로 시스템의 낮은 성능과 관련이 있다. 개발자는 오브젝트의 세부 정보 증가 또는 모델 변경이 플레이어의 눈에 띄지 않고 원활하게 되도록 노력하지만, 항상 그렇게 되지 않는다. 성능이 낮은 하드웨어에서 실행되는 게임의 경우, 로우-폴리 모델이 하이-폴리 모델로 충분히 빨리 교체되지 않을 수 있다. 결과적으로 물체에 접근하는 플레이어는 더 나은 모델로 대체되기 전에 먼저 “영성한” 모델을 보게 될 수 있다.

충돌

충돌은 게임의 오브젝트가 정해진 영역을 가지고 다른 오브젝트와의 충돌에 반응하는 방식이다. 오브젝트 모델이 완성되면 환경 및 다른 오브젝트와 상호작용할 수 있도록 콜리전 메시(collision mesh) 또는 콜라이더(collider)를 생성한다. 이것은 보이지 않지만, 다른 오브젝트와의 충돌을 계산하기 위한 오브젝트의 단순화된 형태이다. 콜라이더는 오브젝트의 물리적 모델이라고 할 수 있다. 플레이어에게 보이지 않지만, 모든 콜라이더의 충돌은 비디오게임 엔진에 의해 처리된다[Buttfield19].

콜라이더는 모델의 메시(mesh)와 모양이 대략 비슷해야 하지만, 상당히 대략적인 근사치로도 충분한 경우가 많다. 건물, 바위, 사고 차량 등 환경에 속한 대부분의 정적 오브젝트의 경우 개발자는 로우-폴리 콜라이더를 사용한다. 이렇게 하면 게임플레이 상으로는 차이를 느낄 수 없지만, 화면에 보이는 모든 오브젝트의 처리 효율성이 향상된다.

이 경우 콜라이더의 크기는, 특히 플레이어가 특정 오브젝트와 직접 상호작용할 수 있는 경우, 보이는 모델과 일치해야 한다.

오브젝트의 콜라이더가 시각적 모델보다 훨씬 작으면 캐릭터가 “텍스처로 들어가거나” 통과할 수 있다. 엄밀히 말하면 “텍스처로 들어간다”(예: 캐릭터가 다른 물체를 통과할 수 있는 경우)라는 문장은 올바르지 않지만, 캐릭터가 다른 오브젝트와 부분적으로 또는 완전히 겹쳤을 때 플레이어가 보게 되는 것을 완벽하게 설명한다.

이러한 장애는 플레이어에게 불공정한 게임 이점을 제공할 수 있으므로, 멀티플레이어 게임에서 가장 치명적이다. 예를 들어, 탱크를 몰아 돌 안으로 들어갔을 때, 다른 플레이어는 보지 못할 수 있다. 하지만, 돌이 적에 대한 공격을 제한하지 않을 수 있다. 그렇기 때문에 테스터는 플레이어 대 플레이어 (PvP) 모드의 맵에서 이러한 오브젝트에 더 많은 주의를 기울여야 한다.

그렇지만 게임의 일부 오브젝트는 장식이며 충돌이 전혀 일어나지 않을 수 있다. 예를 들어, 탱크는 덤불을 쉽게 통과할 수 있으며 플레이어가 작은 돌맹이나 강통을 모두 피하거나 뛰어넘을 필요는 없어야 한다.

게임 메커니즘이 여기에 적용된다. 같은 방식으로 긴 무기를 가진 캐릭터가 좁은 복도에서 여러 방향으로 공격하는 적을 막을 수 있게 될 수도 있다. 현실과 다르게 캐릭터는 원하는 방향으로 자유롭게 회전할 수 있고, 회전할 때 창이 벽을 통과한다는 사실조차 인식하지 못할 수 있다. 또한 캐릭터의 긴 머리가 갑옷의 거대한 견갑을 통과하게 할 수도 있다. 그러나 플레이어가 올라가려고 할 때 갑자기 그것을 막는 낮은 울타리의 존재는 플레이어를 짜증나게 하고 즐거움을 감소시킬 수 있다.

오브젝트의 콜라이더 크기가 보이는 모델보다 크면 캐릭터가 보이지 않는 벽에 기대거나 작은 지지대 위에 공중 부양하는 상황이 발생할 수 있다. 이러한 장애는 의도하지 않은 방법으로 게임을 정복하는 데 사용되기도 한다. 이른바 스피드 러너(speedrunners)는 개발자가 게으르거나 잊어버려서 주변 오브젝트의 콜라이더 크기를 조정하지 않은 위치를 찾게 된다. 그래서 플레이어는 해당 영역의 상당 부분을 우회하여 정복 시간을 줄일 수 있게 된다.

타격 영역(Hit boxes)

캐릭터 또는 게임 오브젝트가 적의 총알/발사체 등으로부터 피해를 볼 수 있다고 가정하면 전투 중 오브젝트 표면의 각 지점의 위치를 계산하고 명중 여부를 판단해야 한다.

복잡한 모양의 오브젝트인 경우 여기에 많은 연산이 필요하고 항상 정확한 것도 아니다.

이러한 계산을 단순화하기 위해 타격 영역의 개념을 사용한다. 예를 들어, 2D 플랫폼 또는 격투 게임에서는 하나 이상의 직사각형을 사용할 수 있다. 그랬을 때 그것들의 상대적 위치도 계산하기 쉽다.

3D 오브젝트의 경우 가장 간단한 형태의 타격 영역은 구체가 된다. 이 경우, 계산을 하려면 중심과 반경만 있으면 된다. 이렇게 하면 특정 시점에 총알이나 다른 물체가 이 구 안에 있는지 쉽게 판단할 수 있다. 하지만, 구체에 비해 빈 곳이 적기 때문에 평행 파이프 타격 영역이 더 적합한 모양으로 밝혀졌으며, 계산도 물체의 중심점과 길이, 너비, 높이 등 세 가지 값만 있으면 된다.

신뢰성을 높이기 위해 모델의 각 영역(예: 팔, 다리, 머리, 꼬리, 심지어 무기까지)에 대한 복수의 타격 영역으로 오브젝트를 묘사할 수 있다.

그러나 모델에서 손상될 수 있는 부분과 피해를 줄 수 있는 부분이 항상 일치하는 것은 아니다. 따라서 일부 게임에서는 동일한 오브젝트에 대해 타격 영역과 피격 영역을 별도로 할당하여 피해를 주는 영역(예: 무기, 주먹 등)과 피해를 받는 영역(예: 머리, 팔다리)을 정의한다. 캐릭터가 피해를 보아야 하는지 여부를 결정하기 위해 무기의 타격 영역과 캐릭터의 피격 영역이 겹치는지 테스트한다.

타격 영역을 다룰 때 테스트의 주요 임무는 공격받은 것으로 보이는 영역이 실제 결과와 일치하는지 테스트하는 것이다.

대규모 게임에서도 플레이어가 자신의 무기가 적에게 닿았지만, 피해를 주지 않았음을 분명히 알 수 있는 순간이 있다. 반대로 타격 영역이 너무 크면 적의 검이 분명히 닿지 않았음에도 캐릭터가 타격을 받을 수 있다.

기타 결함

주요 프로젝트를 진행할 때 모델 제작의 각 단계는 많은 승인과 확인 과정을 거친다. 아티스트, 모델러, 텍스처 아티스트 등이 테스트를 수행한다. 그런데도 테스터는 개발 과정에서 생성된 시각적 결함을 여전히 찾을 수 있다.

역사적 사실과 프로토타입과의 불일치

판타지 또는 공상 과학 게임의 경우, 건갑이 거대한 비현실적 갑옷과 믿기 어려운 물체가 날아다니는 것을 플레이어가 받아들일 수도 있다. 그러나 현실성을 주장하는 게임의 경우, 항공기 프로펠러의 위치가 완전히 잘못되었다고 하거나 특정 전함에는 훨씬 더 많은 함포가 있어야 한다는 사실을 지적하는 세심한 플레이어가 있을 수 있다.

그렇기 때문에 테스터는 테스트 대상이 실제로 어떻게 생겼는지 상상하기 위해 가능하면 테스트 대상의 사진을 적어도 두 장 이상 살펴보는 것이 좋다.

공중 부양 오브젝트

여러 전문가가 동시에 한 장소에 대한 작업을 할 수 있다. 일부는 오브젝트를 추가하거나 제거하고 다른 전문가는 지형(맵)의 형상을 바꾸거나 타일(지형에 적용되는 텍스처)을 수정하고 있을 수 있다. 이 때문에 기존의 "지지대"가 제거되어 오브젝트가 지형에 함몰되거나 공중에 매달리는 결함이 나타날 수도 있다.

텍스처의 시각적 결함

이러한 결함은 큰 맵 및 지형에서 여러 텍스처가 혼합된 위치에서 가장 자주 발견된다. 예를 들어, 잔디 표면이 모래 또는 돌과 연결되는 위치에서 "이음새"가 보일 수 있다. 자세한 내용은 3.1 절을 참조한다.

조명 결함

조명 결함은 다음과 같다:

- 전반적인 조명 결함
- 점광원의 결함
- 탐조등 광원의 결함
- 면적을 갖는 광원의 결함
- 방향성 광원의 결함
- 발광 광원의 결함
- 확산 광원의 결함

최신 비디오게임 엔진에는 광학 법칙에 따라 다양한 유형의 광원을 만드는 고급 기능이 포함되어 있다. 그러나 조명 설정 오류, 개발자 경험 부족 등으로 조명이 이상적이지 않은 게임이 여전히 존재한다. 시각적 구성(빛의 위치, 각도, 색상, 시야, 움직임)은 플레이어가 게임 환경을 인식하는 방식에 큰 영향을 미치며 그들이 감정적으로 빠져들기에 적절한 분위기를 조성한다. 디자이너는 시각적 무결성을 만들어 이에 대처해야 한다[Lee16], [Tavakkoli18], [Romero19].

애니메이션 결함

이 결함은 스키닝이라고 하는 모델에 본(skeleton)을 맞추는 과정에서 발생한다. 모델에 본(skeleton)이 많을수록 더 실제처럼 느껴지는 애니메이션을 만들 수 있다. 주인공의 머리카락 등 다양한 오브젝트를 애니메이션 할 수 있다.

모델의 뼈는 서로 연결되어 있다. 따라서 손이 움직이면 손바닥의 뼈도 움직인다. 이후의 모든 애니메이션은 리깅 및 스키닝 중 모든 것이 얼마나 실제처럼 수행되고 설정되었는지에 따라 달라진다.

디자이너의 실수로 다양한 애니메이션에서 캐릭터의 팔/다리가 길어지고, 일부 모델 구성요소가 나머지 와 "분리되는" 현상이 생길 수 있다. 이러한 오류는 다른 모델과 충돌하는 경우나 모델의 다양한 애니메이션에서 나타나는 경우가 많다.

시각효과(VFX) 결함

게임은 캐릭터의 행동, 이벤트, 자연 현상과 연관된 폭발, 스파크, 연기, 사라짐, 나타남 등을 표현하기 위해 다양한 효과를 사용한다.

애니메이션 작업 시 시각효과는 보통 본(skeleton), 헬퍼(예를 들어, 모델을 만들고 애니메이션을 입히는 과정에서 사용하는 유틸리티 오브젝트), 장면의 오브젝트에 연결된다. 시각효과를 만드는 것은 애니메이션팀과 시각효과 아티스트가 상호작용하는 반복적인 과정이다. 후자는 다음과 같이 애니메이션을 수정하거나 추가하도록 요청받을 수 있다:

- 더 부드러운 검의 궤적을 만들기 위해 키 프레임 추가하기
- 피가 튀는 방향으로 사용되기 위해 헬퍼를 회전시키기
- 카메라 전환을 수정하여 입자 메시(mesh)의 흉한 모습을 방지하기

우선, 이런 효과는 그 효과가 나타나도록 한 이벤트와 동기화하는 것이 중요하다. 예를 들어, 총구에서 나오는 불과 연기는 발사 순간과 총열에서 나오는 총알과 동기화되어야 한다. 그렇지 않으면 이벤트의 사실성이 떨어진다. 이러한 현상은 전통적으로 장애로 간주한다.

VFX 오류를 일으키는 또 다른 문제는 프레임 속도 유지와 효과를 올바르게 표시하는 데 필요한 하드웨어 자원 최적화와 관련된 기술 조건을 준수하지 않는 것이다. 효과의 규모가 아무리 크더라도 화면에 먼지구름이나 폭발이 멈춰 있다면 플레이어가 그것을 보고 기뻐하진 않는다. 그렇기 때문에 게임의 특수 효과에는 엄격한 기술적 제약이 있으며, 그 목적은 표시되는 프레임 수를 안정적으로 유지하는 것이다.

VFX를 테스트할 때 테스터는 게임 기기의 하드웨어 성능이 최적으로 사용되고 있는지, 효과가 그것을 트리거(trigger)하는 이벤트와 동기화되는지 확인해야 한다.

모델 개발 단계에서 가능한 한 많은 결함을 찾는 것이 바람직하다. 예를 들어, 모델의 폴리곤 수 초과나 텍스처 결함 등은 모델 계산에 사용되는 하드웨어에 걸리는 부하를 많이 늘리게 된다. 그리고 필연적으로 비디오 어댑터 프로세서에 과부하가 발생하도록 해서 게임이 오래 지연되게 한다.

이를 테스트하려면 개발자는 LoD 및 충돌 모델과 같은 요소에 대해 미리 결정되고 배포된 요구사항을 받아야 한다.

일반적으로 테스터는 장애 및 실패를 포함한 이러한 결함의 징후를 찾아야 한다. 오브젝트가 폴리곤이 너무 많은 충돌 모델을 가졌지만 그 오브젝트와 상호작용하는 동안 프레임 속도가 높게 유지된다면 결함을 감지하지 못할 수 있다.

3.2 게임의 그래픽 테스트 접근법

3.2.1 아티스틱(Artistic) 테스트

그래픽 오브젝트를 만드는 과정은 여러 단계와 리뷰를 거치게 된다. 누군가는 디자인을 만들고 누군가는 그것을 애플리케이션에 통합한다. 그렇기 때문에 결과물이 디자이너가 의도한 것과 다를 수 있다. 테스터는 디자인의 표준 준수 여부를 테스트하고 결함을 식별할 수 있지만, 아티스트가 더 쉽게

알아차릴 수 있는 결함도 있다. 예를 들어, 배경의 투명도가 충분하지 않거나 애플리케이션의 해상도를 변경할 때 의도한 대로 동작하지 않는 결함을 들 수 있다.

반대로, 아티스트가 모든 것을 예상할 수 없으며, 실행 중인 어플리케이션에서만 이미 장애가 되어버린 결함을 인식할 수도 있다. 따라서 중요한 결함을 제거하기 위해 구현 후 디자이너가 애플리케이션을 리뷰할 수 있도록 제공하는 것이 좋다. 이 과정에서 테스터는 필요한 환경을 준비하고, 테스트 계획을 만들고, 테스트 후 결함을 설명한 다음 이를 테스트하거나 리뷰를 위해 아티스트에게 보낼 수 있다.

콘텐츠가 많은 프로젝트에는 그래픽 테스트에 참여하는 별도의 전문가가 있는 경우가 많다. 주로 모델을 만드는 과정을 구체적으로 이해하는 아티스트들이다. 다음 모델 생성 단계를 확인한다:

- 형상 생성
- 텍스처 생성
- 충돌 모델 생성

이 작업의 목표는 비디오게임 엔진으로 내보내기 전에 모델의 품질을 향상하는 것이다. 예를 들어, 형상 다각형(즉, 다각형의 수), 위치 정확성, 모델 전개 정확성, 너무 긴 다각형의 존재 여부, 모델의 이름 새와 같은 측면을 테스트한다.

아티스틱 테스트는 가장 단순한 형상과 텍스처를 만드는 것부터 모델을 엔진으로 내보내고 맵에 배치하는 작업까지, 다양한 개발 단계에서 수행할 수 있는 크고 복잡한 작업이다. 아티스틱 테스트를 수행하기 위해 테스터는 특별한 도구나 콘텐츠 에디터가 필요하지 않은 경우가 많다. 게임을 실행하면서 직접 실행할 수 있다. 그러나 도구 및 콘텐츠 에디터를 사용할 수 있다면 결함을 훨씬 빠르고 효율적으로 식별할 수 있다.

아티스틱 테스트는 다음에 의해 수행된다:

역할	아티스틱 테스트에서 책임
아티스트	오브젝트를 리뷰할 때 오브젝트를 관찰하고 평가할 때
3D 모델러	오브젝트를 리뷰할 때
슈퍼바이저	오브젝트를 관찰하고 평가할 때
테스팅 엔지니어	모델을 엔진으로 최종 내보낸 후
플레이어	플레이 테스트에 참여할 때

3.2.2 기술적 테스트

기술적 테스트는 다음과 같은 그래픽의 기술 변수와 관련된 일련의 작업(tasks)이 연관성을 가진다:

- 모델 폴리곤 수 제한 준수 (앞서 언급)
- 텍스처 형식
- LoD 스위칭 거리, 사용할 수 있는 자원을 명세와 비교하는 것으로 충분함

기술적 테스트 자체는 테스터와 기술적 아티스트가 수행한다. 여기에는 다음에서 설명하고 있는 절차가 포함된다:

그래픽 콘텐츠 자원에서 사용하지 않는 파일이나 임시 파일 검색

아티스트가 작업할 때 나중에 사용하지 않는 자원을 실수로 저장소에 추가할 수 있다. 예를 들어, 개발자가 사용하지 않는 자원을 포함한 모든 자원을 저장소에 저장할 수 있다. 이 경우, 게임 클라이언트가 하드디스크에 점유하는 공간의 크기가 늘어나게 되고, 이는 성능에 부정적인 영향을 주고 결국 플레이어에게 전달해야 하는 패치의 수가 늘어나게 한다 (예를 들어, 개별 비디오게임 파일 업데이트). 이를 방지하기 위해 사용하지 않는 자원 검색을 수행한다.

클라이언트의 자원에 필요한 모든 콘텐츠 레코드가 있는지 검색

게임 클라이언트 자원에 그래픽 콘텐츠가 다른 자원과 독립해서 존재하지 않는다. 텍스처에 대한 참조는 모델에 작성되고 모델에 대한 참조는 맵에 작성된다. 그러한 레코드에 가장 작은 결함조차도 예기치 않은 결과를 초래할 수 있다.

텍스처 형식 테스트

각 텍스처 유형에 대해 기술 아티스트는 자체 요구사항을 설정한다. 이는 콘텐츠의 성능과 시각적 구성요소 간의 균형을 유지해야 하기 때문이다. 텍스처 형식은 압축 유형과 크기를 말한다.

콘텐츠 테스트 중 클라이언트 로그 테스트

때로는 클라이언트 하위 시스템이 하는 일이 별로 눈에 띄지 않을 수 있다. 예를 들어, 포스트 효과는 태양을 볼 때 렌즈에 나타나는 것처럼 이미지 위에 겹치는 효과를 말한다. 화면의 중첩에 따라 각각의 포스트 효과에 전용 텍스처가 생성된다. 렌즈를 통해 태양을 볼 때 텍스처가 포스트 효과에 할당되었는지 여부와 관계없이 사용자는 렌즈와 태양 광선을 보게 된다. 이 경우 플레이어는 결함이 있어도 결함을 볼 수 없다. 이러한 결함을 찾으려면 리소스에 존재하지 않는 텍스처를 사용해서 렌더링하려고 했다고 확실히 기록돼 있을 클라이언트의 로그를 조사해야 한다.

텍스처, 모델 수에 대한 제한 준수 확인

각 콘텐츠 유형에는 보통 특정 그룹에 적용되는 고유한 제한이 있다. 예를 들어, 파괴할 수 없는 고층 건물의 경우 최대 15,000개의 폴리곤을 사용할 수 있지만, 단층 건물의 경우 10,000개만 사용할 수 있게 설정돼 있을 수 있다. 콘텐츠 편집은 지속해서 이루어지고 저장소에 다운로드하는 횟수가 많기 때문에 모델을 새로 내보낼 때마다 비디오 프로세서 모델 계산에 필요한 하드웨어 자원 부족 사태를 방지하기 위해 제약 및 요구사항 준수를 테스트해야 한다.

기술적 테스트는 단순한 테스트이지만 그것의 중요성을 이해하는 것이 중요하다. 그래픽은 전체 시스템 성능에 큰 영향을 미칠 수 있다. 따라서 기술적 테스트에는 게임의 성능과 크기가 원하는 수준으

로 유지되도록 하기 위해 성능 테스트가 포함된다. 텍스처와 모델의 수를 늘리면 빌드 크기도 증가한다. 텍스처 형식의 불일치는 프레임 속도를 악화시켜 클라이언트에서 사용하는 메모리를 늘어나게 한다. 기술적 테스트는 아티스틱 콘텐츠가 플레이어에게 최대한 완전하게 전달되도록 한다.

3.2.3 게임플레이 테스트

게임플레이 테스트는 게임플레이에 영향을 미치는 요소를 대상으로 하는 테스트 접근법이다. 이는 시각적 모델과 충돌 모델의 적합성 평가와 게임플레이 오브젝트 설정이 게임 모드의 요구사항을 준수하는지 확인하는 것 등을 포함한다.

오브젝트의 게임플레이 테스트는 다음을 포함한다:

- 체력(HP)이 올바르게 적용되는지 테스트한다. 이것은 롤플레이밍 및 컴퓨터 게임에서 오브젝트를 파괴하기 위해 오브젝트에 가해야 하는 최대 피해량을 결정하는 값이다. HP는 오브젝트의 구성 재료, 크기, 유형을 고려하여 자동으로 할당됨에도 불구하고 여전히 결함이 나타날 수 있다. 따라서 HP 할당은 오브젝트 특성에 따라 수동으로 테스트한다.
- 충돌 모델이 시각적 모델과 일치하는지 테스트한다. 앞서 언급했듯이 충돌 모델은 시각적 모델보다 상당히 단순하다. 이에 따라 플레이어가 시각적으로는 모델 뒤에 숨어 있지만 적이 여전히 피해를 줄 수 있는 결함이 발생할 수 있다.

그래픽 테스트는 복잡한 과정으로 유형에 따라 서로 중첩되는 부분이 있다. 위에서 설명한 테스트 접근법은 게임의 그래픽 콘텐츠가 여러 단계에서 철저하게 테스트되어 그래픽 결함이 식별될 수 있도록 한다.

3.3 그래픽 테스트 실행

3.3.1 오브젝트 개발의 여러 단계에서의 그래픽 테스트 실행

그레이 박스 생성

그래픽 오브젝트의 단순화된 모델로 실제 게임플레이를 테스트하는 데 사용하는 모형이다. 이 단계에서 식별하는 결함은 주로 아티스트가 이 모델을 사용할 때의 편의성과 관련이 있다.

시각적 형상 생성

모델링 및 매핑의 다음 단계는 환경을 대변하는 물리적 오브젝트에 3D 투사체를 매핑 및 생성하는 것이다. 여기에는 공간에서의 형상과 위치가 고려된다. 모델의 가시적 형상 및 UV 전개가 만들어지고 3D 오브젝트 표면의 좌표(X, Y, Z)와 텍스처 좌표(U, V) 간의 대응 관계가 확인된다. 인스펙션은 3D 그래픽 에디터를 사용하여 수행하는 경우가 많기 때문에 이 단계의 결함은 아티스트와 3D 모델러도 식별할 수 있다. 일반적으로 이러한 결함에는 다음이 포함된다:

중복 형상, 중복 폴리곤은 성능을 크게 저하할 수 있다.

- 과도한 세부 사항. 예를 들어, 플레이어의 시야를 벗어나 있는 모델이 표면 무늬를 가진 경우, 플레이어가 알아차리지도 못할 이러한 “장식”을 처리하기 위해 추가 하드웨어 자원이 할당된다.
- 불충분한 세부 사항. 이러한 유형의 결함은 과도한 세부 사항 결함과 반대되는 개념이다. 모델이 플레이어에게 명확하게 보이는 경우 각진 것처럼 보이지 않도록 상세하게 표현되어야 한다.
- 렌더링 되지 않은 일부 형상. 예를 들어, 모델에 필요한 부분이 누락되면 비현실적으로 렌더링 될 수 있다.

모델 텍스처 추가

이 단계에서 텍스처를 아티스트 또는 아트 디렉터가 받아들이게 된다. 이 경우 애플리케이션의 올바르지 않은 텍스처 적용, 이음새 없음 및 당김(즉, 잘못된 폴리곤 분류 체계)이 발견되지 않을 수 있다. 여기서의 테스트는 일반적인 색상 팔레트와 오염 수준(예: 먼지의 존재, 불필요한 색상 삽입, 모델에 있는 다른 색상의 임의 픽셀)과만 관련된다. 이 단계에서 식별하는 보편적인 결함은 자동 매핑 및 후속 텍스처 추가로 서로 겹치는 복수의 모델이 겹치는 부분에서 서로 다른 색상의 텍스처가 적용되어 이음새가 눈에 띄는 경우이다.

오브젝트의 LoD 및 충돌 모델 리뷰

이 단계에서 아티스트와 레벨 디자이너 모두 모델의 품질을 평가하는 데 참여한다. 레벨 디자이너는 충돌 모델이 게임플레이에 미치는 영향을 평가한다. 예를 들어, 특정 위치에서 상세 모델을 사용해야 하는지 또는 단순화할 수 있는지를 평가한다. 여기서 식별하는 보편적인 결함은 너무 상세하거나(예: 충돌 모델에 폴리곤 수가 허용된 것보다 많은) 필요한만큼 상세하지 않은(예: 상세화 정도가 불충분한 충돌 모델을 가진) 모델이다.

비디오게임 엔진으로 모델 내보내기

여기서는 거리에 따른 LoD 전환 설정과 오브젝트의 체력값의 정확성을 고려한다.

테스팅은 테스터가 콘텐츠 에디터와 게임 애플리케이션 모두를 활용해서 게임 엔진에서 직접 수행한다. 동시에 대부분의 아티스틱 테스트는 오브젝트에 대한 시각적 평가로 축소되며 많은 테스트가 이미 아티스트와 모델러가 수행한 테스트와 중복된다. 테스팅은 다음을 다루게 된다:

- 오브젝트가 사용될 위치와 상황을 고려한 오브젝트의 LoD 전환 가시성
- 플레이어의 눈에 띄는 오브젝트 형상의 틈과 이음새
- 다양한 그래픽 오브젝트에 새겨진 글의 가시성
- 모델의 파괴 효과(예: 색상 및 크기를 고려한)
- 오브젝트 애니메이션의 시각적 결함

아티스틱 테스팅에서 플레이어는 콘텐츠를 시각적으로 평가하고 다른 그래픽 결함을 식별하는 데

직접 참여할 수 있다.

그래픽 테스트는 게임 맵 테스트와 함께 수행하는 경우가 많다. 이 경우 몇 가지 단계가 더 존재한다.

맵에 오브젝트 배치

이 단계는 테스트에서 가장 중요한 단계의 하나이다. 오브젝트 배치는 레벨 디자이너가 수행한다. 이 과정에서 발생하는 보편적인 결함은 맵(지형) 아래에 “매달리거나” 움푹 들어간 오브젝트이다. 이것은 지형의 형상을 변경하고, 오브젝트를 배치하고, 맵 자체를 편집할 수 있는 많은 전문가가 동시에 맵을 작업할 때 종종 발생한다.

효과 만들기 및 배치

이 단계에서 효과 아티스트는 새로운 특수 효과를 만드는 작업을 한다. 다양한 상황에서 이를 관찰하면 아티스틱, 기술적 렌더링 결함을 감지할 수 있다. 이러한 테스트의 예로는 엔진에서 효과의 시각적 테스트가 있다. 효과 아티스트는 게임 클라이언트에서 각 효과를 확인하여 잘못된 효과 변수(예: 효과의 불충분한 감쇠 시간)를 식별한다.

그래픽 테스트는 플레이 테스트를 사용하여 수행하는 경우가 많다. 이는 게임 상황에서 맵을 테스트하는 모든 테스트와 관련이 있다. 이 테스트의 목표는 광범위하며 모든 개발 단계(예: 맵에서 게임플레이 평가)에 적용된다. 개발이 끝날 무렵에는 플레이어의 관점에서의 맵에 대한 아티스틱 평가와 게임플레이 평가가 포함될 수 있다. 게임 디자이너가 맵의 밸런스를 조정하는 데 필요한 통계를 수집하는 것 외에도 이러한 테스트는 매달려 있는 오브젝트에서부터 그래픽 렌더링 결함(예: 정지, 결함, 장애)에 이르기까지 다양한 아티스틱 및 기술적 문제를 식별할 수 있다.

맵 테스트

이것은 맵이 완전히 준비되고 더 이상 수정되지 않을 때(드문 예외 제외) 수행하는 테스트의 마지막 단계이다. 테스터는 매달린 오브젝트에서부터 테스트 애니메이션까지 찾는 맵 밸리데이션(validation) 체크리스트를 기반으로 전체 맵에 대한 밸리데이션을 수행한다. 테스터는 아티스트보다 더 많은 결함(예: “매달려 있는” 오브젝트)을 식별할 수 있는 테스트에 중점을 둔다. 아티스트 테스트 중 아티스트는 맵을 시각적으로 검토하는 반면 테스터는 특수한 도구 세트를 사용하여 문제가 될 소지가 있는 사항을 그래픽 오브젝트에서 식별한다.

3.3.2 역사적 사실의 정확도를 위한 그래픽 테스트

그래픽 콘텐츠 테스트의 또 다른 영역은 역사적 사실의 정확도 테스트이다. 역사적 사실의 정확도는 역사적 사실의 정확성과 역사적 사실의 진위를 모두 고려한다.

역사적 사실의 진위는 게임의 품질에 영향을 미치고 특정 역사적인 사건을 특징적인 이미지를 사용하여 설명하는 것과 관련된다. 여기에 특정 사건, 인물 등이 언급되지는 않는다.

역사적 사실의 정확성도 게임의 품질에 영향을 미치며, 특정 역사적 사실의 기간에 대한 설명과

관련된다. 모든 주요 사건은 그 시대의 이미지 특성과 관련된 인물 및 그들의 행동을 포함한 동작으로 설명된다. 시대에 이질적인 이미지는 근본적으로 배제된다.

일반적으로 게임은 중독성 있는 게임플레이를 만드는 것에 초점을 두기 때문에 역사적 사실의 진위를 항상 유지하는 것은 불가능하다. 처음부터 결과가 정해진 게임에 플레이어를 참여시키기는 어렵다. 그러나 역사적 사실의 정확성은, 특히 게임 줄거리가 특정 역사 시대와 연결된 경우 특별히 주의가 필요한 게임 영역이다.

역사적 사실의 정확성을 위해 그래픽 콘텐츠를 테스트할 때 고려해야 할 보편적인 요소는 다음과 같다:

- 역사적 사실의 원형과 캐릭터의 유사성
- 건축 오브젝트를 재현할 때의 정확성
- 특정 시대의 무기, 장비, 차량, 의복과 그 특성을 재현할 때의 정확성
- 일상생활 재현의 정확성
- 역사적인 사건과 날짜의 정확성

역사적 사실의 정확도 테스트를 수행할 때 테스터는 광범위한 지식과 시야를 가져야 한다. 개발자는 최대한 현실적인 게임을 만들기 위해 게임을 테스트할 때 역사 컨설턴트를 참여시키는 경우가 많다.

3.4 그래픽 테스트 지원 도구

콘텐츠를 테스트할 때 그래픽 테스터는 게임 콘텐츠 에디터에서부터 자동화 도구와 테스트 스크립트까지 다양한 도구를 사용한다. 일반적으로 게임은 오브젝트 에디터, 월드 에디터, 조명 에디터 등 그래픽 테스트 도구뿐만 아니라 여러 내장 도구가 포함된 비디오게임 엔진을 사용하여 개발한다.

모든 도구는 플레이어가 상호작용할 수 있도록 기존 콘텐츠를 사용자가 지정해서 수정할 수 있게 하는(즉, 커스터마이징 할 수 있는) 기능을 제공한다. 모델 에디터를 사용하면 엔진과 상호작용할 때 나타나는 다양한 트리거, 사운드, 기타 이벤트 등의 효과를 모델에 결합할 수 있다. 월드 에디터를 사용하면 맵을 만들고, 그 위에 오브젝트를 배치하고, 맵의 게임플레이 메커니즘을 수정할 수 있다. 테스터는 대부분의 테스트를 이 에디터에서 수행하게 된다. 그렇기 때문에 개발자와 동일한 도구를 사용하게 된다. 이러한 에디터는 또한 콘텐츠를 설정하고 테스트를 할 수 있는 광범위한 툴킷(toolkit)을 제공한다.

특별한 도구는 일반적으로 그래픽카드 제조업체에서 개발한다. 이러한 도구를 사용하면 게임에서 프레임 캡처하고 2D 및 3D 그래픽을 만들기 위한 다양한 API 세트를 사용하는 모든 애플리케이션을 자세히 분석할 수 있다. 또한, 다음을 평가할 수 있다:

- 게임 프레임이 준비되는 방식 (예: 형태, 텍스처, 드로우 콜(draw calls))
- 성능 문제가 발생하는 영역
- 달리기의 형태
- 3D 모델러, 그래픽 디자이너, 애니메이션 디자이너를 위한 디버깅

이러한 도구의 사용 방법에 대한 자세한 내용은 해당 소프트웨어 명세에 설명되어 있다.

그래픽 테스트 도구에는 게임에서 특정 경로를 따라 캐릭터가 자동으로 이동하는 기능을 제공하는 자동화 스크립트가 포함되어 있다. 이런 기능은 성능 및 호환성 테스트에 사용한다.

4. 사운드 테스트 - 190 분 (K3)

비디오게임 전용 용어

주변(Ambient), 입체효과(binaural effect), 왜곡(distortion), 지연 노이즈(폴리)(lag noise (Foley)), 폐색(occlusion), 에코(reverb), 스킵핑(skipping), 사운드 단절(sound discontinuities), 음향 효과(sound effects), 사운드 영역(sound zones), 음량(volume)

4장 학습 목표

4.1 게임 사운드 콘텐츠의 특징

GaMe-4.1.1 (K1) 게임 사운드 콘텐츠의 특징을 상기할 수 있다

4.2 사운드 콘텐츠의 결함 유형

GaMe-4.2.1 (K1) 사운드 콘텐츠의 결함 유형을 상기할 수 있다

GaMe-4.2.2 (K2) 사운드 콘텐츠의 결함을 분류할 수 있다

4.3 게임 사운드 콘텐츠 테스트 접근법

GaMe-4.3.1 (K2) 콘텐츠-청각 테스트의 주요 접근법을 요약할 수 있다

GaMe-4.3.2 (K2) 음악과 사운드의 믹스를 테스트하는 주요 접근법을 요약할 수 있다

GaMe-4.3.3 (K2) 음악 작곡 테스트를 위한 주요 접근법을 요약할 수 있다

4.4 사운드 테스트 실행

GaMe-4.4.1 (K2) 오디오-음악 콘텐츠 테스트 레벨을 설명할 수 있다

GaMe-4.4.2 (K1) 클라이언트에 사운드를 통합할 때의 특징을 상기할 수 있다

GaMe-4.4.3 (K1) 사운드 테스트에 대한 책임 영역을 상기할 수 있다

GaMe-4.4.4 (K3) 사운드 테스트 접근법을 적용할 수 있다

4.5 사운드 테스트를 위한 도구 지원

GaMe-4.5.1 (K2) 사운드 테스트 도구의 사용법을 요약할 수 있다

4.1 게임 사운드 콘텐츠의 특징

현대의 모든 비디오게임에서 사운드는 중요한 영역이다. 사운드와 음악은 적절한 분위기를 조성하고 위험을 경고하며, 캐릭터의 감정을 전달하고 게임 세계에 대한 완전한 그림을 보완 및 생성한다.

소리가 전혀 없는 영화나 비디오게임을 접하는 사람은 일정 수준의 불협화음을 경험할 가능성이 높다. 뇌가 청력을 활성화하려고 시도하지만 할 수 없기 때문에 이런 불협화음이 발생하게 되며, 눈앞에서 일어나는 일에 대한 인식 오류를 느끼게 된다.

품질이 낮거나 비현실적, 부적절한 또는 제대로 동작하지 않는 사운드는 플레이어를 짜증나게 하고 게임플레이에 대한 몰입감을 깨뜨릴 수 있다.

비디오게임에서 사운드트랙을 만드는 과정은 다음과 같은 비디오게임의 콘텐츠 구성 복잡성을 고려해야 하므로 상당히 오랜 시간이 걸린다:

- 환경 사운드
- 캐릭터 음성
- 배경 사운드트랙
- 음향 효과
- 다양한 사운드 오브젝트
- 음성 파일

이것은 테스트를 복잡하게 만든다.

4.1.1. 사운드 유형

게임에서 볼 수 있는 사운드 설계는 비디오게임마다 다를 수 있으며 다양한 목적으로 만들어질 수 있다. 어떤 유형의 사운드를 구현할지는 어떤 기능을 수행하는지, 어떤 목적으로 만들어지는지 고려해야 한다.

음악

게임의 주요 음악적 특징은 가장 단순한 게임에서도 잘 나타난다. 예전 8비트 비디오게임에 사용됐던 곡들은 지금까지도 고전으로 취급받고 있다.

음악의 주요 기능은 다음과 같다:

- 순간의 '서사'를 강조한다.
- 이벤트의 역동성을 강화한다.
- 분위기를 높인다.
- 플레이어에게 적절한 분위기를 제공하고 캐릭터의 감정 상태를 설명한다.
- 반복되는 멜로디로 플레이어가 무의식적으로 어떤 감정적 반응을 보이도록 한다.

음향 효과

게임 세계가 영웅이 상호작용할 수 있는 오브젝트로 가득 차 있다면, 각 오브젝트는 실제와 같은 사운드를 가지고 있어야 한다. 이런 사운드는 게임 세계를 생생하고 현실성 있게 만든다.

문 열기, 타격, 무기 사격 및 재장전, 구급상자 사용, 연료 용기 폭발, 유리 깨지는 소리 등 모든 음향 효과는 적절한 소리로 구성되고 필요한 순간에 재생되어야 한다.

작은 게임이라도 개발자는 인터페이스 가능한 요소(버튼, 스위치, 메뉴 항목)와 상호작용하는 음향 효과를 추가하게 된다. 이는 플레이어에게 피드백을 제공하고, 플레이어의 행동이 인식되고 분석된다는 이해를 제공한다.

캐릭터 사운드 (폴리, Foley)

게임 속 캐릭터가 내는 소리로는 옷 바스락거리는 소리, 숨소리, 감탄사, 발걸음, 신음 등이 있다. 폴리라는 용어는 음향 효과의 선구자 중 한 명인 잭 폴리의 이름을 따서 명명되었다. 슈팅 게임 등 일부 게임에서 이러한 소리는 캐릭터가 상처를 입었다는 표시가 될 수 있다. 이것은 팝업으로 표시되는 피해 수치보다 플레이어에게 더 명확한 의미 전달이 될 수 있다.

목소리

목소리는 캐릭터에 의해 재현되지만, 별도로 강조하는 것이 좋다. 오늘날의 게임에서 각 NPC는 전문 성우가 녹음한 고유한 목소리를 가지고 있다. 높은 품질의 녹음된 대사는 캐릭터의 감정을 정확하게 전달하고, 캐릭터와 플레이어 간 감정적인 공감대가 생기도록 할 수 있기 때문에 개발자는 음성 연기에 많은 돈과 노력을 기울이게 된다.

주변 소리

주변 소리는 특정 장소의 특징적인 소리를 말한다. 음악 테마와 마찬가지로 주변 소리는 분위기를 강조하고 플레이어에게 필요한 분위기를 조성한다. 주변 소리는 해당 소리를 내는 구체적인 오브젝트가 없다. 특정 위치, 상황 또는 게임 단계에 단순 수반되는 환경 음이다.

주변 소리는 일반적으로 배경에서 재생되며, 캐릭터의 동작에 영향을 받지 않는다. 이러한 소리의 예로 다음을 들 수 있다:

- 숲의 나뭇잎이 바스락거리는 소리와 새소리
- 바다의 부딪히는 파도 소리
- 술집 손님들의 불명확한 대화 소리
- 경찰 사이렌, 지나가는 차 소리 등

이런 소리는 플레이어가 공간을 탐색하고 게임의 주변 세계를 더 잘 인식하는 데 도움이 된다.

4.1.2. 음향 효과와 기술

사운드트랙은 최신 오디오 장비 및 기술을 활용하여 현실과 같은 효과를 내기 위해 사운드 에디터로 처리되는 경우가 많다.

사용자가 게임의 가상 세계에 빠져들게 하기 위해서는 실제와 같은 사운드만으로는 부족할 수 있다. 상황의 현실성을 높이기 위해 개발자가 일부 사운드를 과장하기도 한다. 이런 기법은 싸우는 장면에서 타격음 외에 싸우는 사람이 움직이는 소리까지도 실제보다 훨씬 크게 들리는 영화 등에서 자주 목격할 수 있다.

이런 효과를 내기 위해 다양한 기법과 기술을 활용한다.

폐색 (Occlusion)

가장 많이 사용하는 기법의 하나는 어떤 장벽(예, 빌딩 벽)을 소리가 통과할 때 나타나게 되는 폐색효과이다. 현실성을 높이기 위해 이런 상황에서 사운드 전문가는 단순히 음량만 낮추지 않고, 어떤 특수 효과를 추가하게 된다.

음원과 청자 사이에 빈 벽이 있을 때, 물체가 서로 다른 방에 있지만 같은 시선상에 있을 때(출입구 양쪽), 물체가 같은 방에 있지만 서로 볼 수 없는 경우(물체 간 기둥이 있을 때) 중심 사운드와 주변 사운드를 다르게 구현한다.

반향 (Reverberation)

반향 효과는 (에코로) 공간의 크기와 깊이를 전달하는 데 사용한다.

이 경우 밀폐된 공간에 있는 음원에서 나는 사운드는 벽에 반사되어 수많은 에코가 발생하게 되고 점차 사라지게 된다. 이 효과는 캐릭터가 동굴, 작은 방, 또는 사운드의 반사가 용이한 표면을 가진 방에 있을 때 잘 동작하게 된다.

고급 사운드 엔진을 사용하면 에코를 만들고 총성, 폭발, 목소리, 발걸음 등의 음량을 높이는 것을 쉽게 할 수 있다.

입체효과 (Binaural effect)

입체효과(라틴어에서 둘을 의미하는 bini와 귀를 뜻하는 auris의 합성어)는 사람이 2개의 귀로 동시에 듣는다는 것과 사람이 머리를 돌리는 경우 사운드가 한쪽 귀에 먼저 도달한다는 사실을 기반으로 한다. 이런 효과를 가진 사운드를 녹음하기 위해 특수 마이크가 사용되며, 그 효과를 체감하기 위해서는 헤드폰을 사용할 필요가 있다. 사운드가 양쪽 귀에 특정한 방향에서 도달하도록 하기 위해 필요하다. 사운드 환경은 플레이어가 게임에 더 깊이 빠져드는 데 도움이 될 수 있다. 입체효과로 인해 플레이어는 사운드가 어느 방향에서 오는지 알 수 있다(예: 적이 어디에서 나타날지).

4.1.3 사운드 영역

프로젝트에 따라 소위 말하는 사운드 영역을 사용해서 위치에 따라 사운드 환경을 나누는 경우도 있다. 예를 들어, 롤플레잉 게임에서 캐릭터가 마을에 들어갈 경우, 마을과 관련된 사운드가 재생되거나 전

락 게임에서 플레이어가 자기 기지에 가까이 갔을 때 자원이 수집되는 소리를 듣게 되는 경우를 들 수 있다.

사운드 영역은 전환점이 있어야 한다. 전환점에 가까워지면 한쪽 영역의 소리가 희미해지고, 다른 쪽 영역의 소리가 나타나야 한다.

사운드 영역 구별을 실생활에서 접할 수 있는 사례로는 방 창문을 여닫을 때를 생각할 수 있다. 창문을 열면 주변 도로의 소음이 명확하게 들리게 되며, 창문을 닫을 경우 그런 소리가 아예 들리지 않거나 상당히 조용하게 들리게 된다.

4.2 사운드 콘텐츠의 결함 유형

사운드 콘텐츠 결함에는 세 가지 유형이 있다.

- 사운드 누락
- 잘못된 사운드
- 사운드 재생 문제

누락된 음향 효과

캐릭터의 특정 행동 또는 특정 순간에 필요한 소리가 나지 않는 경우이다.

잘못된 사운드 재생

오브젝트 또는 환경의 사운드를 잘못 설정했을 때 발생하게 된다. 예를 들어, 대포 발사 소리가 권총 처럼 들리거나, 모터보트가 비행기처럼 들리거나, 캐릭터(게임 내에서, 아니면 게임플레이 밖에서)가 다른 사람의 대사를 말하는 경우를 들 수 있다.

게임 캐릭터가 현대의 말을 쓰거나 최근 영화나 음악을 인용하는 경우, T-34 탱크가 T-72 탱크 소리를 내는 등 음향 효과가 역사적 사실의 정확성을 위반하는 경우도 있다. 드물게 나타나지만, 테스트 중인 게임의 핵심 가치로 사실성과 진정성이 명시되어 있다면 유의미한 결함이 될 수 있다.

올바른 사운드의 잘못된 재생

오디오 드롭 (Audio drop)

이러한 결함이 발생하면 연결 상태가 좋지 않은 전화를 사용할 때처럼 재생된 사운드 중 일부가 사라질 수 있다. 이 때문에 플레이어는 대화에서 특정 문장의 의미를 오해할 수 있다. 이것이 현재 재생 중인 유일한 사운드인 경우 테스터는 문제를 쉽게 찾을 수 있다. 그러나 여러 소리가 동시에 재생되고, 그중 하나가 없는 결함을 식별하기는 훨씬 어렵다.

스킵핑 (Skipping)

음악 또는 오디오 효과에서 발생하는 스킵핑은 사운드 파일 자체의 손상 외에 다른 원인이 있을 수 있다. 스킵핑은 성능 문제로 발생하기도 한다. 이 경우, 프레임 속도에 문제가 생기면 발생하게 된다.

왜곡 (Distortion)

성능 문제로 인해 일부 문장이 왜곡되고, 들리지 않을 수 있다.

재생 지연 (Playback Lag)

게임에서 사운드 설계가 재생 중인 애니메이션, 오브젝트 또는 게임 상황보다 뒤쳐지는 순간이 있다.

사운드 콘텐츠 결함과 가능한 원인 예제

1. 오브젝트/환경 사운드 누락
결함: 금속 표면 위에서 캐릭터가 움직일 때 발소리가 들리지 않고 조용히 움직인다.
원인: 개발자가 표면의 사운드를 설정하거나 연결하는 것을 잊었다.
2. 오브젝트 또는 주변 소리가 너무 크거나 작다.
결함: 해당 위치의 특정 오브젝트가 불타고 있지만 화재로 인한 소음이 너무 크거나, 반대로 매우 조용하여 비현실적으로 보인다.
원인: 효과의 음량이 잘못 설정되었다.
3. 오브젝트 또는 환경의 사운드가 잘못 설정되었다.
결함: 물 위에서 모터보트를 운전하는 데 게임은 경주용 자동차의 소리를 재생한다.
원인: 코드에 오브젝트에 대한 잘못된 사운드 파일이 설정되었다.
4. 대상에 따른 사운드의 위치가 잘못 설정되었다.
결함: 게임에서 망치로 칼을 계속 두들기는 대장장이가 있지만, 망치 소리가 방의 다른 구석에서 재생된다.
원인: 사운드 에디터에서 사운드 효과가 잘못된 위치로 설정되었다.
5. 사운드 반경이 잘못 구성되었다.
결함: 총소리가 반경 몇 미터 이내에서만 들린다. 플레이어가 더 멀리 이동하면 소리가 전혀 들리지 않아 현실적이지 않다.
원인: 사운드 반경이 잘못 설정되었다.
6. 사운드 왜곡
결함: 플레이어는 딱딱거리는 소리와 쉿 하는 소리가 나기 시작하는 것을 경험했다.
원인: 손상된 오디오 파일을 사용하고 있다.
7. 소리의 지속적인 반복
결함: 사운드가 반복되고, 계속 재생된다.
원인: 사운드 반복이 멈춰야 할 때 포인터가 올바르게 설정되지 않았다.
8. "끊김" 형태의 사운드 왜곡
결함: 게임 캐릭터 중 하나가 움직일 때, 움직임의 사운드 설계에 왜곡이 발생했다.
원인: 사운드 조합에 문제가 있거나, 게임 클라이언트가 전반적으로 불안정하여 문제가 발생할 수 있다.

9. 오디오 재생 시간 지연

결함: 컷신(cutscene) 중에 캐릭터가 말하기 시작하지만, 입술 움직임보다 조금 늦게 대사가 재생된다.

원인: 캐릭터의 입술 움직임 애니메이션이 해당 사운드 파일의 시작과 동기화되지 않았다.

4.3 게임 사운드 콘텐츠 테스트 접근법

게임의 사운드 설계는 효과, 캐릭터 음성 또는 음악의 형태로 표현될 수 있다. 게임 오디오 콘텐츠의 테스트는 공식적이거나 비공식적인 방식으로 수행할 수 있다. 사운드 테스트에 대한 공식적인 접근법에는 모든 오디오 파일의 존재 여부, 정확성, 음량 및 무결성 테스트가 포함된다. 비공식 사운드 테스트는 게임의 현실감 또는 분위기 유지에 대한 평가이다. 예를 들어, 공포 장르 게임에서 “무거운” 음악 반주를 제공하여 상황을 강조하고, 주기적으로 플레이어의 긴장 상태를 유지한다.

테스터는 이미 검토된 사운드 오브젝트의 명세에서 테스트 케이스를 도출한다. 이 명세에서 테스터는 사운드가 재생되어야 하는 위치, 게임 장면 및 상황을 명확하게 이해해야 한다. 때에 따라 생성된 사운드 콘텐츠의 세부 정보를 알아내기 위해 오디오 디자이너와 상의해야 할 수도 있다. 테스터가 비디오 게임 자원 에디터를 사용하여 플레이어의 동작 또는 게임 환경과 사운드 파일의 동기화를 테스트 할 수도 있다.

테스터가 수행하는 게임의 사운드 설계 테스트는 여러 단계를 포함한다.

4.3.1 콘텐츠 청음 테스트

테스터는 사운드를 듣고 사운드 에디터 또는 비디오 게임 자원 에디터를 사용하여 생성된 콘텐츠의 오디오 변수를 테스트한다. 모든 사운드 파일은 테스트되어야 한다.

이것은 가장 광범위한 단계의 하나이다. 요구사항에 따라 다음 사운드 특성을 확인한다:

- 오브젝트와 설정이 적합해야 한다. 예를 들어, 캐릭터가 철제 갑옷을 입고 있다면 움직일 때 금속 소리가 직물 소리보다 커야 한다. 다이너마이트의 폭발음이나 바위가 떨어지는 소리가 너무 작거나, 모래 위를 이동할 때 인도를 밟는 소리가 난다면 가상 세계에 몰입하는 것을 방해할 수 있다.
- 배우의 목소리. 말로만 구성된 문장에서 배우의 목소리는 말하는 캐릭터와 게임 설정에 맞아야 한다. 일반적으로 목소리는 배우의 성별과 나이와 맞아야 한다. 예를 들어, 같은 분위기를 유지하기 위해 남부 출신 캐릭터는 현대 아일랜드 사람의 억양이 아닌 남부 출신 사람이 가질 억양을 써야 한다.
- 짜증이 나지 않아야 한다. 다양한 사운드 샘플을 함께 사용하면, 때때로 사람의 귀에 불쾌감을 줄 수도 있다.
- 음량. 하나의 게임 안에서 여러 파일을 사용할 경우, 각 사운드 파일의 음량은 동일해야 한다.

- 음향 효과. 사운드와 음성 해설에 적용된 효과도 올바르게 사용돼야 한다. 장소나 장면이 다양한 정보로 채워지면 음향 효과로 세부 사항을 강조하게 된다. 큰 사무실에서는 주변 시스템 장치가 내는 소리, 프린터의 웅웅거리는 소리, 에어컨 소음, 키보드를 치는 소리, 작업자 간 대화 등을 한 사람이 모두 들을 수 있다.

모든 오브젝트를 신중하게 더빙하면 현실적인 그림을 그리는 데 도움이 된다. 예를 들어, 플레이어가 마을로 들어설 때 소의 울음소리가 반드시 들릴 필요는 없지만, 제재소 앞을 지나갈 때는 제재소의 특징적인 소리를 기대할 가능성이 크다. 숲 속의 작은 개울은 사운드 설계가 없을 수도 있지만, 영웅이 폭포 옆에 서 있을 때 폭포 소리가 안 난다면 결함으로(또는 설정에서 소리가 꺼졌다고) 생각할 수 밖에 없다.

4.3.2 음악과 게임 사운드의 믹스 테스트

프로젝트의 규모와 개발팀의 전문성이 프로젝트마다 다르므로 역할별로 수행하는 기능을 명확하게 구분할 수는 없다. 성우가 필요하지 않거나, 폴리 아티스트(소음 생성 전문가)의 작업을 오디오 디자이너에게 맡기거나, 한 명의 전문가가 직접 샘플을 만들거나 구입하여 클라이언트에 연결하고 그 결과로 재생되는 사운드를 테스트하는 상황이 있을 수 있다.

오디오 디자이너의 역할은 사운드를 만들고 오디오 파일을 녹음하는 것이다. 이러한 오디오 파일은 용도에 따라 테스트되어야 한다. 콘솔 게임은 TV에 내장된 스피커와 홈시어터 시스템 모두에서 올바르게 재생되어야 한다.

고품질의 적절해 보이는 사운드라도 최종 믹싱이 올바르게 되지 않으면 플레이어의 혼란과 짜증을 유발할 수 있다. 예를 들어, 동작의 소리가 동작 자체보다 훨씬 오래 지속되거나, 음량이 적절하지 않을 수 있다.

테스터는 비디오게임 엔진에 내장된 리소스 에디터를 사용하거나, 자기 경험 또는 제공된 문서를 기반으로 완성된 사운드와 새로운 사운드 변경 사항을 클라이언트에 적용했을 때의 정확성을 테스트해야 한다.

게임플레이 중의 플레이어는 다양한 오브젝트에서 재생되는 수많은 사운드와 백그라운드에서 재생되는 음악을 듣기 때문에 모든 오디오 구성 요소가 서로 조화를 이루는 것이 중요하다. 테스터는 프로젝트의 사운드 구성 요소를 평가하고 이상한 점을 찾음과 동시에, 음량이 비현실적인 경우도 식별해야 하며, 자신의 상식을 기반으로 사운드의 품질 및 자연스러움도 함께 평가하는 것이 중요하다. 예를 들어, 게임에서 캐릭터가 문을 열 때 나는 소리를 플레이어가 문이 아니라 캐릭터의 등 뒤에서 듣는 상황을 생각할 수 있다. 테스터가 화면에서 일어나는 일과 관련하여 소리의 적시성을 정확하게 평가할 수 있는 것도 중요하다.

4.3.3 음악 작곡 테스트

작곡 또는 믹싱은 개별적으로 녹음된 트랙으로 최종 녹음을 만드는 단계이다. 이 단계는 원본 녹음

트랙을 선택 및 편집(때로는 복원)하고, 이들을 하나의 프로젝트에 통합하여 각종 효과를 적용하는 오디오 녹음 후속 작업이다. 편집은 독립적인 작업 단계가 되는 경우가 많다.

전자 음악을 사용하는 게임에서 믹싱은 오디오 콘텐츠를 만든 후에 진행된다. 이런 프로젝트의 경우 녹음 단계를 건너뛰는 경우가 대부분이다. 전자 음악을 만드는 것과 믹싱하는 것 사이의 경계가 모호하기 때문이다. 많은 가상 신시사이저가 이미 다양한 사운드 처리 템플릿을 가지고 있기 때문에 프로젝트에서는 이미 부분적으로 믹싱된 사운드를 재차 믹싱하게 된다.

결과적으로 다중 채널 프로젝트가 모노, 스테레오 또는 다중 채널 녹음 형태로 만들어지며, 많은 경우 마스터링이라는 과정을 통해 최종 형태에 도달하게 된다.

마스터링이 완료되면 마스터링 된 음악의 테스트가 시작된다. 테스터는 음악 구성 요소의 결함, 사운드 끊김과 간섭을 식별하기 위해 테스트하고 발견된 오류를 보고해야 한다.

4.4 사운드 테스트 실행

4.4.1 게임 소프트웨어 개발 수명 주기에서 오디오-음악 콘텐츠 테스트 레벨

사운드 테스트는 개발 후반에 시작하는 경우가 많다. 개발자가 게임용 콘텐츠(오브젝트/캐릭터 모델, 맵, 아이템 등)를 만들고 나서, 사운드 구성 요소가 만들어지게 된다. 개발 단계에서 오브젝트에 사용자 지정 애니메이션이 클라이언트에 통합된 완성된 모델이 있고 테스트 준비가 완료된 상태가 되더라도 사운드는 없을 수 있다.

사운드 샘플은 사운드를 만든 전문가(예: 오디오 디자이너, 폴리 아티스트, 음향 효과 전문가)가 미리 테스트해야 한다. 개별 사운드의 테스트에 이 전문가들이 참여하게 되며, 이후 개별 사운드는 클라이언트에 추가로 통합하기 위해 개발로 보내게 된다.

4.4.2 게임에 사운드 통합

사운드 디자인을 게임에 통합한 후 테스터는 다음과 같은 사운드 특성을 평가한다:

- 소리 및 음악을 켜고 끄는 기능
- 음량을 변경하는 기능
- 오디오 파일명의 단일 형식 준수 여부 및 프로젝트 폴더에 올바르게 배포됐는지 여부
- 음원의 위치, 게임 맵에서 소리가 도달해야 하는 거리
- 사운드가 통합된 시스템 성능
- 다양한 오디오 시스템과의 사운드 호환성: 헤드폰, 스피커, 모니터 스피커

4.4.3 참여자의 책임

프로젝트의 음악 및 사운드 설계를 만드는 과정에는 다양한 작업을 수행하는 여러 유형의 전문가가 참여한다:

역할	책임
작곡가	게임의 사운드와 음악 제작에 참여한다.
오디오 엔지니어	녹음, 음향 증폭 및 방송 장비의 작동에 관여한다.
사운드 엔지니어	사운드 전문 지식에서 최고급 전문가로 오브젝트, 게임 장면 및 비디오 게임 전체의 최종 사운드를 결정하는 전문가이다.
오디오 디자이너	녹음된 사운드(샘플)가 있는 원본 오디오 파일에서 콘텐츠를 만들고, 사운드 에디터에서 조정한다.
폴리 아티스트	현실에 존재하는 소리를 더욱 생생하고 풍부하게 만들고, 현실에 없는 음향 효과를 만드는 역할을 한다. 일반적인 사물의 소리 샘플을 녹음하고 그것을 결합하여 레이저 검의 웅웅거리는 소리, 좀비가 으르렁거리는 소리 또는 포털이 열리는 소리와 같은 효과를 만든다.
성우	게임에서 들을 수 있는 목소리가 필요한 캐릭터에 생명을 불어넣기 위한 목소리를 연기한다.
개발자	게임 클라이언트 코드에 만들어지고 수정된 오디오 콘텐츠를 통합한다.
레벨 디자이너	사운드 대상을 레벨별로 분배한 다음, 사운드 소스에 사운드 파일을 연결한다.
내레이션 디자이너	컴퓨터 게임 개발에서 스토리와 내레이션을 담당하는 전문가. 스토리 전개와 관련된 순간(컷신, 인트로, 대화, 게임의 중요한 순간 등)에 재생되는 사운드와 음악을 작업한다.
게임 디자이너	게임 문서(예: 비디오게임 설계 문서)의 개발에 관여한다. 이 문서는 단순한 언어로 게임의 메커니즘과 기능을 설명한다. 따라서 엔진이 개발되기 전에도 게임 디자이너는 게임에 대한 전반적인 비전을 개발하게 된다. 개발 과정에서 그들은 또한 "컨설턴트"의 역할을 부분적으로 수행하며 제안/변경 사항이 프로젝트의 주요 아이디어를 준수하는지 여부를 승인한다. 그들은 또한 게임 테스트에 직접적으로 참여한다.
테스터	완성된 전체 사운드와 클라이언트에서 변경된 새로운 사운드의 정확성을 테스트한다.

4.4.4 사운드 오브젝트 테스트의 테스트 활동 수행 절차 및 접근법

사운드 오브젝트를 테스트할 때 테스터는, 특정 버전의 게임에 사운드가 얼마나 적절하게 설정되었는지에 대한 완전한 정보를 얻기 위해 필요한 절차와 작업을 수행해야 한다. 작업 목록은 테스트해야 하는 오브젝트에 직접적으로 영향을 받는다. 예를 들어, 클라이언트에 추가된 무기의 사운드 설계에 대한 최종 테스트를 위해 테스터는, 클라이언트에서 무기를 사용할 수 없는 경우 무기를 추가하고, 테스트 맵에 새로운 콘텐츠를 불러오고, 무기에 연결된 모든 사운드(발사, 재장전, 다시 재장전, 발사 모드 전환 클릭 등)를 최대한 테스트하는 데 필요한 모든 작업 등의 여러 가지 작업을 수행해야 한다.

게임 내 각 오브젝트마다 테스트 절차와 접근법이 다르다는 것이 중요하다. 예를 들어, 캐릭터가 상호 작용하는 오브젝트가 아닌 환경 오브젝트를 테스트할 경우, 테스터는 사운드 설계의 구현(예: 캐릭터가 다가갈 때 시냇물이나 강물 흐르는 소리가 재생되는지)을 테스트해야 한다. 테스터는 특정 오브젝트에 사용 가능한 모든 사운드에 대한 정보를 어디에서 찾을 수 있는지 이해해야 한다. 또한, 내장된 자원 에디터 및 사운드 에디터를 활용하여 사운드 콘텐츠의 무결성을 테스트하고 캐릭터 동작 및 게임 환경과 동기화해야 한다.

4.5 사운드 테스트를 위한 도구 지원

비디오게임 엔진 오디오 에디터

테스터가 항상 비디오게임 엔진의 오디오 에디터를 사용할 수 있는 것은 아니다. 따라서 콘텐츠 벨리데이션을 위한 소프트웨어 표준 세트를 테스트에 사용하게 된다. 이러한 에디터는 개발팀 내 테스터(있다면) 또는 오디오 에디터를 직접 다루는 전문가(예: 작곡가, 오디오 디자이너 등)가 사용한다. 오디오 디자이너는 테스트에 참여해야 한다. 그들은 레벨 조정, 이퀄라이저 조정, 모든 요소를 함께 또는 개별적으로 믹싱/정렬하는 등의 테스트를 수행하게 된다. 동작하는 게임 버전에 요소가 추가되면, 오디오 디자이너는 그것을 쉽게 가져오고 필요한 경우 편집할 수 있다.

사운드 콘텐츠의 유지보수를 용이하게 하기 위해 모든 사운드 샘플은 서로 다른 사운드트랙에 저장돼 있어야 하며 폴더로 정리되고 이름, 타임스탬프, 버전이 지정되어야 한다. 또한, 사운드 어댑터의 하드웨어 자원에 부담을 주지 않도록 오디오 트랙을 압축된 디지털 형식으로 변환하는 것도 효과적이다.

맵/장소 에디터

테스터는 가끔 개발용 소프트웨어를 사용하는 경우가 있는데, 가장 흔히 사용하는 도구는 맵/장소 에디터이다. 이를 통해 테스터는 오디오 콘텐츠와 클라이언트에 있는 모든 콘텐츠를 테스트할 때 역량을 확장할 수 있다. 게임 개발자는 레벨 맵 상에서 직접 시뮬레이션 된 게임플레이를 지원하는 기능을 맵 에디터에 추가한다. 그렇기 때문에 테스터는 맵 에디터에서 게임 클라이언트로 전환하지 않고도 필요한 테스트를 수행할 수 있게 되므로 시간을 크게 절약할 수 있다.

맵에서 사용할 수 있는 대부분의 음향 효과와 음원은 레벨 에디터를 사용하여 정렬하고 조정한다. 맵 에디터에서 테스터는 클라이언트 위치에서 같은 장소에 있는 다른 오브젝트 또는 캐릭터와 함께 음원의 위치를 볼 수 있다. 따라서 전문가는 에디터를 통해 음원을 테스트할 때 재생되는 사운드와 사운드 전파 거리를 테스트할 수 있다. 전파 거리는 테스터 또는 최종 사용자가 어떤 경계를 지날 때 고정된 사운드 소스에서 발생하는 사운드/음악이 들리기 시작하는 지점을 나타내는 구 모양 또는 “영역”으로 표시된다.

사운드 전파 거리가 “영역”의 형태로 구현되는 경우, 테스터는 “영역”의 경계를 넘을 때 소스에서 사운드가 재생된다는 것을 테스트한다. 더 실제와 같은 소리를 위해 음원에 접근하면 음량이 커지는지

테스트할 필요도 있다. 구 형태일 경우에도 소리 전파 거리 테스트는 전파 영역의 모양에서만 차이가 있다. 에디터는 각 클라이언트의 음원 구성의 바인딩과 위치를 테스트할 수도 있게 한다.

이러한 정보를 통해 테스터는 사운드 콘텐츠의 현실감, 사운드 전파 거리, 사운드의 정확성(위치, 오브젝트, 특성)을 평가하고 결함을 식별하면 개발자에게 알릴 수 있다. 예를 들어, 특정 음원의 잘못 설정된 거리에 대한 결함을 설명할 수 있다. 또는 특정 사운드(즉, 특정 음원)의 음량이 “인접한” 다른 음원보다 너무 큰 경우도 있을 수 있다. 결함을 설명하면서 테스터는 에디터를 통해 특정 문제가 발생하는 위치를 알려줘서 개발자가 문제 영역을 더욱 쉽게 찾을 수 있도록 하여 개발자의 작업에 도움을 줄 수 있다.

5. 게임 레벨 테스트 - 65 분 (K2)

테스팅 용어

플레이 테스트(Playtest)

비디오게임 전용 용어

색상 코딩(Color coding), 비디오게임 레벨(video game level), 접근 불가 구역(inaccessible places), 레벨 에디터(level editor), 레벨 프로토타입(level prototype), 이야기(narrative), 구조적 형상(structural geometry), 트리거(trigger)

5 장 학습 목표

5.1 게임 레벨 설계 원칙과 개념

GaMe-5.1.1 (K1) 게임 레벨의 구성 요소를 상기할 수 있다

GaMe-5.1.2 (K2) 게임 레벨의 보편적인 결함을 분류할 수 있다

5.2 게임 레벨 테스트의 단계 및 실행

GaMe-5.2.1 (K2) 게임 레벨 생성의 여러 단계에서 수행하는 테스트를 요약할 수 있다

GaMe-5.2.2 (K2) 게임 레벨 테스트에 참여하는 전문가의 책임 영역을 비교할 수 있다

5.3 게임 레벨 테스트를 위한 도구 지원

GaMe-5.3.1 (K2) 게임 레벨 테스트를 위한 도구 사용을 요약할 수 있다

5.1 게임 레벨 설계 원칙과 개념

5.1.1 “레벨”이라는 용어와 게임 프로젝트의 장르에 따른 특수성

게임 레벨은 게임의 가상 세계의 독립된 영역으로, 플레이어는 보물을 찾거나 모든 상대를 물리치거나 출구를 찾는 등 특정 작업을 완료해야 한다. 레벨을 완료하는 데 필요한 보편적인 작업(tasks)은 위치와 아주 밀접한 관계를 가진다. 그 외에도 레벨을 완료하는 데 필요한 다수의 작은 작업(tasks)이 별도로 있을 수 있으며, 퀘스트 또는 레벨을 완료하는 데 필수적이지 않은 사이드 퀘스트가 포함된 경우도 있다. 많은 비디오게임 엔진에서 레벨은 로딩 화면으로 구분된다.

레벨 설계는 게임 레벨을 위한 맵, 장소, 작업, 임무, 기타 환경 생성과 관련된 게임 개발 단계이다. 레벨 설계에는 조합해서 의도한 게임 경험을 전반적으로 만드는 게임 오브젝트와 게임 메커니즘의 형상, 플레이어 경로상의 장애물, 게임의 스토리라인, 기타 요소가 포함된다.

레벨을 만들기 위해 일반적으로 레벨 에디터라는 특수한 소프트웨어를 사용한다.

게임 레벨은 하나로 서로 연결된 많은 구성 요소로 구성된다. 주요 영역은 다음과 같다:

구조적 형상

구조적 형상 또는 기본 레벨 형상은 게임 공간의 가장 중요하고 조형적인 요소 중 하나다. 게임 캐릭터가 움직일 수 있는 지형과 표면을 설정하는 것이 구조적 형상이다.

구조적 형상은 또한 캐릭터가 이동할 수 있는 레벨 공간을 제한한다. 예를 들어, 레벨 경계에 통과할 수 없는 산이나 기타 지형 구조물이 있을 수 있다.

레벨의 시작 지점과 종료 지점이 명확하게 표시된 경우, 구조적 형상은 플레이어가 레벨의 끝까지 가기 위해 어디로 가야 하는지에 대한 지침을 제공할 수 있다.

게임이 진행되는 위치에 따라 다양한 요소가 레벨을 형성하는 오브젝트 역할을 할 수 있다. 예를 들어, 도시에서는 건물, 도로, 다리, 지하도일 것이고, 도시를 벗어나게 되면 들판, 언덕, 바위, 계곡, 동굴 등의 자연경관 요소가 있을 것이다.

게임 환경

게임 환경 오브젝트는 레벨 디자이너가 필요한 게임 공간을 더욱 현실적으로 만드는 데 사용한다.

예를 들어, 마을의 거리는 집, 건물 그리고 도로로 구성되어 있으므로 디자이너는 다양한 크기의 오브젝트로 채우게 된다. 오브젝트는 나무, 울타리, 이동 수단과 같은 큰 물체와 우물, 통, 벤치와 같은 중간 크기의 물체, 정원의 식물, 돌 등과 같은 작은 물체로 구별된다.

조명

다양한 유형의 광원을 사용하면 게임 레벨에 원하는 분위기를 넣을 수 있다. 예를 들어, 플레이어를 올바른 방향으로 안내하거나 주의를 끌거나, 상대방으로부터 숨길 수 있다.

사운드 반주

다양한 동작이나 이벤트로 재생되는 배경 사운드트랙 또는 음향 효과. 자세한 내용은 "4. 사운드 테스트"에서 찾을 수 있다.

게임 기능

게임 기능에는 특정 레벨에서 게임플레이를 구성하기 위한 설정 시스템이 포함된다. 설정은 다음과 같은 유형으로 구별된다:

레벨의 통과와 관련된 설정에는 다음이 포함된다:

- 캐릭터와 적의 출현 지점
- 승리와 패배 조건
- 게임의 자동 저장 지점
- 특정 게임 이벤트 또는 스크립트를 실행하는 트리거(trigger) 시스템. 예를 들어, 2 분마다 비행기가 전투 상공을 비행하거나, 캐릭터가 건물에 들어오면 스토리 영상이 재생되거나, 경비원이 보안 구역의 경계를 넘으면 경보가 울리는 것을 생각할 수 있다.

플레이어가 상호작용할 수 있는 오브젝트(예: 문, 떨어진 물건, 파괴할 수 있는 오브젝트, 함정 등)를 설정한다.

캐릭터가 레벨을 벗어나거나 구조물에 갇히는 것을 방지하는 레벨의 물리적 셸(shell)을 설정한다.

일반적으로 게임 기능의 설정은 플레이어에게 숨겨져 있으며 레벨 에디터에서만 볼 수 있다.

5.1.2 레벨 디자인 결함 유형 이해

일반적으로 레벨을 구현할 때 가장 많이 나타나는 결함이 있다. 이러한 결함을 추적하기는 어려울 수 있으므로 한동안 레벨에서 식별하지 못할 수 있다.

일부 전용 맵 에디터에는 맵에서 이러한 결함을 감지하는 기능이 내장되어 있다. 레벨 디자이너는 맵이나 레벨을 만드는 마지막 단계에서 이러한 도구를 사용하는 경우가 많다. 그러나 대부분의 경우 맵에서 결함을 찾는 가장 좋은 방법은 문제를 식별하고 보고할 수 있는 숙련된 플레이어가 테스트하는 것이다.

레벨에서 식별하는 결함의 대부분은 게임 오브젝트의 모양과 위치, 조명, 충돌 모델 등과 관련이 있다. 이러한 결함에 대한 자세한 설명은 "3. 그래픽 테스트"를 참조할 수 있다.

그러나 다음 결함은 레벨 설계와 직접적인 관련이 있다.

지오메트리 (Geometry)

지오메트리 결함은 플레이어의 캐릭터가 레벨/맵의 어떤 영역에 부분적으로 갇힌 상황을 말한다. 일반적으로 이러한 결함은 맵의 가장자리, 돌출된 곳, 경사면에서 발견된다. 플레이어는 캐릭터를 오브젝트 안으로 들어가게 하거나 가까이 다가가다 갇히게 된다. 점프, 굽힘 또는 다른 움직임을 통해 캐릭터가 그러한 상황에서 벗어날 수 있는 경우도 있지만 그렇지 않은 경우도 있다. 그럴 때 플레이어

는 레벨/맵을 다시 시작하거나, 마지막 저장 지점으로 돌아가거나 특수한 버튼/키 조합을 사용하여 충돌 결함이 나타나지 않는 가장 가까운 위치로 이동해야 한다.

예를 들어, 게임 캐릭터가 맵의 형상에 갇혀 빠져나갈 수 없거나, 접근이 불가능해야 할 영역에 들어갈 수도 있다. 레벨 결함으로 특정 플레이어 또는 팀이 더 좋은 위치에서 출발하도록 할 수 있는 멀티 플레이어 게임에서는 이러한 결함을 식별하는 것이 특히 중요하다.

많이 발생하는 또 다른 상황은 갇혔을 때 사망하는 것이다. 즉, 캐릭터가 시각적으로 레벨/맵의 표면 아래로 계속 떨어지는 것이다. 결국 캐릭터가 죽고 레벨을 다시 시작해야 하거나, 공허 속으로 끝없이 떨어져 게임을 다시 시작해야 하는 상황이 발생하게 된다.

접근 불가 구역

비디오게임에서, 특히 멀티플레이어 게임의 맵에서 플레이어가 다른 플레이어보다 좋은 위치를 얻는 장소가 있을 수 있다. 예를 들어, 언덕 위의 기관총 사수는 넓은 사격 영역을 갖게 된다. 그러한 위치를 가장 먼저 취하는 플레이어가 상대방보다 약간의 이점을 얻지만, 이는 개발자의 의도이자 레벨 설계 요소이며 결함이 아니다.

그러나 레벨 디자이너의 실수로 우연히 그러한 장소가 만들어지기도 한다. 개발자의 의도는 해당 구역이 플레이어가 접근할 수 없는 구역이 되는 것이었다. 그러나 고유한 기능(예: 다른 캐릭터보다 높은 점프력)의 도움을 받은 특정 게임 캐릭터가 그 위치에 들어갈 수도 있다. 이러한 방식으로 플레이어는 경쟁자보다 불공정한 이점을 얻을 수 있다. 다른 플레이어는 그것을 감지하거나 방해할 수 없을 수 있다.

복잡한 게임플레이

목표 포인터가 부족하거나 레벨을 완료하는 데 필요한 행동이 불명확한 것과 관련된 문제. 예를 들어, 플레이어는 접근할 수 있는 영역에 있는 모든 적을 물리쳤지만, 더 나아가기 위해 무엇을 해야 하는지 이해할 수 없을 수 있다.

게임 레벨 밸런스

레벨 밸런스 설정에서 많은 결함이 발생한다. 예를 들어, 게임의 현재 단계에서 상대하기엔 너무 복잡한 봇을 상대해야 해서 레벨을 통과할 가능성이 완전히 없어지는 경우를 들 수 있다. 여러 팀이 상대방보다 먼저 맵의 원하는 지점에 도달하려고 하는 다른 게임을 생각해 볼 수도 있다. 여기서 레벨 밸런스 결함은 팀별로 시작 조건이 같지 않을 때 발생할 수 있다.

부적절한 제한 및 메커니즘

이러한 문제는 게임플레이에 영향을 미치지 않으며 게임에 이점을 제공하지 않지만, 게임에 대한 인식과 몰입도를 저하한다.

예를 들어, 플레이어가 캐릭터의 경로를 막는 허리 높이의 울타리를 볼 수 있지만 그것을 극복하는

것은 불가능한 경우를 들 수 있다. 레벨 디자이너가 계획한 대로 이곳은 플레이어가 접근할 수 없는 구역이 되며, 그렇게 의도된 것이다. 그러나 플레이어의 관점에서 볼 때 이 상황은 게임에 대한 몰입을 망치는 결함으로 보인다. 마찬가지로, 막대 사이의 거리가 충분한 것으로 보이지만 캐릭터가 막대 뒤에서 빠져나올 수 없는 상황을 플레이어는 결함으로 생각할 수 있다.

이야기(narrative)

스토리텔링의 전반적인 스타일, 게임의 플롯 위반과 관련된 레벨 결함이다. 이러한 결함은 게임플레이 자체에 미치는 영향은 미미하지만, 사용자의 몰입도와 게임에 대한 전반적인 인식에 부정적인 영향을 미친다. 예를 들어, 줄거리에 의하면 핵 폭격을 받은 도시인데, 도시에 들어선 캐릭터가 온전한 건물을 발견하게 되면 이상하게 느낄 수 있다.

오브젝트의 색상 코딩 변경

게임 중에 사용자가 마주치는 오브젝트는 다양한 속성을 가지고 있다. 일부는 장식이고 플레이어가 상호작용할 수 있는 것도 있다. 속성이 다른 오브젝트를 색상으로 구분하는 것은 레벨 설계의 중요한 부분이다. 따라서 한 번 선택된 색상 코딩은 게임 전체에서 바뀌지 않는 것이 바람직하다.

예를 들어, 플레이어가 레벨에서 총 한 방에 폭발하는 빨간색 통을 발견했다고 해보자. 플레이어가 이러한 통을 다시 발견하면 동일한 동작을 기대하게 된다. 따라서 총으로 쏘아 폭발하게 수 있는 모든 통은 항상 같은 색(일반적으로 빨간색)이어야 한다.

플레이어가 부술 수 있는 상자, 플레이어가 열 수 있는 문, 플레이어가 잡고 올라갈 수 있는 바위와 돌출점 등은 다른 오브젝트와 모양 및 색상이 달라야 한다.

그렇지 않으면 플레이어는 다음에 해야 할 일을 이해하는 데 더 많은 시간을 쓰게 된다.

5.2 게임 레벨 테스트의 단계 및 실행

5.2.1 게임 레벨 설계 및 테스트의 기본 단계

게임 레벨은 게임 메커니즘, 시각적 오브젝트, 사운드트랙, 인공지능, 기타 구성 요소를 결합한 것이다. 각 구성 요소에 대한 테스트 접근법은 이 실라버스의 연관된 장에서 자세히 설명하고 있다.

게임 레벨 테스트는 이러한 모든 구성 요소를 합쳐서 테스트하는 것으로 모든 구성 요소를 하나로 합쳐 놓고 고려하고 테스트하는 시스템 테스트와 유사하다.

게임 레벨 테스트는 레벨을 만드는 과정의 초기부터 시작한다. 테스트 대상, 테스트의 목적과 구현 절차, 다양한 전문가의 책임 영역은 게임 레벨 개발의 단계에 따라 달라진다.

3D 게임을 예제로 레벨을 설계하고 테스트하는 주요 단계를 살펴보자.

게임 레벨 프로토타이핑 (디자이너 블록아웃/그레이 박스)

이 단계에서는 이전에 작성한 스케치를 기반으로 특수 오브젝트를 가지고 만들고자 하는 게임 레벨

을 생성하게 된다. 게임 레벨의 레이아웃을 개략적으로 형성하기 위해 회색 큐브, 구, 원통, 평면 등과 같이 게임플레이에 직접적인 영향을 미치는 요소만 사용한다.

이러한 단색의 대략적인 기하학적인 모양만 사용하면 어떤 장소의 상당 부분을 제거하거나 추가하는 등 레벨의 구조를 빠르게 수정할 수 있으며, 필요한 경우 모든 것을 깨끗하게 지우고 다시 시작할 수 있다. 때로는 이 단계에서 모든 게임 메커니즘을 사용할 준비가 되지 않았기 때문에 디자이너는 계속해서 변화하는 개발 조건에 맞게 레벨을 조정하면서 점진적으로 구현해야 한다.

레이아웃은 모든 비율과 축척을 고려하여 만들어진다. 이를 위해 레벨 디자이너는 게임 디자이너와 함께 게임 캐릭터의 크기 및 기타 변수와 관련된 일련의 지표를 구성한다. 따라서 게임 디자이너는 캐릭터의 점프 거리와 높이, 캐릭터가 취할 수 있는 자세(예: 똑바로 서기, 쪼그려 앉기, 눕기)와 자세에 따라 캐릭터의 크기를 어떻게 바꾸는지에 대한 정보, 또 캐릭터가 미끄러지지 않고 이동할 수 있는 표면의 최대 경사각에 대한 정보 등을 제공한다. 이러한 지표는 오브젝트의 크기와 레벨 환경 요소에 직접적인 영향을 미친다.

레벨 디자이너가 레벨의 크기를 결정하고 오브젝트에 올바른 축척을 설정하면 레벨에 게임플레이를 지원하는 다른 기능을 구현한다. 여기에는 캐릭터의 출현 지점, 게임 이벤트의 발생 지점(예: 트리거) 등이 포함된다.

일반적으로 이 단계의 게임 레벨 개발은 여러 번 반복되며, 그동안 테스트는 문제가 있는 요소를 식별하고 제거하는 데 도움을 준다.

게임 레벨 프로토타입 테스트

프로토타이핑 단계에서는 구체적인 요구사항이 확정되지 않기 때문에 탐색적 테스트 및 기타 경험 기반 테스트 기법이 가장 효과적인 것으로 간주한다.

생성된 레이아웃으로 맵에서의 게임플레이를 테스트하기 위해 첫 번째 플레이 테스트를 진행한다. 첫 번째 플레이 테스트를 수행하는 테스트 참가자는 게임 소프트웨어 개발팀의 테스터와 아웃소싱 테스터가 될 수 있다. 개발자는 테스트 참가자로부터 피드백을 받아 솔루션의 성공적인 도입 여부를 판단하고 필요한 경우 레이아웃에서 오브젝트를 변경, 이동 또는 제거할 수 있다.

또한, 프로타입에 대한 플레이 테스트를 수행하면서 게임 메커니즘과 그것이 모델의 오브젝트 배열 및 크기와 조합되는 방법도 테스트한다.

예를 들어, 레벨에 있는 대피소가 캐릭터를 적으로부터 숨길 만큼 큰지, 표면에 있는 틈새의 너비가 캐릭터가 뛰어넘을 수 있는 정도인지, 캐릭터가 필요한 모든 난간을 오를 수 있는지, 천장이 너무 낮아서 머리를 부딪히지 않는지 등을 확인한다.

레벨의 레이아웃이 가져오는 게임의 복잡성이 원하는 정도인지, 또 게임플레이의 전반적인 느낌이 적절한지도 테스트해야 한다. 예를 들어, 게임 중 많은 적과의 총격전이 예정돼 있다면 레벨에 충분한 수의 대피소가 있어야 한다. 하나의 강한 끈질긴 적(보스)과 싸우기로 되어 있다면, 레벨 공간을 제한하고 불필요한 오브젝트를 제거해야 한다.

지오메트리(Geometry) 프로토타이핑 (아트 블록 아웃/화이트 박스)

배경 아티스트가 만든 임시 3D 모델이 레벨 디자이너가 만든 프로토타입 오브젝트를 대체하고 필요한 오브젝트를 추가한다. 핵심 작업(tasks)은 게임플레이 자체를 변경하지 않고 레벨의 시각적 구성요소를 처리하는 것이다.

그러나 이러한 교체는 게임플레이를 복잡하게 만드는 수준의 결함을 일으킬 수 있다. 예를 들어, 임시 트리 모델이 레벨 출구를 막을 수 있다. 이 경우 레벨은 다시 레벨 디자이너의 손에 들어간다. 그리고 메커니즘이 올바르게 동작하는지, 플레이어가 어디든 갈 수 있는지, 필요한 모든 것을 볼 수 있는지 다시 테스트해야 한다.

형상 프로토타입 테스트

이 단계에서 테스트는 레벨에 아티스틱 그림을 입힌 후에 게임플레이를 방해하는 결함이 없는지 확인하기 위해 수행한다.

이러한 결함의 예로는 레벨 통로를 막는 임시 3D 모델 바위가 있는 경우, 또는 플레이어의 시야를 차단하는 나뭇가지 등을 생각할 수 있다.

이러한 문제를 식별하기 위해서 다시 플레이 테스트를 수행하여 게임 메커니즘을 테스트하게 된다. 여기서 오브젝트 모델의 크기와 위치가 요구되는 지표 및 게임플레이 정확성과 일치하는지 검증한다. 이와 병행하여 오브젝트의 모양, 조명, 환경 등에 대한 테스트를 수행할 수 있다(자세한 내용은 "3. 그래픽 테스트" 참조).

최종 버전 만들기

이 단계에서 레벨 디자이너는 전체 개발팀의 소프트웨어를 하나로 통합한다. 생성된 모든 콘텐츠로 하나의 플레이 공간을 만든다. 이 단계에서 장식 정보가 레벨에 추가되고 오브젝트의 모양, 목적, 위치가 현실과 유사해진다. 예를 들어, 종이, 문구류, 컴퓨터 모니터가 책상 위에 등장하고 플레이어는 그것을 부수고 책상에서 던져버릴 수 있게 된다.

최종 버전 테스트

이 단계에서는 성능 및 호환성 테스트를 수행한다. 레벨의 성능을 다양한 기기에서 테스트하고 초당 프레임 수를 측정하고, 또 다양한 거리, 조명, 그림자 조건에서 오브젝트의 시각적 모델 표현 품질도 테스트한다.

플레이 테스트의 하나로 테스터는 모든 게임 오브젝트의 충돌 모델 존재 여부, 오브젝트에 물리적 및 시각적 모델이 연결돼 있는지 등을 테스트한다.

예를 들어, 충돌 모델의 테스트는 다음과 같이 진행한다: 테스터는 게임 캐릭터를 어떤 게임 오브젝트(예: 큰 돌) 근처로 옮겨서 여러 방면에서 "돌 위로 걸어가기"를 시도한다. 돌의 충돌 모델이 적절하게 구성된 경우 캐릭터의 몸은 시각적으로 돌에 닿을 뿐, 돌을 통과하거나 돌 속으로 완전히 들어가지는 못해야 한다.

5.2.2 참여자의 책임 영역

다양한 전문가가 게임 레벨 또는 맵을 만드는 작업에 참여한다. 작업을 병렬로 수행하는 경우가 많으면, 따라서 다양한 결함이 나타난다.

높은 품질의 제품을 얻고자 하는 관점에서 이러한 전문가의 책임과 역할을 고려해보자:

역할	책임
게임 레벨 디자이너	레벨의 형상과 모양, 오브젝트와 트리거 포인트 위치, 대피소의 크기 등을 담당한다. 이 모든 것은 핵심 게임플레이를 지원하고 게임에 대한 사용자 참여를 촉진해야 한다.
아티스트	아티스틱 관점에서 맵의 모든 오브젝트는 올바른 텍스처, 조명, 기타 시각 효과를 가지고 있어야 한다. 이것은 이미지의 모양(어두운 영역과 밝은 영역의 올바른 배치)과 사실감 모두에 적용된다. 예를 들어, 콘크리트 블록으로 만든 오두막이 숲 속에 있다면 이상하고, 어울리지 않게 보일 것이다.
테스터	일반적으로 테스터는 모든 오브젝트가 이미 배치되고 구성된 레벨 및 맵의 최종 버전을 가지고 작업한다. 테스터의 주요 임무는 플레이어가 기술적, 아티스틱 결함을 대면하지 않고 주어진 맵에서 플레이할 수 있는지 테스트하는 것이다. 맵의 모든 오브젝트와의 상호작용을 테스트한다: ● 오브젝트가 물리적 모델(충돌 모델)을 가지고 있는가? ● 보이지 않는 장애물이 있는가? ● 맵 밖으로 나갈 수 없는가?

5.3 게임 레벨 테스트를 위한 도구 지원

레벨 또는 맵의 테스트를 용이하게 하기 위해 3D 에디터, 레벨 에디터, 비디오게임 엔진을 포함한 다양한 도구를 사용한다.

그러나 테스터는 레벨을 만든 개발자와 동일한 도구를 사용한다.

비디오게임 엔진에는 일반적으로 다양한 테스트 및 결함 식별을 위해 맵과 오브젝트를 사용자 정의할 수 있는 기능이 있다. 자체 개발한 엔진은 테스트의 효율성을 높이기 위해 테스트팀의 요청에 따라 수정할 수도 있다.

예를 들어, 레벨 표면의 형상 변화로 인해 일부 물체가 공중에 떠 있거나 표면 밑으로 가라앉을 수 있다. 이러한 결함은 모든 오브젝트를 눈으로 검사하여 식별할 수 있으며, 맵 에디터가 지원하는 경우 특수 알고리즘을 써서 테스트를 더욱 쉽게 할 수 있다.

또한, 에디터에서 여러 오브젝트의 텍스처 표현을 비활성화하면 테스터는 맵에서 통과할 수 없는 장소를 찾을 수 있으며, 맵 디자이너가 의도하지 않은 주행 가능한 경로를 찾을 수 있다. 이것은 멀티플레이어 게임에서 특히 중요한데, 일부 플레이어가 다른 플레이어보다 의도하지 않은 이점을 얻을 수 있기 때문이다.

6. 게임 컨트롤러 테스트 - 95 분 (K2)

테스팅 용어

준수(Compliance), 인체공학 테스트(ergonomics testing), 기능 테스트(functional testing)

비디오게임 전용 용어

가속도계(Accelerometer), 비디오게임 컨트롤러(video game controller), 게임패드(gamepad), 자이로스코프(gyroscope), 레이싱 휠(racing wheel), 터치스크린(touchscreen), 트랙볼(trackball)

6 장 학습 목표

6.1 게임 컨트롤러의 원리와 개념

GaMe-6.1.1 (K2) 일반적인 입력 장치와 특수 장치를 구분할 수 있다

GaMe-6.1.2 (K2) 다양한 입력 장치의 예를 제시할 수 있다

GaMe-6.1.3 (K1) 다양한 유형의 게임 컨트롤러를 상기할 수 있다

GaMe-6.1.4 (K2) 게임 컨트롤러의 특성과 가능한 발생 원인을 기반으로 게임 제품 결함을 분류할 수 있다

6.2 게임 컨트롤러 테스트 접근법

GaMe-6.2.1 (K2) 게임 컨트롤러를 테스트할 때 커버해야 하는 테스트 컨디션의 예를 제시할 수 있다

GaMe-6.2.2 (K2) 게임 테스트 중 UX 전문가, 테스터, 게임 디자이너의 작업(tasks)을 구별할 수 있다

6.3 게임 컨트롤러 테스트를 위한 도구 지원

GaMe-6.3.1(K2) 게임 컨트롤러의 동작을 테스트할 때의 도구 사용을 요약할 수 있다

6.1 게임 컨트롤러의 원리와 개념

6.1.1 게임 컨트롤러의 유형

게임 컨트롤러는 콘솔 및 컴퓨터 게임에 사용하는 입력 장치이다. 컨트롤러는 일반적으로 콘솔이나 개인용 컴퓨터에 연결한다. 게임 컨트롤러를 사용하여 플레이어는 게임 오브젝트의 움직임과 동작을 제어한다. 이때 제어 대상은 게임에 따라 다르지만, 대부분의 경우 게임 캐릭터 중 하나가 대상이 된다.

일반적인 입력 장치

폰, PC, 콘솔, 슬롯머신과 같은 게임 기기는 최소 다음 중 하나의 장치를 사용할 수 있도록 보장한다:

입력 장치	설명
게임패드	게임 콘솔의 기본 입력 장치이다.
키보드와 마우스	이러한 장치는 개인용 컴퓨터에서 보편적인 입력 장치이기 때문에 컴퓨터 게임에도 일반적으로 사용한다. 일부 게임 콘솔도 마우스와 키보드를 연결해서 게임을 제어할 수 있게 한다.
레이싱 휠	회전용 손잡이와 하나 이상의 버튼이 있는 컨트롤러이다.
트랙볼	바닥에서 반쯤 튀어나온 공처럼 보인다. 손바닥으로 문질러서 공을 회전시킨다.
터치스크린	폰, PDA, 휴대용 콘솔 및 최신 슬롯머신에 사용한다. 특히, 일부 게임에서는 터치스크린을 대상으로 하는 게임플레이 요소가 있다.

특수 장치는 특정 유형의 게임을 대상으로 한 장치이다.

특수 장치	설명
조이스틱	원래 범용 게임 장치였지만 빠르게 진행되는 게임에서 키보드/마우스 또는 게임패드가 선호되면서 조이스틱은 비행 시뮬레이터 장르의 게임을 위한 특수 장치가 됐다. 그러나 습관적으로 게임패드를 종종 "조이스틱"이라고도 한다.
레이싱 게임패드	콘솔에서 레이싱 게임을 단순화하는 데 사용한다. 사실 이것은 운전대(아날로그 가속 및 브레이크 버튼도 있는 경우가 있음)가 추가 내장된 일반적인 게임패드이다. 이러한 장치는 본격적인 레이싱 휠보다 훨씬 저렴하다.
스티어링 휠	민간 비행 시뮬레이터에 사용된다(군용 비행 시뮬레이터는 조이스틱을 사용한다).
페달	근본적으로 디자인이 다른 자동차와 비행 시뮬레이터에 사용된다.
컴퓨터 스톱	엔진 제어 레버라고도 한다. 비행 시뮬레이터에 사용한다.
변속 레버	운전 시뮬레이터용이다.
레이저 건	화면의 물체를 사격할 때 사용한다.
그래픽 태블릿	마우스 대신 커서를 제어하는 데 사용한다.
리듬 컨트롤러	음악 게임에서 기타, 드럼 또는 DJ의 콘솔과 같은 악기를 시뮬레이션하는 데 사용한다.

특수 장치	설명
댄스 플랫폼	춤을 출 수 있는 바닥(슬롯머신 화면과 비슷한 구조를 가짐)이다. 발로 밟을 수 있는 여러 개의 버튼이 있는 플랫폼이다. 이러한 게임의 게임플레이는 춤을 추는 것처럼 보이도록 필요한 버튼을 순서대로 밟는 것이다.
게임용 키보드	특정 게임의 특성에 따라 필요한 버튼과 매크로 생성을 위한 버튼이 추가로 있는 특수 키보드이다.
낙시대 컨트롤러	낙시 시뮬레이터 게임용이다.
마이크	마이크 또는 헤드셋은 추가 입력 장치로 사용하며, 캐릭터와 플레이어가 커뮤니케이션할 수 있도록 비디오게임에 명령을 전달한다.
열차 제어판	철도/전차 차량 제어판 시뮬레이터이다.

모션 캡처 기술

- 2000년대 초반부터 머리 추적 시스템은 비행 시뮬레이터와 거동이 불편한 사람들을 위해 사용해왔다.
- 적외선 센서와 가속도계를 활용해서 공간에서 자신의 위치를 추적하는 원격 제어 장치.
- 3D 공간에서 컨트롤러의 움직임을 추적하고 이미지를 인식하는 카메라.
- 음성 명령, 신체 자세, 표시된 오브젝트 또는 이미지를 활용하는 장치.

6.1.2 게임 컨트롤러 특성과 관련된 결함

컨트롤러와 관련된 결함의 원인은 여러 가지가 있을 수 있다. 결함은 소프트웨어 자체, 여러 컨트롤러 구성 요소의 결함, 심지어 개발자가 제조업체의 컨트롤러 사용 지침을 준수하지 않았기 때문에 발생할 수 있다:

- 오래된 컨트롤러 드라이버
- 컨트롤러 모델과 애플리케이션의 비 호환성
- 개별 장치 또는 생산 회차(batch) 전체의 결함
- 게임플레이와 지침의 불일치

가장 흔한 결함은 게임 중 컨트롤러를 교체할 때 안내 메시지가 부족하거나 전혀 표시되지 않는 것이다. 키 매핑(mapping)은 게임마다 다를 수 있다. 또한, 플레이어가 따로 재지정할 수도 있다.

드라이버 버전이 오래되었거나 드라이버가 없다면 컨트롤러가 예상대로 동작하지 않을 수 있다. 업데이트를 통해 소프트웨어 결함을 제거할 수 있는 경우, 컨트롤러의 기술적 오작동과 문제는 새로운 버전을 릴리스해야만 수정할 수 있다.

그러나 레이싱 휠 또는 기타 컨트롤러의 움직임을 정확하게 인식하지 못하는 오류는 하드웨어 결함이나 소프트웨어 계산 결함으로 인해 발생할 수도 있다. 게임 컨트롤러의 제어 신호를 정확하게 읽는 것이 중요한 게임의 경우, 비디오 게임 설계 문서에 인식해야 하는 값이 간격별로 명시되어 있어야 한다.

또한, 인기 플랫폼에 비디오 게임을 출시할 때 플랫폼 제조업체는 컨트롤러의 게임 내 이미지에 대한 요구사항을 제공할 수 있다. 게임 내 이미지는 게임에서 사용하는 컨트롤러 이미지를 말한다. 예를 들어, 키 설정 메뉴나 게임플레이 도움말 메뉴에 게임패드를 도식적으로 표현한 이미지가 있을 수 있다. 이미지는 게임 내부에만 사용되지 않고 소프트웨어의 포장 박스나 디지털 커버에도 컨트롤러가 표시될 수 있다. 이런 요구사항은 일반적으로 잘 알려진 퍼블리셔나 콘솔과 컨트롤러를 모두 생산하는 제조업체가 요청한다. 기업은 게임 개발자를 위한 테스트 문서에 애플리케이션에 컨트롤러를 표시할 때의 모양과 상표 등을 포함한 전반적인 방법에 대한 요구사항을 제시할 수도 있다.

또한, 컨트롤러의 가속도계와 자이로스코프가 사용되는 비디오게임의 경우 플랫폼 제조업체는 안전 요구사항을 강제할 수 있다. 가장 간단한 예로, 게임플레이를 위해 컨트롤러를 흔들어야 하는 경우를 생각할 수 있다. 내부의 모션 센서는 컨트롤러를 3D 공간에서 사용할 수 있도록 한다. 이러면 게임을 시작하기 전에 사용자가 컨트롤러에 부착된 고정 스트랩을 착용해야 함을 표시해야 한다. 그렇지 않으면 컨트롤러를 갑자기 움직였을 때 사용자의 손에서 미끄러져 주변 장치가 손상되거나 심한 경우 사람을 다치게 할 수도 있다.

6.2 게임 컨트롤러 테스트 접근법

게임 컨트롤러를 사용한 테스트는 일반적으로 게임 기능이 표준 컨트롤러(PC의 경우 키보드/마우스, 콘솔의 경우 게임패드)를 사용할 준비가 된 순간부터 시작한다.

이러한 유형의 테스트에는 다음이 포함될 수 있다:

- PC/콘솔에서 컨트롤러 연결/분리
- 컨트롤러의 배터리 잔량 부족
- 특정 컨트롤러 제조업체에 대한 게임의 지원
- 애플리케이션이 컨트롤러부터 데이터를 수신할 수 있도록 하는 특정 API 지원
- 하나 또는 복수의 컨트롤러 사용
- 컨트롤러의 일반적이지 않은 사용 (네거티브 테스트)
- 진동 (진동 유무 및 강도)

기능 테스트

소프트웨어에 컨트롤러를 입력 장치로 사용해야 하는 경우 애플리케이션의 모든 기능이 연결된 장치와 올바르게 상호작용해야 한다. 사용자는 컨트롤러를 수정하지 않고도 사용자 인터페이스 요소와 게임플레이를 직접 제어할 수 있어야 한다. 할당된 제어 요소: 버튼, 스틱, 레이스 휠 회전, 음성 명령 또는 공간에서의 움직임은 게임 캐릭터의 동작과 일치해야 한다. 또한, 테스터는 컨트롤러가 장치에서 분리되면 가능한 경우 애플리케이션이 일시 중지되는지 확인해야 한다. 게임플레이 구현이 실시간으로 수행돼서 중지할 수 없는 경우, 게임패드 연결을 끊어도 서버와 플레이어의 연결이 끊어지지 않아야 하며 정보 메시지가 함께 제공되어야 한다.

보안 테스트

컨트롤러는 기술적 취약점이 있을 수 있으며 콘솔을 해킹할 수 있는 개방된 접근 지점이 있을 수 있다. 보안 위험을 완화하려면 개발자 모드에 대한 무단 접근 권한 획득, 비행 모드를 사용한 네트워크를 통한 장치 차단 방지 등 보안 테스트를 수행해야 한다[ISTQB_AL_SEC]. 게임 컨트롤러의 보안 테스트에는 소프트웨어 기능에 영향을 미치는 하드웨어 결함을 식별하는 접근법이 혼합되어야 한다.

인체공학 테스트

인기 있는 브랜드의 컨트롤러에 있는 버튼은 용도가 어느 정도 정해져 있다. 게임패드의 X, A와 X, □ 버튼은 애플리케이션에서 상호작용 항목과의 "확인", "선택 확인", 또는 "상호 작용"에 널리 사용되며, B와 O 버튼은 "취소", "거부" 또는 "반환"을 위한 버튼으로 사용된다.

버튼 조합은 소프트웨어로 설정하고 제어한다. 버튼 조합을 테스트할 때 테스터는 다양한 플레이어 그룹(장르, 연령, 장애 등 기준)에 인체공학적으로 편리한지, 즉 플레이어가 여러 버튼을 동시에 또는 순차적으로 누를 수 있는지 주의 깊게 살펴봐야 한다.

게임 컨트롤러의 적합성 테스트

게임을 개발할 때 UI/UX 전문가와 게임 디자이너의 역할은 필수적이다. 이들의 역할은 보기 좋고 기능적으로 편리하며 해당 장르에서 일반적으로 통용되는 관습 또는 표준을 준수하는 인터페이스를 개발하는 것이다. 테스트의 목적은 컨트롤러에서 전송된 입력과 인터페이스에서 수행된 작업의 상호작용이 올바르게 동작하는지 테스트하고 전반적인 UX(사용자 경험)를 테스트하는 것이다.

테스터는 앞서 언급한 것처럼 개발자가 널리 사용되는 컨트롤러 레이아웃을 사용하고 있는지 확인해야 한다(예: X, A와 X, □ 버튼을 "확인" 버튼으로, B와 O 버튼은 "취소" 버튼으로 사용). 키보드의 경우 W, A, S, D 버튼은 방향 버튼으로, Ctrl 또는 C는 웅크리기 또는 포복을 위해, 또 스페이스바는 점프 버튼으로 사용한다. 마우스 왼쪽 버튼의 전통적인 기능은 일인칭 슈팅 게임에서 사격 또는 공격이며, 다른 게임에서는 전략의 선택이 되기도 한다. 마우스 오른쪽 버튼은 일반적으로 일인칭 슈팅 게임에서 조준하거나 전략 게임에서 병력을 이동하는 데 사용한다.

테스터는 어떤 컨트롤러를 사용해도 플레이어가 다른 플레이어보다 월등히 유리하지 않는지 확인해야 한다. 게임패드는 목표물을 조준하는 속도와 정확도 측면에서 키보드/마우스보다 현저히 떨어지기 때문에 적과 추적 대상에 대한 자동 타겟팅을 추가하는 경우가 많다. 조준점은 자동으로 상대에게 "고정"된다. 이 경우, 테스터는 조준점이 어느 정도 거리에서부터 상대를 조준하기 시작하는지, 그리고 상대의 머리처럼 취약한 부분을 조준하는지 확인해야 한다. 머리를 조준할 경우, 추가 데미지를 줄 수 있기 때문이다.

6.3 게임 컨트롤러 테스트를 위한 도구 지원

비디오를 녹화하거나 컨트롤러를 도식적으로 표현해 게임 컨트롤러 테스트를 지원하는 소프트웨어 도구는 다양하다.

캡처/재생 도구를 사용하면 PC 화면을 녹화할 수 있다. 이러한 도구는 결함 보고에 사용하며 결함의 특성과 재현 절차를 보여주게 된다.

또 다른 유형의 도구는 게임패드의 도식적 표현과 명령어를 화면에 보여준다. 이러한 도구는 결함을 식별하고 분석하기 위해 필요한 컨트롤러 동작의 이해를 돕는다.

컨트롤러 입력, 게임패드 스틱의 편향도 외에도 컨트롤러의 진동 성능에 대한 테스트를 지원하는 서비스도 있다([URL3] 참조). 이러한 서비스는 게임패드에 필요한 모든 테스트 도구를 포함하고 있어 컨트롤러 상태를 평가하는 데 도움이 된다.

키보드의 키 입력을 표시하는 특별한 도구도 있으며 테스트 실행에 유용할 수 있다.

7. 현지화 테스트 - 155 분 (K3)

테스팅 용어

준수(Compliance), 국제화(internationalization), 현지화(localization)

비디오 게임 전용 용어

문화적 적응(Cultural adaptation), 역사적 사실의 정확성(historical accuracy), 현장(locale)

7 장 학습 목표

7.1 현지화 테스트의 원칙 및 개념

GaMe-7.1.1 (K1) 현지화 테스트 단계를 인식할 수 있다

GaMe-7.1.2 (K1) 국제화 및 현지화의 주요 목적을 상기할 수 있다

GaMe-7.1.3 (K2) 국제화 및 현지화 역량을 비교할 수 있다

7.2 현지화 결함의 유형과 그 원인

GaMe-7.2.1 (K2) 현지화 결함 및 원인을 분류할 수 있다

7.3 현지화 테스트 접근법 및 실행

GaMe-7.3.1 (K1) 전체 및 부분 현지화 테스트를 인식할 수 있다

GaMe-7.3.2 (K3) 현지화 테스트 유형을 분류할 수 있다

GaMe-7.3.3 (K2) 작가, 편집자, 번역가, 현지화 테스터의 테스트 작업(tasks)을 요약할 수 있다

7.4 현지화 테스트를 위한 도구 지원

GaMe-7.4.1 (K2) 게임 현지화 테스트를 위한 도구 사용을 요약할 수 있다

7.1 현지화 테스트의 원칙 및 개념

7.1.1. 현지화 및 국제화

비디오 플레이어, 게임, 운영 체제 등 모든 제품의 현지화 과정은 유사한 단계, 접근법, 기법을 가지고 있으며 소프트웨어 개발 회사에서 채택하고 있는 고유한 내부 절차에서 차이가 있다[Chandler11].

게임 개발은 코드 작성을 통해 아이디어를 구현하고 필요한 콘텐츠를 보완하여 플레이어에게 출시하는 것이다. 개발 초기에 중요한 단계는 게임을 출시할 시장을 결정하는 것이다.

- 이 단계가 항상 고려되어야 하는 것은 아니지만 이는 향후 게임 판매에 부정적인 영향을 미칠 수 있다.
- 현지 시장에서의 성공적인 판매를 위해서는 현지 버전의 게임이 반드시 필요하다.
- 대부분의 게임은 제품을 목표 시장에 맞게 조정해야 한다는 사실을 감안해서 개발한다.
- 게임을 목표 시장에 맞게 조정하면 매출이 몇 배로 증가할 수 있다.

현지 버전의 게임 개발이 끝난 후 다른 시장에 게임을 출시할 수 있으므로(예: 회사가 다른 시장에 게임을 출시할 자금이 있거나 특정 국가에서 게임을 홍보할 준비가 된 파트너가 나타난 경우), 게임을 개발할 때 팀은 게임을 다른 시장에 추가로 출시할 수 있다는 점을 고려해야 한다. 그렇지 않으면 후속 조정에 막대한 돈과 자원이 필요할 수 있다.

국제화

특정 지역 적용의 리스크와 어려움을 피하기 위해 국제화라는 프로세스를 사용한다. 국제화는 거의 모든 시장에서 사용할 수 있도록 제품을 조정하는 것이고 현지화는 특정 지역에서 사용할 수 있도록 수정하는 것이다.

현지화와 달리 소프트웨어 국제화는 소프트웨어의 후속 번역 및 현지화를 용이하게 하기 위해 개발 단계에서 수행하는 일련의 활동이다.

국제화는 개발 초기 단계에서 수행하며 대부분의 경우 언어학자-번역가의 개입 없이 수행하며 소프트웨어 개발자의 책임이다. 현지화에는 많은 추가 지식을 갖춘 전문 번역가(경우에 따라 원어민)가 참여한다.

국제화에는 다음이 포함된다:

- 현지화 및 국제적 사용에 문제가 없는 방식으로 소프트웨어를 만들고 개발한다. 유니코드를 사용하거나 (필요한 경우) 문자 인코딩 방법을 제공할 수 있다.
- 현지화 과정 이전에는 사용할 기회가 없는 요소를 사용할 수 있는 기회를 제공한다. 예를 들어, 양방향 텍스트를 위한 프레임워크를 DTD(문서 형식 정의) 마크업 언어에 추가하는 것을 생각할 수 있다. 또는 라틴 알파벳이 아닌 타이포그래피 문자 또는 세로 텍스트를 위한 기호를 CSS(Cascade Style Sheet)에 추가하기도 한다.
- 지역적, 언어적 또는 문화적 참조를 지원하는 기능. 여기에는 일반적으로 전문 소프트웨어에 저장된 사전에 정의한 현지화 데이터 또는 이전에 생성한 번역의 사용이 포함된다. 예를

들면, 날짜 및 시간 표시 형식, 현지 달력, 숫자 표시 형식 및 숫자 표시 체계(로마 및 아라비아 숫자), 목록 선택 및 표시, 개인 이름 및 연락처 양식 사용 등이 있다.

- 코드 또는 콘텐츠에서 현지화한 요소를 추출하여 현지화한 버전을 나중에 다운로드하거나 사용자 선호에 따라 선택할 수 있도록 한다. 일반적으로 이는 불러오기가 가능한 언어 팩 형태로 구현한다.

이 목록은 콘텐츠, 프로그램 또는 제품을 다른 언어로 현지화 하는 과정을 포함하고 있지는 않다. 제시된 소프트웨어 개발 기술과 접근법은 향후 현지화를 쉽게 할 수 있도록 한다.

현지화

현지화는 소프트웨어를 해당 국가의 문화에 맞게 조정하는 과정이다. 현지화의 한 가지 구체적인 영역이 사용자 인터페이스, 문서, 관련 소프트웨어 파일을 번역하는 것이다[Retsker81].

게임 현지화에는 다음이 포함된다:

- 게임 대화 및 자막, 도움말, 설명과 메시지, 캐릭터 이름, 오브젝트 이름 등 텍스트 번역
- 화면 보호기, 인터페이스, 메뉴 항목 번역
- 텍스트와 그래픽 다시 그리기
- 배우 선정, 더빙 및 사운드 파일 녹음
- 현지화 된 자료를 게임에 통합
- 프로젝트 웹사이트와 인쇄물 번역 및 적용
- 광고 자료(뉴스, 보도 자료 및 마케팅 자료) 번역
- 현지화 버전 출시 후 게임 지원 (업데이트, 뉴스, 패치)

따라서 현지화 테스트는 게임이 문화, 언어, 종교, 정치, 기타 특성 및 참고 자료를 기반으로 대상 고객에 얼마나 잘 적응했는지 테스트하는 것이다. 일반적으로 문화적, 언어적 측면을 고려하며, 특히 사용자 인터페이스, 문서, 파일을 다른 언어로 번역하는 것과 통화, 숫자, 시간, 전화 번호 등의 형식을 고려해야 한다.

현지화 테스트에는 언어적, 문화적 요구사항과 특정 국가 또는 지역의 특성에 따라 게임의 콘텐츠를 테스트하는 작업이 포함된다. 이러한 유형의 테스트는 최종 버전의 게임이 사용자에게 도달하기 전에 현지화 된 버전에서 현지화 결함이나 번역 문제를 발견하는 데 도움이 된다. 현지화 테스트의 목적은 다양한 시장 및 지역 실정(현장)에 맞게 현지화 된 게임의 여러 버전에서 결함을 찾아 수정하는 것이다.

현지화는 단순히 여러 언어로 번역하는 것이 아니며 현지화 테스트와 언어 테스트는 같은 것이 아니라는 점을 유의해야 한다. 언어 테스트는 주로 맞춤법, 문법, 문체 결함을 찾기 위한 테스트로 구성된다.

7.1.2 게임과 애플리케이션 소프트웨어 현지화의 차이

게임과 애플리케이션 소프트웨어의 현지화 테스트의 차이는 현지화 과정에서 고려되어야 하는 다양한 측면이 이 두 가지 유형의 제품에서 서로 크게 다르다는 것을 이해하는 데에서 비롯된다.

주요 차이는 다음과 같다:

- 대상 고객을 위한 그래픽 콘텐츠 조정
- 오디오 콘텐츠의 현지화
- 텍스트 콘텐츠의 현지화 및 적용
- 장르 및 문학적 특성 준수

일반적으로 많은 양의 그래픽 콘텐츠를 대상 고객에 맞게 조정해야 한다. 이러한 유형의 그래픽에는 표지판, 캐릭터, 게임 오브젝트, 레벨 맵, 기호 및 도구, 게임 아이템, 광고 화면 보호기 등이 포함된다. 오디오 콘텐츠도 신중한 현지화가 필요할 수 있다. 게임의 오디오 콘텐츠를 배우가 대상 고객의 언어로 다시 녹음하고, 콘텐츠 자체도 조정해야 하는 경우가 많다.

혼합 스타일의 텍스트 콘텐츠는 적절한 번역과 조정이 필요하다. 게임에 사용할 수 있는 스타일은 다음과 같다:

- 과학 스타일 (메커니즘, 안내에 대한 설명)
- 저널리즘 스타일 (신문, 잡지, 기사)
- 아티스틱 스타일 (개인 일기, 책)
- 공식 비즈니스 (서류, 각종 문서)
- 일상적인 구어체 (캐릭터의 대화)

예를 들어, 모험 장르(퀘스트)에서는 플레이어가 안내에 따라 어떤 장치를 수리하거나 조립하고 신문에서 필요한 정보를 찾고 사람들을 심문해야 하기 때문에 과학, 저널리즘, 일상적인 구어체 스타일을 사용할 수 있다.

역사적 사실의 정확성, 대상 고객의 종교적, 문화적, 정치적, 이념적 특성 등을 고려하여 게임의 콘텐츠를 의무적으로 조정해야 하는 요구사항도 있다. 경우에 따라 이 요구사항을 위반하면 대상 고객이 제품에 대해 안 좋은 인식을 갖게 되고 특정 국가 또는 지역에서 사용이 금지될 수 있다.

게임을 현지화 할 때는 장르 및 문학적 특성을 준수할 필요도 있다. 예를 들어, 멀티플레이어 RPG의 경우 번역할 때 해당 장르에서 허용되는 플레이어의 종족 이름, 오브젝트 이름 등을 고려해야 한다. 때로는 문체적으로 비슷하고 장르에서 통용되는 개념을 사용하여 콘텐츠를 조정해야 하는 경우도 있다.

게임의 고품질 번역을 위해서는 추가적인 지식과 정보가 필요하다. 예를 들어, 게임에서 다른 게임, 미디어 콘텐츠, 실제 사건을 참조할 수 있다.

다음은 게임 현지화 중 현지화 테스트의 기초가 되는 몇 가지 접근법이다.

7.1.3 현지화 테스트 단계

현지화 테스트에는 번역된 콘텐츠, 다양한 인터페이스 요소, 결함 및 시스템 메시지의 정확성을

테스트하는 것과 자주 묻는 질문 및 도움말에 대한 테스트가 포함된다.

번역 테스트

번역 테스트의 목적은 게임의 다국어 인터페이스에서 번역 결함을 찾고 우편 주소, 이름과 성, 통화, 날짜 및 시간 형식 등의 정확성을 테스트하는 것이다.

때로는 개발 중 숫자와 숫자 값의 형식을 국가 표준(예: 측정 단위)에 맞게 조정해야 한다. 테스트할 때 테스터는 사용자에게 속도, 길이, 무게, 온도 등을 명확하게 알리기 위해 특정 국가에서 어떤 측정 시스템을 채택하고 있는지 항상 기억해야 한다.

“시속 62마일로 이동 중”이라는 메시지는 미터법을 사용하는 국가의 플레이어가 이해하기 어려울 수 있다. 이 경우 속도의 수치를 변경하는 것만으로는 충분하지 않으면 측정 단위도 변경해야 한다.

위의 내용은 게임에서 구매에 사용하는 통화에도 적용된다. 가격 현지화의 가장 간단한 예는 제품을 출시한 지역의 통화로 자동 변경하는 것이다. 환율에 따라 환전하는 것 외에도 해당 지역의 통화 기호도 사용해야 한다. 그러나 애플리케이션을 개발할 때 테스터는 소프트웨어 배포 시 지역에 맞는 가격을 따로 책정할 수도 있다는 사실을 고려해야 한다.

다양한 비디오게임에서 인앱 스토어 물품 가격은 현지 통화로 변환되며 지역마다 달라지기도 한다.

게임을 테스트할 때는 그래픽에 주의를 기울여야 한다. 그래픽은 게임이 출시되는 국가의 현실과 맞아 야 한다. 예를 들어, 도로 표지판은 국가마다 다를 수 있다. 또한, 현지 공휴일을 나타내는 사진과 장면이 추가되어야 할 수 있다. 이슬람 국가에서는 그래픽이 매우 많이 수정된다. 사람과 동물 이미지를 모두 제거하고 아라베스크 문양(기하학적 요소와 식물 요소로 구성된 복잡한 동양 중세 장식 문양)을 추가한다.

중국에서 설날을 축하하는 것은 특히 인기가 있다. 많은 게임에서 인터페이스를 등불, 용, 불꽃 놀이와 같이 국가를 상징하는 문양으로 장식한다. 또한, 중국 신화 속 영웅의 캐릭터 스킨을 쓰거나, 또 폭죽과 로켓이 발사되거나 돼지와 용을 타는 애니메이션과 같은 한정판(제한된 시간 동안 사용 가능한) 콘텐츠가 게임에 추가될 수 있다.

게임에서는 농담도 수정해야 하는 경우가 많으며, 때로는 비디오게임이 출시될 국가의 사고방식에 맞게 줄거리까지 조정해야 하는 경우도 있다.

현지화는 텍스트 작업에만 국한되지 않고 문화적 특성과 도덕적, 윤리적 측면을 조정하는 작업도 포함한다.

일반적으로 현지화 테스트에는 테스터가 가장 주의를 기울여야 하는 몇 가지 주제와 방향이 있다:

- 신비주의와 사탄 숭배를 포함한 종교
- 섹스, 노출이 심한 복장을 한 캐릭터와 저속한 언어
- 문화나 사람들과 관련된 편견과 고정관념
- 전쟁, 군사적 분쟁, 테러
- 역사에 대한 각국의 서로 다른 견해를 포함한 정치

예를 들어, 비디오게임에 코란의 한 구절이 나오는 경우 대부분의 이슬람 국가에서 이 비디오게임을 금지할 가능성이 높다.

현대 사회는 정치적 올바름을 엄격하게 통제하기 때문에 그것의 준수는 진지하게 받아들여져야 한다. 비디오게임 번역가는 게임의 기술적 요소로 인한 어려움이나 게임 현지화를 위한 준비 부족 외에 수준 높은 번역을 수행하기 위한 추가로 필요한 지식 등 번역하는 게임과 무관한 여러 가지 어려움에 직면할 수 있다.

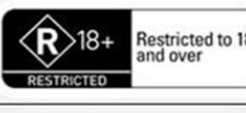
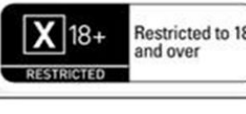
추가 지식에는 다른 게임에 대한 배경지식, 게임에 포함된 미디어 콘텐츠 또는 현실 사건에 대한 지식도 포함된다. 이러한 참고 자료는 이전에 번역된 방식과 맞게 번역되어야 한다. 따라서 번역가는 전 세계에서 일어나는 최신 사건은 물론 다양한 인기 영화와 게임에 대해 알고 있어야 한다.

규정 준수(Compliance) 테스트

게임과 애플리케이션의 연령 제한 시스템은 각국의 법률과 문화 특성을 고려해야 한다. 이를 통해 개발자는 보다 정확하게 콘텐츠 제한을 정의하고, 의도한 대상에게 애플리케이션을 배포할 수 있다.

현지화 테스트를 할 때 지역의 법적 요구사항을 고려해야 한다. 예를 들어, 러시아의 성인 연령은 18세이고 미국은 주에 따라 18세에서 21세까지 다양하며, 일본에서는 20세부터 성인으로 간주한다. 다양한 콘텐츠를 가진 게임은 국가별로 시행 중인 연령 제한 시스템을 따라야 하는 경우가 많다.

같은 게임이라도 국가마다 연령 등급이 다를 수 있다.

ESRB (USA)	PEGI (EU)	RARS (Russia)	ACB (Australia)	USK (Germany)
				
				
				
				
				
				

다양한 국가의 게임 등급

<https://www.kaspersky.com/blog/gaming-age-ratings/11647/>

법률 및 법적 요구사항을 준수하기 위해 현지화를 테스트할 때 여러 상황을 고려해야 한다. 그리고 이 때 단순히 대상 언어만이 아닌 게임이 제공되는 시장을 가진 국가의 법적 그리고 문화적 특성을 고려해야 하는 경우가 있다.

또한, 게임이 각국에서 지원하는 통화와 그것의 운영에도 주의를 기울일 필요가 있다. 예를 들어, 어떤 국가에서 해당 국가의 중앙은행을 제외한 모두에게 외환 거래가 금지되어 있는 경우, 해당 시장에 게임을 출시하려고 하는 게임 퍼블리셔는 심각한 문제를 겪을 수 있다.

간접적으로 통화 거래에는 다양한 캐릭터 스킨, 소모품, 업그레이드 재료 또는 탈것이 들어있는 루트박스, 즉 "상품이 들어있는 박스"의 구매도 포함된다.

2016년 12월에 중국 문화부는 2017년 5월부터 온라인 게임 퍼블리셔가 모든 가상 아이템과 서비스의 획득 확률을 공개하도록 하는 법안을 제정한다고 발표했다.

호주에서는 루트박스를 사용하는 게임은 돈 또는 기타 상품가치 있는 것을 획득할 수 있는 경우, 도박 규제 대상이 된다. 게임 내에만 존재하는 물건이라도 그것의 가치를 해당 물건의 인기와 연관해서만 판단할 수 있는 경우에도 문제가 될 수 있다.

색상과 기호

국가에 따라 의미가 다를 수 있는 색상과 기호도 현지화를 테스트할 때 고려해야 한다.

예를 들어, 빨간색은 중국인에게 인내와 믿음을 의미하며, 인도에서는 순결을 상징한다. 반면에 유럽은 이 색에서 죄와 희생을 본다. 남아프리카 사람들에게는 슬픔의 색이 된다. 미국과 일본에서 빨간색이 위험과 테러를 상징하는 반면, 이집트에서는 애도와 연관된다. 따라서, 색 구성은 중요하며 게임 프로젝트를 테스트할 때 테스터는 그것에 주의를 기울여야 한다.

또한, 테스트를 진행하면서 테스터는 지역별 키보드 레이아웃 및 단축키 지원에 주의를 기울여야 한다.

애플리케이션이 타사 자원(예: 클라우드 저장소 또는 소셜 네트워크)와의 통합을 사용하는 경우, 해당 자원의 지역별 가용성도 고려해야 한다.

따라서, 현지화와 국제화 테스트 프로세스는 다르다:

국제화	현지화
UTF 인코딩	번역
데이터 형식	법적 요구사항
텍스트 방향	통화 및 외환 거래
	색상과 기호
	키보드 레이아웃 및 단축키
	타사 자원과의 통합

7.2 현지화 결함의 유형과 그 원인

7.2.1 게임의 현지화 결함의 가능한 원인

큰 기업이라도 “문화적” 문제로부터 자유롭지 않다. 애플리케이션을 출시할 때 테스터는 선례와 기존 규정을 숙지해야 한다. 경우에 따라 하나의 문구나 농담으로 인해 특정 국가에서 게임의 판매가 완전히 금지될 수 있다. 다음과 같은 현지화 결함의 원인(예: 특정 지역에서 금지된 세부 사항 또는 콘텐츠를)을 고려해야 한다.

- 이미지
- 사운드 및 사운드트랙
- 현실을 그대로 반영하거나 역사적 사실에 기반한 장면
- 문구 또는 인용문

7.2.2. 현지화 결함 및 리스크

대상 언어 또는 제품 자체에 대한 지식이 부족하면 현지화 테스트가 상당히 복잡해질 수 있으며, 특히 제품이 의료, 기술, 스포츠 등 시뮬레이션을 활용하는 장르에 속해 있으면 더욱 그러하다[URL4]. 또한, 다양한 지역의 특성을 연구하는 데 시간이 걸리기 때문에 현지화 테스트는 긴 과정이 될 수 있다는 점을 염두에 두어야 한다.

주요 현지화 결함은 다음과 같이 분류할 수 있다:

기술적 측면

- 모지베이크(깨진 텍스트)/의미를 알 수 없는 문자(Garbage characters). 잘못된 문자 인코딩을 사용하여 텍스트를 디코딩하면 왜곡된 텍스트가 나타날 수 있다. 그 결과 문자가 일괄적으로 전혀 엉뚱한, 많은 경우 완전히 다른 글자 체계의 문자로 대체된다. 보통 이런 경우 텍스트는 읽을 수 없게 된다.
- 현지화 된 문자열이 인터페이스에 설정된 경계에 맞지 않는다(잘림, 스크롤 발생). 한 언어에서 다른 언어로 번역한 용어는 길이가 다를 수 있다.
- 오프셋. 애플리케이션이 현지화 된 후 의도한 정렬을 유지하기 위해서는 사용자 인터페이스 요소의 레이아웃을 다시 구성해야 할 수 있다.
- 대상 언어로 번역되지 않은 텍스트. 테스터는 대화 상자 텍스트, 문서 또는 사용자 인터페이스의 이미지 또는 스크린샷에 주의를 기울여야 한다. 사용자의 기대치를 충족하려면 이러한 모든 콘텐츠를 현지화해야 한다.
- 겹침. 설정창이나 대화 상자의 특정 컨트롤이 다른 컨트롤과 겹칠 때 발생한다.
- 누락된 텍스트. 애플리케이션을 번역하거나 개발할 때 텍스트가 누락될 수도 있다.
- 잘못된 글꼴/크기. 국가마다 서로 다른 기본 글꼴과 크기를 사용한다. 예를 들어, 아시아 국가는 일반적으로 기본 글꼴 크기를 9로 설정하는 반면, 미국은 글꼴 크기를 8로 설정한다.

일반적으로 이 문제가 기능에 지장을 주지는 않지만 텍스트를 읽기 어렵게 만들어 사용자 경험에 큰 영향을 미친다.

- 잘못된 단축키. 대상 언어 또는 키보드에 필요한 문자가 없어 단축키를 사용할 수 없는 경우가 있다.
- 텍스트 템플릿에 있는 변수. 언어 유형(분석적, 합성적)에 따라 여러 형태의 축약, 대소문자, 복수형을 사용하며 어미와 소리가 바뀌거나 바뀌지 않을 수 있으므로, 변수는 단어의 형태에 영향을 미칠 수 있다. 개발자가 가장 흔히 범하는 오류 중 하나는 변수가 포함된 문장을 여러 부분으로 나누는 것이다. 대상 언어의 문법 구조는 현지화에서 가장 중요한 부분이다. 실제로 개발자는 변수를 추가할 때 여러 언어에서 그 의미가 구문의 구성을 바꿀 수 있다는 점을 고려하지 않는 경우가 많다.
- 원본과 번역의 사운드 시퀀스가 일치하지 않는다. 번역할 때 음절 수가 다르기 때문에 캐릭터가 하는 말이 원본보다 길어지거나 반대로 짧아질 수 있다. 게임의 비디오 클립은 사전 녹화한 경우가 많기 때문에 현지화 담당자는 대본 번역에 더운 신중하게 접근해야 한다. 번역을 정확하게 하는 것과 음절 수를 비슷하게 유지하는 것 사이의 균형을 유지하는 동시에 캐릭터의 입술 움직임과 사운드의 동기화(립싱크)도 고려하는 것이 중요하다.

번역 결함

- 고유명사, 날짜, 숫자, 역사적인 사건, 공휴일, 현실, 전문 용어, 욕설, 구어체, 약어, “연상되는” 이름, 별명 등의 부정확한 번역 또는 음역.
- 모순되는 용어. 같은 용어는 애플리케이션 전체에서 일관되게 번역해야 한다. 번역한 용어가 일관되지 않는 경우가 종종 발생한다.

콘텐츠의 문화적 적용

- 유머. 농담, 유머, 풍자에 대한 태도, 또 특정 개체와 연관된 유머가 허용되는지 등을 고려해야 한다.
- 종교. 종교적 장소, 의식, 행사를 포함한 종교적 문제에 대한 청중의 태도. 일반적이지 않은 숭배(사탄주의, 이교도)에 대한 청중의 태도.
- 사건의 역사적 사실에 대한 정확성과 인식. 역사적인 사건, 군사적 갈등, 과학 분야의 발견, 이미 존재하는 특정 객체의 모양과 특성, 사람들의 생활 방식 특성 등에 대한 해석.
- 국가 문화 및 세계관의 특성(어린이에 대한 태도, 성, 폭력, 다른 문화 등 포함). 국가적인 고정 관념, 전통 음식, 의복, 사람들의 생활 방식, 어린이에 대한 태도, 성 소수자에 대한 태도, 동물에 대한 폭력 등.
- 법적 제한. 연령 등급, 아동 보호 분야 법률, 종교적 신념, 폭력, 마약, 성관계, 인종, 테러, 정부 및 국가 시스템에 대한 선동, 특정 기호 사용 금지 등을 고려해야 한다.
- 과도한 현지화. 모든 것을 현지화 할 필요는 없으며 상표, 로고, 약어, 제품명 등 일부 요소는 번역 없이 원래 모습을 유지해야 한다.

7.3 현지화 테스트 접근법 및 실행

7.3.1 전체 현지화 테스트와 부분 현지화 테스트의 차이

전체 현지화 테스트

전체 현지화 테스트는 결함을 찾기 위한 현지화에 대한 철저한 테스트를 의미한다. 이 유형은 고객을 위한 현지화를 새로 개발할 때, 모든 문자열이 번역되고 아직 테스트를 진행한 적이 없는 경우에만 효과적이다. 모든 유형의 테스트가 포함되므로 비용이 가장 많이 든다.

부분 현지화 테스트

이전에 테스트한 현지화에서 텍스트가 변경됐을 때 사용한다. 유효성 검사를 통해 주요 업데이트의 변경사항 중 핵심 스토리에 영향을 주는 텍스트와 그것의 현지화 내용을 식별하게 된다. 수정된 텍스트만 테스트하며, 이 텍스트는 게임의 참조 텍스트와 맞아야 한다. 이 테스트 접근법은 비용을 최소화 하면서 효율성을 달성하는 측면에서 최적의 방법으로 간주된다.

7.3.2 게임 소프트웨어 개발 수명 주기에서의 현지화 테스트 프로세스와 접근법

현지화 테스트는 번역, 지원 파일, 인터페이스 요소의 올바른 정렬 및 적용, 텍스트 작성 메커니즘을 검증한다.

현지화 테스트의 목적은 게임이 다국어 인터페이스와 기능을 지원하는지, 현지화 문제(다른 언어로 번역된 내용, 날짜 및 숫자 형식, 우편 주소, 이름과 성의 순서, 통화 등)는 없는지 확인하는 것이다.

현지화 테스트 프로세스는 다음을 포함한다:

- 지원 언어 목록 결정 및 연구
- 사용자 인터페이스 요소, 시스템 메시지, 결함 등을 포함한 번역의 정확성 테스트
- '도움말'과 관련 문서(있는 경우)의 번역 확인

현지화 테스트에는 현지화 팀에서 번역한 문장과 현지화 참조 문장을 비교하는 작업이 포함되며 다음을 찾고자 한다:

- 문법, 구두법, 구문 결함
- 테스트한 현지화의 지역 요구사항 위반(시간, 날짜, 측정 단위 표기법과 법률 준수 등)
- 필요한 기술 데이터(변수, 텍스트 서식 정보 등)의 부재
- 현지화 참조 내용에 제시된 예술 스타일 및 정황 위반
- 게임 인터페이스의 텍스트 표시 결함(너무 긴 텍스트, 글꼴 표시 형식 위반 등)

테스트팀은 위의 모든 테스트를 수행해야 한다. 그러나 게임이 번역되는 각 언어에 대한 지식을 갖춘 테스터를 찾기가 매우 어렵기 때문에 현지화 대상 지역의 문법, 구두법, 구문 또 예술 스타일과 정황을 테스트하는 것이 항상 가능한 것은 아니다. 이러한 테스트의 실행은 현지화 담당자의 몫이다. 테스터는 테스트된 현지화의 지역 요구사항 위반을 확인하고 필요한 기술 데이터가 부족하지 확인하며, 게임 인터페이스의 텍스트 표시 결함을 식별한다(자세한 내용은 "7.3.3 현지화 테스트 유형"을 참조).

현지화 테스트 준비 단계

이 단계는 다음을 포함한다:

- 테스터에게 필요한 모든 제품 문서 제공.
- 테스터가 사용된 용어를 올바르게 해석할 수 있도록 용어집과 번역 메모리 생성.
- 이전에 이미 현지화 된 제품의 경우 평가 목적으로 이전 버전 제공.
- 현지화 테스트 중 식별된 모든 결함을 기록할 문서 또는 플랫폼인 결함 관리 도구 선택 및 구성.

지역 및 문화적 특성 테스트

현지화 테스트에서 가장 중요한 단계 중 하나다. 스크린샷 또는 현지화 된 게임 빌드를 사용한다.

다음 사항을 테스트해야 한다:

- 선정된 지역의 날짜 및 시간 표시 형식
- 전화번호 및 주소 형식
- 색 구성표
- 제품 이름의 지역 표준 준수
- 통화 표시 형식
- 단위

언어 테스트

언어 기능을 테스트한다. 테스터는 다음을 확인해야 한다:

- 동일한 용어의 사용
- 문법적 결함
- 철자 결함
- 구두법 메커니즘 준수
- 텍스트 방향(오른쪽에서 왼쪽 또는 왼쪽에서 오른쪽)의 적절성
- 브랜드, 도시, 장소, 위치 등 명칭의 정확성

사용자 인터페이스(또는 모양)

다음 사항을 확인하는 것이 중요하다:

- 이미지의 모든 캡션은 현지화해야 한다.
- 현지화 된 버전의 레이아웃은 원본과 동일해야 한다.
- 페이지/화면의 줄 바꿈은 대상 언어의 메커니즘을 따라야 한다.
- 대화, 팝업, 알림이 올바르게 표시되어야 한다.
- 텍스트 길이가 기존 제한을 초과하지 않고 텍스트가 올바르게 표시되어야 한다(때로는 번역된 텍스트가 원본보다 길고 버튼 크기에 맞지 않는 경우가 있음).

기능

현지화 된 애플리케이션이 제대로 동작하는지 테스트할 필요가 있으므로 다음에 주의해야 한다:

- 현지화 된 제품의 기능
- 정보 입력 기능
- 다양한 지역 및 언어의 특수 문자 지원
- 키보드 단축키 지원
- 다양한 폰트 지원
- 다양한 형식의 구분 기호 지원

비디오 및 스케일 테스트는 다음에 대한 테스트를 포함한다:

- 눈금의 길이와 눈금을 사용한 게임 요소의 일치
- 성별에 따른 축척과 캐릭터의 일치
- 사운드의 순도 (즉, 간섭의 부재 외에도 원본과 번역의 음량 동기화 정도)
- 실제적인 측면을 포함한 사운드트랙의 신뢰성
- 소리의 스타일 및 문화적 특성 (억양, 말하는 특징)

7.3.3 현지화 테스트 유형

“포장” 현지화

게임이 물리적 형태로 출시되어 판매되는 경우, 패키지에 적힌 내용도 현지화 된다. 물리적 형태가 아닌 플랫폼에서 판매되는 경우, 스토어의 해당 페이지(설명 및 스크린샷)가 번역된다. 포장 현지화의 범위는 이것으로 제한된다.

인터페이스 현지화

게임에서 인터페이스, 도움말 페이지, 버튼 레이블 등의 설명과 내용은 번역된다. “플레이” 버튼에 표시된 내용이 스토리와 다른 언어로 된 경우 이상한 현지화가 될 수 있다.

텍스트 현지화

일반적으로 자막까지 포함한 게임의 모든 텍스트는 번역된다. 예를 들어, 사용자가 게임에서 듣는 것은 미국 흑인이 사용하는 언어이지만, 보는 자막은 다른 언어로 돼 있을 수 있다.

음성 연기 현지화

말과 대화는 번역되고 배우가 목소리로 연기한다. 음성 연기를 통한 현지화가 적절한 수준으로 이루어지면 이질적이지 않게 인식된다.

그래픽 현지화

모든 게임에는 어떤 형태로든 엔진, 디자인, 그래픽 오브젝트, 텍스처 등에 텍스트가 아닌 글이 (예:

게임 속 울타리에 새겨진 글씨) 포함돼 있다. 그래픽 현지화는 내부의 모든 글씨가 번역돼야 함을 의미한다. 예를 들어, 신문, 상점 간판, 메모 등이 이런 글씨를 가지고 있을 수 있다.

완전 현지화 - 문화적 적응

게임을 완전히 다시 구현하는 경우가 문화적 적응이다. 소스 코드와 메커니즘만 남기게 된다. 개발자는 텍스트, 플롯, 대화, 캐릭터 모델을 다시 만들고, 비디오게임 엔진에 완전히 다른 게임을 만들 수 있다. 매우 드물게 수행되지만 이러한 현지화 방법을 사용하는 경우가 있다. 이는 이러한 수정 없이는 게임을 특정 시장에 팔 수 없고, 해당 지역의 게이머가 이용할 수 없는 경우에 수행한다.

7.3.4 참여자의 책임 영역

역할	책임
작가/이야기 디자이너 (Writer/Narrative Designer)	● 현지화 전략 및 계획 수립 ● 용어의 정의, 게임 장르 전용 개념 및 프로젝트와 관련된 번역이 어려운 용어 번역 ● 편집자 및 번역가에게 문맥 설명
번역가	● 텍스트 번역
편집자	● 번역된 자료의 교정 ● 텍스트가 현지화 대상 언어의 도덕적, 윤리적 측면과 함께 문화적 특성을 준수하는지 테스트 ● 번역 스타일의 일관성 제어
현지화 테스터	● 번역의 정확성 테스트(발언의 문맥과 의미, 텍스트의 일관성, 플레이 스타일 준수) ● 번역의 기술적 준수 테스트(모양, 인터페이스, 폰트의 기능, 구분자, 특수문자 등) ● 번역의 (내부 및 외부 애플리케이션과) 기술적 통합 제어

7.4 현지화 테스트를 위한 도구 지원

현지화 테스트 도구는 테스트의 여러 단계에서 다양한 문제를 해결하는 데 사용할 수 있다.

이러한 도구에는 다음이 포함된다:

- 시각적 문자열 비교 도구. 이러한 도구는 참조 문자열과 대상 현지화 문자열을 비교하는 데 도움이 된다.
- 자동 문자열 비교 도구. 이 유형의 도구에는 보조 스크립트, 프로그램, 유틸리티가 포함되며, 그 결과 테스트 대상 현지화에서 어떤 유형의 결함이라도 그것에 대한 데이터를 얻을 수 있다.
- 누락되거나 중복된 파일이 있는지 현지화 파일 구조를 테스트하기 위한 도구.

- 현지화 테스트를 위한 자동화된 도구의 예는 다음과 같다:
 - 두 가지 버전의 현지화(참조 및 검증) 스크린샷을 비교하여 현저한 차이가 있는 경우 실패를 보고하는 도구
 - 참조에 변경 사항이 있으면 대상 현지화에도 변경 사항이 적용됐는지 확인하는 도구
 - 참조와 대상 현지화 문자열 페어(pair)별로 변수와 숫자 값을 비교하는 도구

8. References

8.1 Standards

[ISO25000] ISO/IEC 25000:2014, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Guide to SQuaRE

[ISO25010] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models

8.2 ISTQB documents

[ISTQB_AL_SEC] ISTQB Advanced Level Security Testing Syllabus, Version 2016

[ISTQB_ALTA_SYL] ISTQB Advanced Level Test Analyst Syllabus, Version 3.1.2

[ISTQB_ALTTA_SYL] ISTQB Advanced Level Technical Test Analyst Syllabus, Version 4.0

[ISTQB_ALTM_SYL] ISTQB Advanced Level Test Manager Syllabus, Version 2012

[ISTQB_CTFL_MAT] ISTQB Mobile Application Tester, Version 2019

[ISTQB_EXAM_S&R] ISTQB Exam Structure and Rules, Game Testing, Version 1.0

[ISTQB_FL_AT] ISTQB Foundation Level Agile Tester Syllabus, Version 2014

[ISTQB_FL_PT] ISTQB Foundation Level Performance Testing Syllabus, Version 2018

[ISTQB_FL_SYL] ISTQB Foundation Level (Core) Syllabus, Version 2018

[ISTQB_GLOSSARY] ISTQB Glossary of Terms used in Software Testing, <https://glossary.istqb.org/>

[ISTQB_UT_SYL] ISTQB Foundation Level Usability Testing Syllabus, Version 2018

8.3 Books

[Nystrom14] Nystrom, R. (2014). Game programming patterns. Genever Benning, ISBN: 978-0990582908. URL: <https://www.gameprogrammingpatterns.com/>

[Gregory18] Gregory, J. (2018). Game engine architecture. AK Peters/CRC Press., ISBN: 978-1466560017. URL: <https://www.gameenginebook.com/>

[Buttfield19] Buttfield-Addison, P., Manning, J., & Nugent, T. (2019). Unity game development cookbook: essentials for every game. O'Reilly Media., ISBN: 9781491999158

[Lee16] Lee, J. (2016). Learning unreal engine game development. Packt Publishing Ltd., ISBN: 9781784398156

[Tavakkoli18] Tavakkoli, A. (2018). Game Development and Simulation with Unreal Technology. CRC Press., ISBN-13: 978-1498706247

[Romero19] Romero, M., & Sewell, B. (2019). Blueprints Visual Scripting for Unreal Engine: The faster way to build games using UE4 Blueprints. Packt Publishing Ltd., ISBN: 9781789347067

[Chandler11] Chandler, H. (2011). The Game Localization Handbook 2nd Edition, Jones & Bartlett Learning, ISBN: 0763795933

[Retsker81] Retsker, Ya. I. (1981). Textbook for translation from English into Russian. M.: Prosveschenie. (In Russian).

8.4 Links (Web/Internet)

Note: All references are current as of April 28, 2022.

- [ISTQB-Web] <https://www.istqb.org/>
- [URL1] <https://research.ncl.ac.uk/game/mastersdegree/workshops/technicalrequirementschecklists/Technical%20Requirements%20Checklist%20Workshop.pdf>
- [URL2] https://docs.microsoft.com/en-us/gaming/gdk/_content/gc/live/get-started/live-xbl-overview
- [URL3] <https://gamepad-tester.com/>
- [URL4] <https://igda-website.s3.us-east-2.amazonaws.com/wp-content/uploads/2021/04/09142137/Best-Practices-for-Game-Localization-v22.pdf>